Some notes for the course

INFORMATION AND CODING

Armando J. Pinho

DRAFT

***Important note**: This document contains a summary of **some of the topics** addressed in the course "Information and Coding". As such, it should be regarded as a guide and a complement, and by no means the only source of information nor a substitute of the classes.*

(Draft version of September 13, 2026)

Contents

1 Some concepts of digital signal processing **7**

1.1 Signals . . . 7

1.1.1 Some characteristics of the signals . . . 9

1.1.2 Analog-to-digital conversion . . . 10

1.1.3 Multi-dimensional signals . . . 15

1.1.4 Some elementary signals . . . 17

1.1.5 Operations on the independent variable . . . 19

1.2 Systems . . . 20

1.2.1 Properties of systems: Stability . . . 22

1.2.2 Properties of systems: Memory . . . 24

1.2.3 Properties of systems: Causality . . . 24

1.2.4 Properties of systems: Invertibility . . . 25

1.2.5 Properties of systems: Linearity . . . 25

1.2.6 Properties of systems: Time invariance . . . 26

1.3 Linear and time-invariant systems . . . 26

1.3.1 Signal representation by superposition of impulses . . . 26

1.3.2 Impulse response . . . 27

1.3.3 Properties of the convolution operation . . . 29

1.3.4 The impulse response and some properties of systems . . . 30

1.3.5 Response to complex exponentials . . . 32

1.4 The discrete Fourier transform (DFT) and the frequency domain . . . 34

1.4.1 Some properties of the discrete Fourier transform . . . 36

1.4.2 The discrete cosine transform . . . 38

1.5 The Z transform and digital filters . . . 39

1.5.1 The time-shift property . . . 40

1.5.2 System stability . . . 43

1.5.3 Frequency response . . . 43

2 Data, information and some related problems **47**

3 A combinatorial measure of information **51**

4 A probabilistic measure of information **55**
4.1 About codes . . . 56
4.2 Definition of information according to Shannon . . . 61
4.3 Source modeling . . . 64
4.4 Some more on Shannon entropy . . . 70
4.4.1 Entropy, joint entropy and conditional entropy . . . 70
4.4.2 Relative entropy . . . 73
4.4.3 Mutual information . . . 74

5 Data compression **81**
5.1 Variable-length coding . . . 81
5.1.1 Optimal codes . . . 81
5.1.2 A counting argument . . . 83
5.1.3 Huffman codes . . . 83
5.1.4 Shannon-Fano codes . . . 85
5.1.5 Alphabet extension . . . 86
5.1.6 Some other variable-length codes . . . 88
5.1.7 Golomb code . . . 90
5.2 Dictionary based compression . . . 91
5.2.1 Tunstall codes . . . 92
5.2.2 String parsing . . . 95
5.2.3 LZ77 compression . . . 95
5.2.4 LZSS compression . . . 98
5.2.5 LZ78 compression . . . 98
5.2.6 LZW compression . . . 100
5.3 Arithmetic coding . . . 103
5.3.1 Motivation . . . 103
5.3.2 Encoding . . . 103
5.3.3 Decoding . . . 105
5.3.4 Implementation issues . . . 106

6 Predictive coding **109**
6.1 Principles . . . 109

6.2 Predictive coding techniques 110
6.2.1 Simple 1D prediction 110
6.2.2 Predictive image coding techniques 110
6.2.3 Predictors 110
6.3 Lossless predictive coding 112
6.3.1 Linear prediction: the lossless mode of JPEG 113
6.3.2 The nonlinear predictor of JPEG-LS 114
6.4 Temporal prediction and motion compensation 114

7 An algorithmic measure of information **119**
7.1 Motivation 119
7.2 Turing machines 120
7.3 Kolmogorov complexity (or algorithmic entropy) 125

References **129**

1 Some concepts of digital signal processing

1.1 Signals

In very general terms, a **signal** is a representation of a certain quantity that varies over time and/or space or other variables of interest. For example, the voice of a person can be represented as a time signal that measures changes in air pressure.

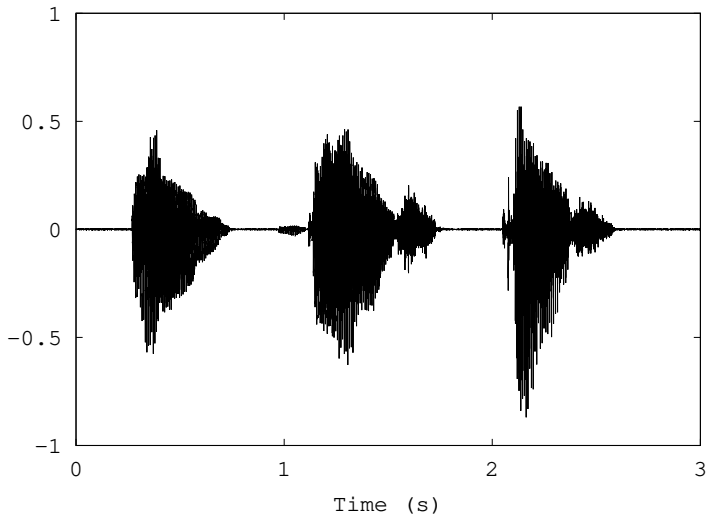


Figure 1.1: Audio signal corresponding to the portuguese words “um dois três” (“one two three”).

This pressure signal can be picked up by a microphone, converted into an electrical signal (audio signal), amplified and sent to a speaker, which transforms the audio back into sound signals. Figure 1.1 shows a small example of an audio signal, with a duration of 3 seconds, corresponding to the portuguese words “um dois três” (“one two three”).

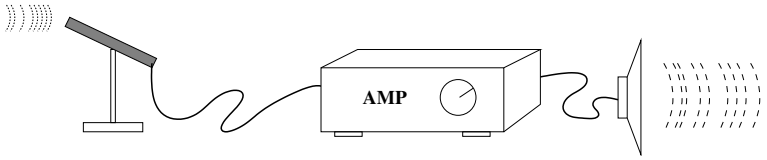


Figure 1.2: Sound capture, amplification and reproduction system.

Signals are transformed by **systems**. In the previous example, we can clearly identify three systems (Fig. 1.2): a microphone (converts audible variations of air pressure into electrical signals), an amplifier (increases the energy of the signal picked up by the microphone) and a speaker (converts the amplified audio signal into sound waves).

In Fig. 1.3, we can see another example of a system, as well as signals of various types, namely:

pressure, electrical, optical and electromagnetic. In fact, the concept of signal is so vast, that any quantity dependent on one or more variables can be considered a signal.

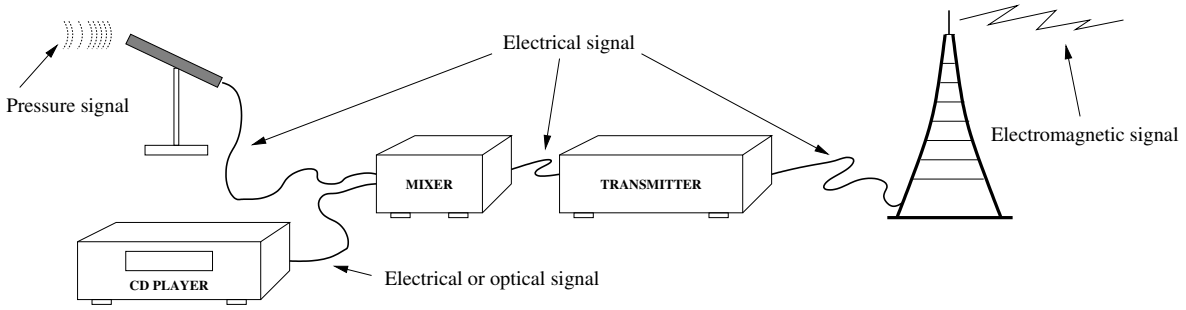


Figure 1.3: Audio broadcasting system.

Although a signal can depend on different variables, it is common to use the variable time when we want to represent a generic signal. In this case, the signal is represented, for example, by $x(t)$. Therefore, the signal shown in Fig. 1.1 could be represented, for mathematical manipulation purposes, by $x(t)$. So, we could say that the value of the signal at instant t_0 is $x(t = t_0)$ or, more simply, $x(t_0)$. Although this last notation can be confused with the representation of a complete signal, in general the context takes care to clarify its meaning.

A signal can have **scalar** or **vector**, **real** or **complex** values. In the example shown, the value of $x(t)$ could represent, for example, the electrical voltage at the output of the microphone, in which case we have $x(t) \in \mathbb{R}$. Since both t and $x(t)$ have values in $\mathbb{R}$, we say that $x(t)$ is an **analog** signal, that is, both the time variable and the signal value vary continuously. However, there are signals that only have values defined at certain instants of time, t_i. These are called **discrete-time** signals. For example, the maximum daily temperatures over a year form a discrete-time signal, where the t_i are the days of the year. In this case, saying that $x(129)$ is equal to 34.8 corresponds to saying that on the 129th day of the year the maximum temperature was 34.8 degrees.

A signal can also have **discrete values**. Take, for example, the number of people who are in a queue at a service counter. In this case, $x(t) \in \mathbb{N}_0$, that is, this signal has discrete values.

In summary, we have the following four combinations:

- **Signals continuous both in time and amplitude**
  $x(t) \in \mathbb{R}$ or $\mathbb{C}, t \in \mathbb{R}$
  Example: voltage at the output of a microphone
- **Signals continuous in time and discrete in amplitude**
  $x(t) \in \{\dots, x_{i-1}, x_i, x_{i+1}, \dots\} \subset \mathbb{R}$ or $\mathbb{C}, t \in \mathbb{R}$
  Example: number of people in a queue
- **Discrete-time and continuous-amplitude signals**
  $x(n) \in \mathbb{R}$ or $\mathbb{C}, n \in \mathbb{Z}$
  Example: maximum daily temperature

- **Signals discrete in time and amplitude**
  $x(n) \in \{\dots, x_{i-1}, x_i, x_{i+1}, \dots\} \subset \mathbb{R}$ or $\mathbb{C}, n \in \mathbb{Z}$
  Example: number of people served per hour by a service

Signals of the last type (discrete in time and amplitude) are particularly interesting to deal with a computer. On the one hand, a computer has discrete memory cells, which can be used to store values of a signal at certain instants of time (discrete-time). On the other hand, due to the finite precision with which values can be stored in a computer, it is very convenient that the signal can only have values within a discrete and finite set (discrete-amplitude). Because signals that are discrete in time and amplitude are the most suitable for processing in a digital computer, we call them **digital signals**.

We also consider signals with values that are not numeric. We call them **symbolic signals**. In general, the values of these discrete-time sequences (some times called "categorical time series") do not have a mathematical structure, besides being members of some set. In fact, most of the data compression methods that we will address later assume that the signals are symbolic. However, note that although all (numeric) digital signals can be seen as symbolic signals, the converse is not always true.

1.1.1 Some characteristics of the signals

In addition to the continuous-time / discrete-time, continuous-amplitude / discrete-amplitude and numeric / symbolic characteristics, a signal can also be classified according to several other properties, including

- Whether it is deterministic or stochastic
- In the case of being deterministic, whether or not it is periodic
- In the case of being stochastic, whether or not it is stationary
- Whether it is finite-time or infinite-time

The **deterministic signals** are generated through algorithms that do not have randomness and, therefore, are predictable. For example, the signal $x(t) = \sin(t)$ is deterministic. This means that, for example, the graph of this signal for $0 \leq t \leq 4\pi$ is always the same, as shown in Fig. 1.4.

On the other hand, **stochastic signals** are generated by systems that have randomness, that is, whose value at a certain instant of time cannot be exactly determined *a priori*. The output audio signal from a microphone is an example of a stochastic signal. In fact, despite many times the same word is repeated in front of a microphone, the generated signal will always be different. However, even though, we are able to understand the word, meaning that some features of these signals should be kept constant and, therefore, may be used, for example, to perform voice recognition.

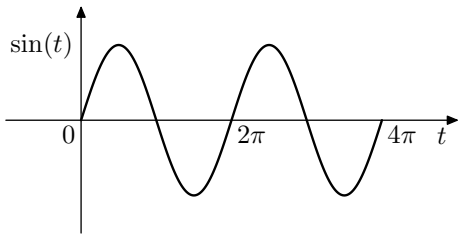


Figure 1.4: Example of a deterministic signal.

A deterministic signal can be periodic, i.e., it can repeat itself exactly after a certain time interval, called the **period**. In the case of continuous-time signals, we have $x(t) = x(t+T), \forall t$, where T is the period. Obviously, if $x(t)$ is periodic with period T, then it is also periodic for periods kT with $k \in \mathbb{N}$. The smallest value of T, for which the relation $x(t) = x(t+T), \forall t$, is valid, is called the **fundamental period**. Unless otherwise indicated, whenever we refer to the period of a signal we are referring to its fundamental period. For discrete-time, the period is measured in number of samples, N, i.e., a signal $x(n)$ is periodic with period N if $x(n) = x(n+N), \forall n$.

A stochastic signal can be **stationary**, which means that its statistical characterization is maintained over time.[1] For example, this could indicate that the mean and variance of the signal remain constant over time. However, most interesting signals are not stationary. The human voice is one such example. Nevertheless, when analyzed in sufficiently small time intervals, we can often consider that stationarity is verified.

A signal is of **finite duration** if it starts at a certain point in time $t_i > -\infty$ and ends at another point in time, t_f, such that $t_i < t_f < +\infty$. In case this does not happen, we say that the signal is of **infinite duration**.

1.1.2 Analog-to-digital conversion

There are many signals that we would like to be able to process on a computer, but they are not digital signals. See the example of the audio signal picked up by a microphone. In this case, we can resort to the **analog-to-digital conversion** to transform continuous-time and continuous-amplitude signals into digital signals. Analog-to-digital conversion involves two operations: **sampling** and **quantization**.

The **sampling operation** is responsible for the passage from continuous-time to discrete-time. For this purpose, signal values are collected at certain time points. Typically, these instants of time are regularly spaced from T_s to T_s seconds. We call T_s the **sampling period** (Fig. 1.5), and $f_s = 1/T_s$ the **sampling frequency**. Therefore, when we say that a signal is sampled at 500 Hz, we are indicating that 500 samples are taken for every second of the signal.

After the $x(t)$ signal has been sampled from T_s in T_s seconds, we have a sequence of values that correspond to the values of $x(t)$ at those instants of time, i.e., we have the sequence of

[1] There are strict ways to define stationarity, but they are beyond what we want to present here.

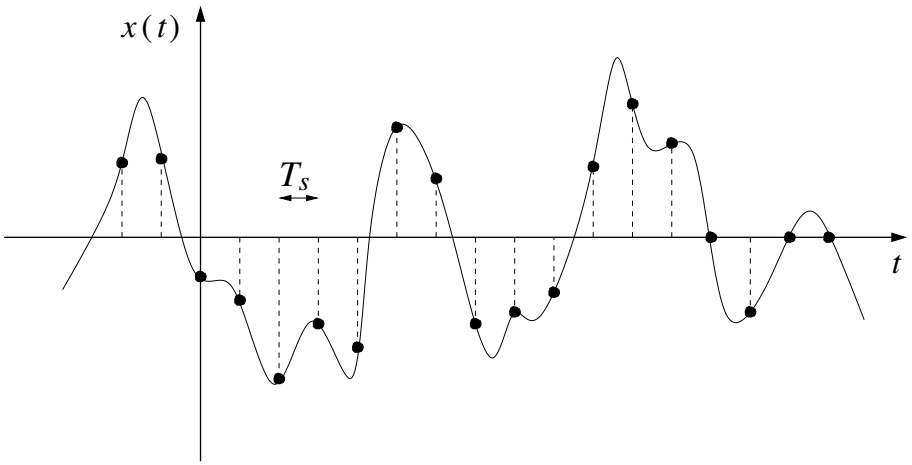


Figure 1.5: Signal sampling.

values

$$\ldots, x(-2T_s), x(-T_s), x(0), x(T_s), x(2T_s), \ldots, x(nT_s), \ldots,$$

which we represent by $x(nT_s)$ or simply by $x(n)$.[2]

Depending on certain properties of $x(t)$, it is possible to find values of T_s for which the sequence $x(nT_s)$ fully describes $x(t)$. In other words, in that case it will be possible to reconstruct exactly $x(t)$ based on the samples $x(nT_s)$.

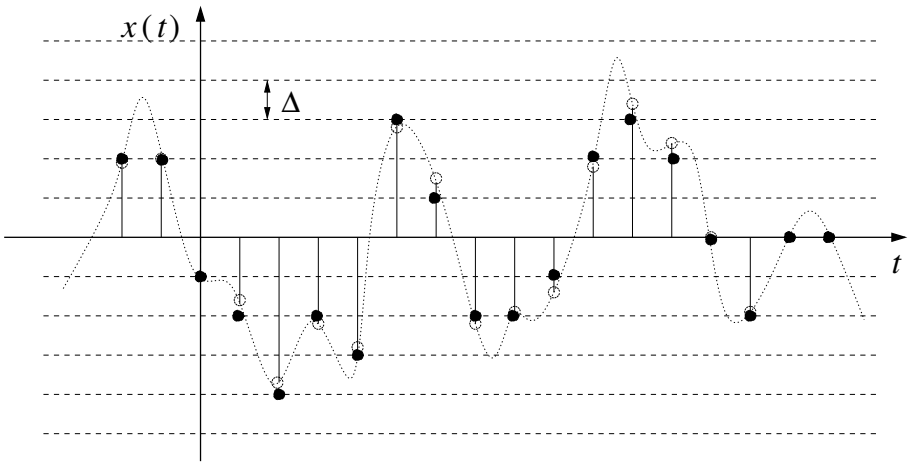


Figure 1.6: Signal quantization.

The **quantization operation** is responsible for converting a continuous-amplitude signal into

[2]By representing the sequence $x(nT_s)$ by $x(n)$ we are simplifying the notation, but we also introduce some loss of rigor. In fact, $x(n)$ designates any sequence of values, indexed by the integer variable n, which may or may not correspond to a temporal sequence and, even then, may not correspond to evenly spaced samples. However, we believe that the simplification of the notation justifies this inaccuracy, although whenever there may be doubt, we will opt for the complete notation.

a discrete-amplitude one, by restricting the number of possible different values that the signal may have (represented by the horizontal lines in Fig. 1.6). Quantization introduces errors (or **noise**, according to signal processing terminology) in the sample values. Unlike sampling, which is possible without introducing an error, it is not possible to do quantization without introducing irreversible errors. However, it is possible to calculate maximum values for the error made and reduce it as much as desired. One of the most common measures to express the introduced error is the **signal-to-noise ratio**, which is defined by the expression

$$\mathrm{SNR} = 10 \log_{10} \frac{E_x}{E_r} \quad \mathrm{dB} \ \ (\mathrm{decibel}), \tag{1.1}$$

where E_x and E_r indicate, respectively, the **energy**[3] of the signal and of the noise, being the energy of a discrete-time signal $x(n)$, E_x, calculated through

$$E_x = \sum_{n=-\infty}^{\infty} |x(n)|^2. \tag{1.2}$$

Note that, according to (1.2), it may not be possible to calculate the energy of a certain signal. This happens if the respective summation does not converge. In fact, there are many signals of infinite energy, an obvious example being the class of periodic signals. In this case, we can calculate the **power** of the signal, i.e., its energy normalized per unit of length (or, as we usually call it, time). The power of a discrete-time signal is given by

$$P_x = \lim_{N \to \infty} \frac{1}{2N+1} \sum_{n=-N}^{N} |x(n)|^2. \tag{1.3}$$

Note that, according to the definitions just presented, a finite-energy signal has zero power. These signals are called **energy signals**. On the other hand, a signal with non-zero power necessarily has infinite energy, and is called a **power signal**.

In the case of power signals, the signal-to-noise ratio defined in (1.1) changes to the form

$$\mathrm{SNR} = 10 \log_{10} \frac{P_x}{P_r} \quad \mathrm{dB}, \tag{1.4}$$

where P_x and P_r indicate, respectively, the power of the signal and of the noise.

The calculation of the energy according to (1.2) or of the power according to (1.3), applies to the cases where the signal is deterministic, or else when it is an instance of a stochastic signal. However, we may also be interested in determining the "average" signal-to-noise ratio associated with a digitalization process of a certain class of stochastic signals. In this case, we will have to use probabilistic tools and, in particular, the expected value (or mean value) of a random variable, X, which is given by

$$E[X] = \int_{-\infty}^{\infty} x f_X(x)\, dx, \tag{1.5}$$

[3] In signal processing terms, the energy of a signal measures, in a certain sense, the "size" of the signal. Contrarily to the physics notion of energy, this one does not have units.

where $f_X(x)$ represents the probability density function of X.[4] Then, the power of a stochastic signal is obtained by calculating

$$P_x = E[X^2] = \int_{-\infty}^{\infty} x^2 f_X(x)\, dx. \tag{1.6}$$

Let us then assume that a signal, uniformly distributed over the interval $[-A/2, A/2]$,[5] is processed by a quantizer with 2^b levels. If the quantization levels are evenly spaced, then this spacing will be $\Delta = A/2^b$ and the maximum quantization error will be $r_{\max} = \Delta/2$ (Fig. 1.7 shows an example with 4 levels).

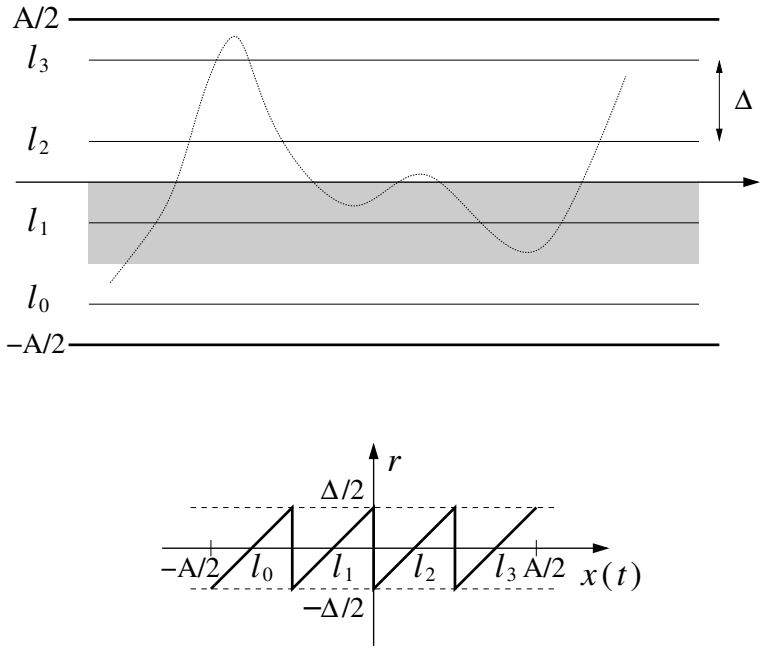


Figure 1.7: Example of quantization to four levels: l_0, l_1, l_2, l_3. Note that all signal values that occur within the shaded range will be represented by the value l_1.

Since the signal is uniformly distributed, we have

$$f_X(x) = \frac{1}{A}$$

and also

$$f_R(r) = \frac{1}{\Delta},$$

since the random variable associated with the quantization error also has a uniform distribution.

[4] If a random variable X is characterized by a probability density function $f_X(x)$, then the probability that X has values between a and b is given by $\int_a^b f_X(x)\, dx$.

[5] We say that a random variable, X, is uniformly distributed over the interval $S_X = [-A/2, A/2]$, if the probability that it has values in a certain sub-interval of S_X of length $L \leq A$ is L/A.

So, the power of the quantization noise is

$$P_r = E[R^2] = \int_{-\Delta/2}^{\Delta/2} r^2 \frac{1}{\Delta} dr = \frac{\Delta^2}{12}$$

and the signal power is

$$P_x = E[X^2] = \int_{-A/2}^{A/2} x^2 \frac{1}{A} dx = \frac{A^2}{12}.$$

Because

$$\frac{A}{\Delta} = 2^b,$$

we have

$$\text{SNR} = 10 \log_{10} \frac{P_x}{P_r} = 10 \log_{10} \frac{A^2}{\Delta^2} = 20 \log_{10} 2^b \approx 6.02b \quad \text{dB}. \tag{1.7}$$

The relation (1.7) is well known and is generally used to estimate the signal-to-noise ratio obtained when representing each sample of the signal with b bits. As we can see, for each additional bit, the signal-to-noise ratio increases by approximately 6 dB. However, we cannot forget that this relation was derived assuming the uniform distribution (and zero mean) of the signal to be quantized, a characteristic that is often not verified.

Example 1.1

Let a signal, $x(t)$, be sampled at 1 kHz, which samples are to be stored in 6 bits. Considering that $x(t)$ has values between -1 and 1, we intend to determine an expression for the levels of a uniform quantizer, in order to minimize the maximum error. We also want to know what will be the generated binary data rate.

The spacing between quantizer levels is $\Delta = \frac{2}{2^6} = \frac{1}{2^5}$. Therefore, the 64 levels must be positioned at

$$l_k = -1 + \frac{\Delta}{2} + k\Delta = -1 + \frac{1}{2^6} + \frac{k}{2^5}, \quad k = 0, 1, \ldots, 63$$

so that the maximum error is minimized. In this case, the maximum error is $\frac{\Delta}{2} = \frac{1}{2^6}$.

Since 1000 samples are generated per second and each sample is stored in 6 bits, then the binary rate generated is $6 \times 1000 = 6$ kbps (kilobits per second).

Note that, generically, if the signal to quantize has a variation between $A_{\min}$ and $A_{\max}$, then the levels of the uniform quantizer are given by

$$l_k = A_{\min} + \frac{\Delta}{2} + k\Delta, \quad k = 0, 1, \ldots, 2^b - 1,$$

where

$$\Delta = \frac{A_{\max} - A_{\min}}{2^b}$$

and b is the number of bits used to represent each sample.

Example 1.2
The audio signal stored in a CD is sampled at 44.1 *kHz, each sample being represented with* 16 *bits. It is intended to estimate the signal-to-noise ratio resulting from the uniform quantization process applied to the audio signal, and also calculate the bit rate at the output of a CD player.*

If the audio signal were evenly distributed, we would have a signal-to-noise ratio equal to $16 \times 6 = 96$ *dB. However, as in general the distribution of an audio signal is not uniform, this value should be understood merely as an estimate.*

The binary rate at the output of the CD player is given by $44\,100 \times 16 \times 2 = 1\,411\,200$ *bps (bits per second), i.e., approximately equal to* 1.4 *Mbps (megabits per second). Note that the "*$\times 2$*" factor results from the fact that CD audio is stereo and, therefore, there are two channels.*

1.1.3 Multi-dimensional signals

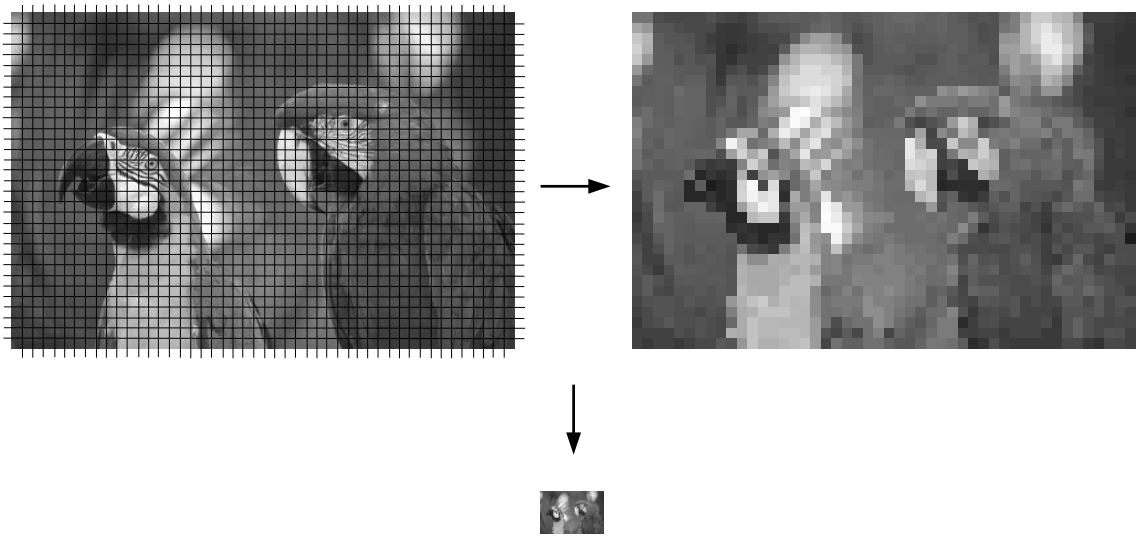
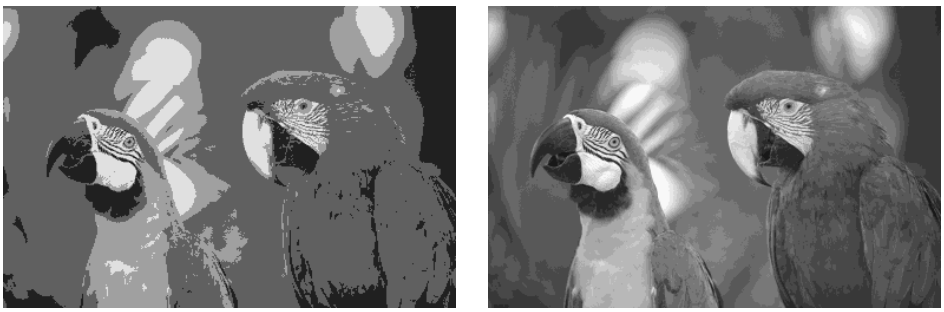


Figure 1.8: Image sampling.

We are daily confronted with information carried in the form of signals with two dimensions (images) and three dimensions (video). An image can be seen as a two-dimensional signal, $f(x, y)$, where x and y represent spatial coordinates. The value and meaning of $f(x, y)$ at a given point in space, (x, y), depends on the source responsible for producing the image. However, it is generally assumed that $f(x, y) \geq 0$ and that both (x, y) and $f(x, y)$ assume continuous values. Therefore, to convert $f(x, y)$ into a digital image, it is necessary, on the one hand, to perform a spatial sampling and, on the other hand, to quantize the amplitude of each resulting sample. Fig. 1.8 illustrates the process of sampling an image, whereas Fig. 1.9 shows the effect of quantization.



2 bits/pixel 4 bits/pixel



6 bits/pixel 8 bits/pixel (original)

Figure 1.9: Quantization effects on images.

A digital image is represented by a rectangular matrix of scalars or vectors, $f(i,j)$, which we call picture elements or *pixels*. In general, we have $f(i,j) \in \mathcal{I} \subset \mathbb{N}_0^n$, i.e., each pixel is a non-negative integer or a vector of non-negative integers.

The most common types of images are (Fig. 1.11 presents an example of each of these four types):

- **Black and white or binary**
  $f(i,j) \in \{0,1\}$
- **Gray level**
  $f(i,j) \in \{0,1,\ldots,2^b-1\}$
- **Indexed color (indexes in a color map)**
  $f(i,j) \in \{0,1,\ldots,2^b-1\} \xrightarrow{\alpha} \mathcal{I} \subset \{0,1,\ldots,2^{b'}-1\}^3$
- **Color (for example, RGB)**
  $f(i,j) \in \{0,1,\ldots,2^b-1\}^3$

To a temporal sequence of digital images we call **digital video**, which we represent by $v(i,j,n)$ or by $v(i,j,nT_s)$ if we want to evidence the sampling period over time. In this case, the period T_s is the time that elapses between the presentation of two consecutive video frames. The inverse of T_s is the **frame rate**.

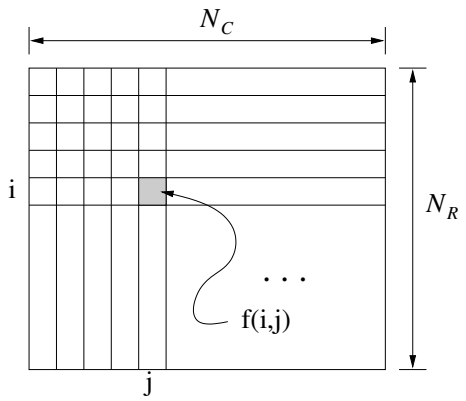


Figure 1.10: Digital image, where N_C and N_R respectively designate the number of columns and rows of the image.

For each value of n there corresponds a digital image, that is, we can write $f(i,j) = v(i,j,k), k =$ constant, which we denote by **frame** k of the video sequence v.

1.1.4 Some elementary signals

Although most real-life signals have complex shapes, simple signals are often used to study systems. There are several reasons for this approach, among them the ease of doing mathematical manipulations, the fact that some of these simple signals approximate some real-life signals relatively well, and probably the most important reason, because it is possible to represent complicated real-life signals by combining simple signals.

One of these elementary signals is the **unit step** (Fig. 1.12), defined as

$$u(n) = \begin{cases} 0, & n < 0 \\ 1, & n \geq 0 \end{cases} . \tag{1.8}$$

The **unit impulse** (Fig. 1.13) is another important elementary signal, and is defined through

$$\delta(n) = \begin{cases} 0, & n \neq 0 \\ 1, & n = 0 \end{cases} . \tag{1.9}$$

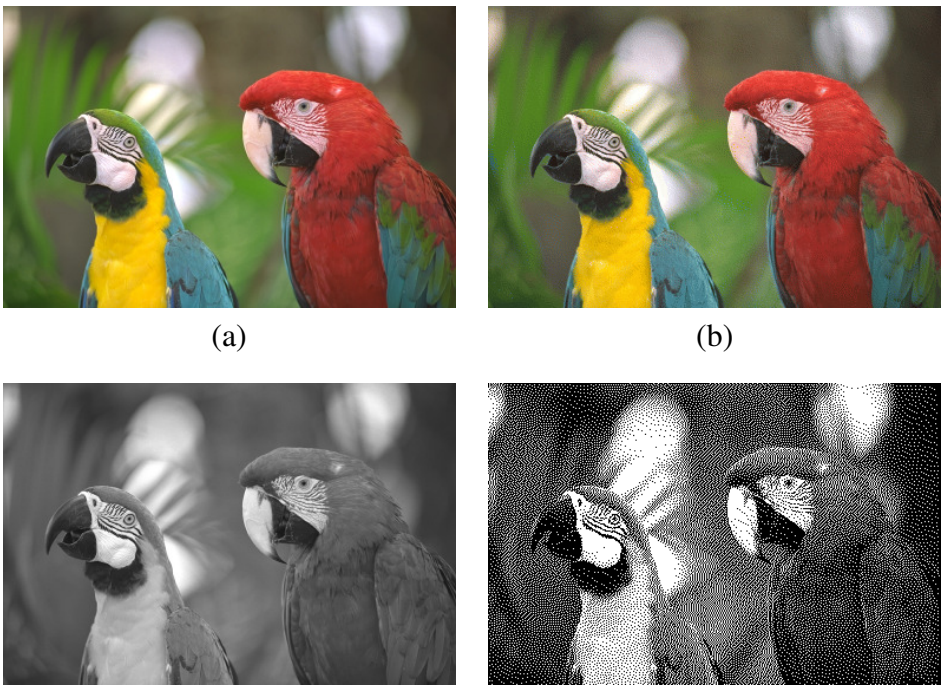
The **exponential** signals,

$$x(n) = a^n, \quad a \in \mathbb{R}, \tag{1.10}$$

also play an important role in signal processing (recall what happens, in terms of the evolution of $x(n)$, as a function of a).

The **sinusoidal** signals are also very important in signal processing. These are related to the **complex exponentials**, since, by the Euler relation, we have

$$e^{\mathrm{j}\theta} = \cos\theta + \mathrm{j}\sin\theta, \tag{1.11}$$



(a) (b)

(c) (d)

Figure 1.11: Examples of common digital images: (a) color; (b) indexed color (256 colors); (c) gray level (256 levels); (d) binary (2 levels).

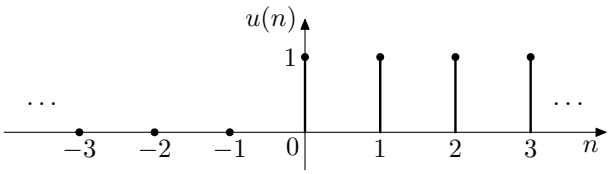


Figure 1.12: Discrete-time unit step.

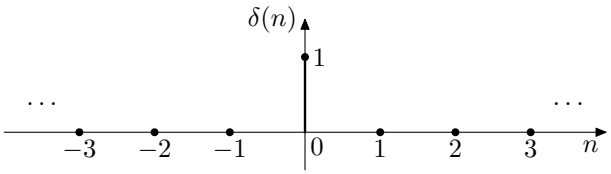


Figure 1.13: Discrete-time unit impulse.

where $\mathtt{j} = \sqrt{-1}$ is the imaginary unit.

From this result, two very important relations can be drawn, namely,

$$\cos\theta = \frac{e^{\mathtt{j}\theta} + e^{-\mathtt{j}\theta}}{2} \tag{1.12}$$

and

$$\sin\theta = \frac{e^{\mathtt{j}\theta} - e^{-\mathtt{j}\theta}}{2\mathtt{j}}. \tag{1.13}$$

In continuous-time, a sinusoidal signal is represented by

$$x(t) = \sin(2\pi f t) = \sin(\omega t), \tag{1.14}$$

where f and ω represent the sine frequency, respectively, in Hz and rad/s. Regarding the discrete-time case, we have

$$x(n) = \sin(2\pi f n T_s) = \sin\left(2\pi \frac{f}{f_s} n\right) = \sin(2\pi F n) = \sin(\Omega n), \tag{1.15}$$

where T_s and f_s represent, respectively, the sampling period and the sampling frequency. Note that $F = f/f_s$ and $\Omega = 2\pi f/f_s$ represent **normalized frequencies**, because its physical meaning depends on the sampling frequency used. Their units are, respectively, cycles per sample and radians per sample. For example, if $F = 0.25$ cycles per sample and $f_s = 1000$ Hz, then it represents the physical frequency of $1000 \times 0.25 = 250$ Hz. But, if $f_s = 500$ Hz, then the same $F = 0.25$ now represents the physical frequency of $500 \times 0.25 = 125$ Hz. This property has very important implications on how discrete-time systems are designed and analysed.

Also, note that sampling a periodic signal with period T does not necessarily result in a periodic signal with the same period (in fact, the resulting signal may not be even periodic). Also note that

$$\sin\left(2\pi \frac{f}{f_s} n\right) = \sin\left(2\pi \frac{f + k f_s}{f_s} n\right), k \in \mathbb{N}_0. \tag{1.16}$$

(Show why and try to discover some implications of this property).

1.1.5 Operations on the independent variable

There are several operations in signal processing that can be described through modifications or transformations of the independent variable, t or n. Among them are time inversion, time delay and advancement, time compression and expansion.

Let us consider the signal

$$y(t) = x(-t). \tag{1.17}$$

The $y(t)$ signal consists of the **time inversion** of the $x(t)$ signal. This is the effect that happens, for example, when we play a recording in the opposite direction. The equivalent for the digital case is obvious: if the original signal is $x(n)$, then the time-inverted signal is $x(-n)$.

Let us now consider the signal

$$y(t) = x(t - \tau). \tag{1.18}$$

Note that the signal $y(t)$ has a value equal to $x(0)$ for $t = \tau$. Therefore, if $\tau > 0$, this value happens temporally after $t = 0$. Hence, the signal $y(t)$ is said to be a **delayed** version of the

signal $x(t)$. In the case where $\tau < 0$, the signal $y(t)$ displays the value $x(0)$ before the instant $t = 0$, so in this situation the signal $y(t))$ is an **advanced** version of $x(t)$. Here, too, the adaptation to the discrete-time situation is obvious,

$$y(n) = x(n - k), \quad k \in \mathbb{Z}. \tag{1.19}$$

Consider now the signal

$$y(t) = x(\alpha t), \quad \alpha \in \mathbb{R}. \tag{1.20}$$

The $y(t)$ signal is a version of $x(t)$ that is "faster" (if $|\alpha| > 1$) or "slower" (if $|\alpha| < 1$), thus obtaining **temporal compression** in the first situation and **temporal expansion** in the second case. As an illustrative practical example of these operations, we can point out the case in which a recording is reproduced at double the speed (temporal compression) or at half the original speed (temporal expansion).

Any linear transformation of the independent variable can be expressed by combining the operations just described (time inversion, time expansion or compression, and delay or advance). For example, the signal $x(\alpha t - \tau)$ results from applying a delay (advance if τ is negative) of τ time units, followed by a compression (expansion if $|\alpha| < 1$) and even an inversion if $\alpha < 0$. Fig. 1.14 presents examples of the various operations on the independent variable, as well as combinations between them.

1.2 Systems

In the context of signal processing, a system is a device, process or algorithm that, given an input signal $x(n)$, produces an output signal $y(n)$. We express the action of a system, H, on a signal, $x(n)$, by writing

$$y(n) = H[x(n)]. \tag{1.21}$$

Fig. 1.15(a) presents a graphical representation of a generic system with N inputs and M outputs. Our attention will, however, go to systems with one input and one output, as is the case with the system illustrated in Fig. 1.15(b).

One of the simplest systems to describe mathematically is the **ideal amplifier**,

$$y(n) = H[x(n)] = \alpha x(n), \quad \alpha > 0.$$

The constant α is called the **gain** of the amplifier. If $0 < \alpha < 1$, the system works as an attenuator, i.e., the amplitude of the output signal will be smaller than the amplitude of the input signal. If $\alpha > 1$, the system will amplify the input signal, i.e., the amplitude of the output signal will be greater than the amplitude of the input signal. Fig. 1.16 shows the relation between the input signal, $x(n)$, and the output signal, $y(n)$, of an amplifier with gain α.

The **ideal half-wave rectifier** is also an easy system to represent mathematically,

$$y(n) = H[x(n)] = \begin{cases} x(n), & x(n) \geq 0 \\ 0, & x(n) < 0 \end{cases}, \tag{1.22}$$

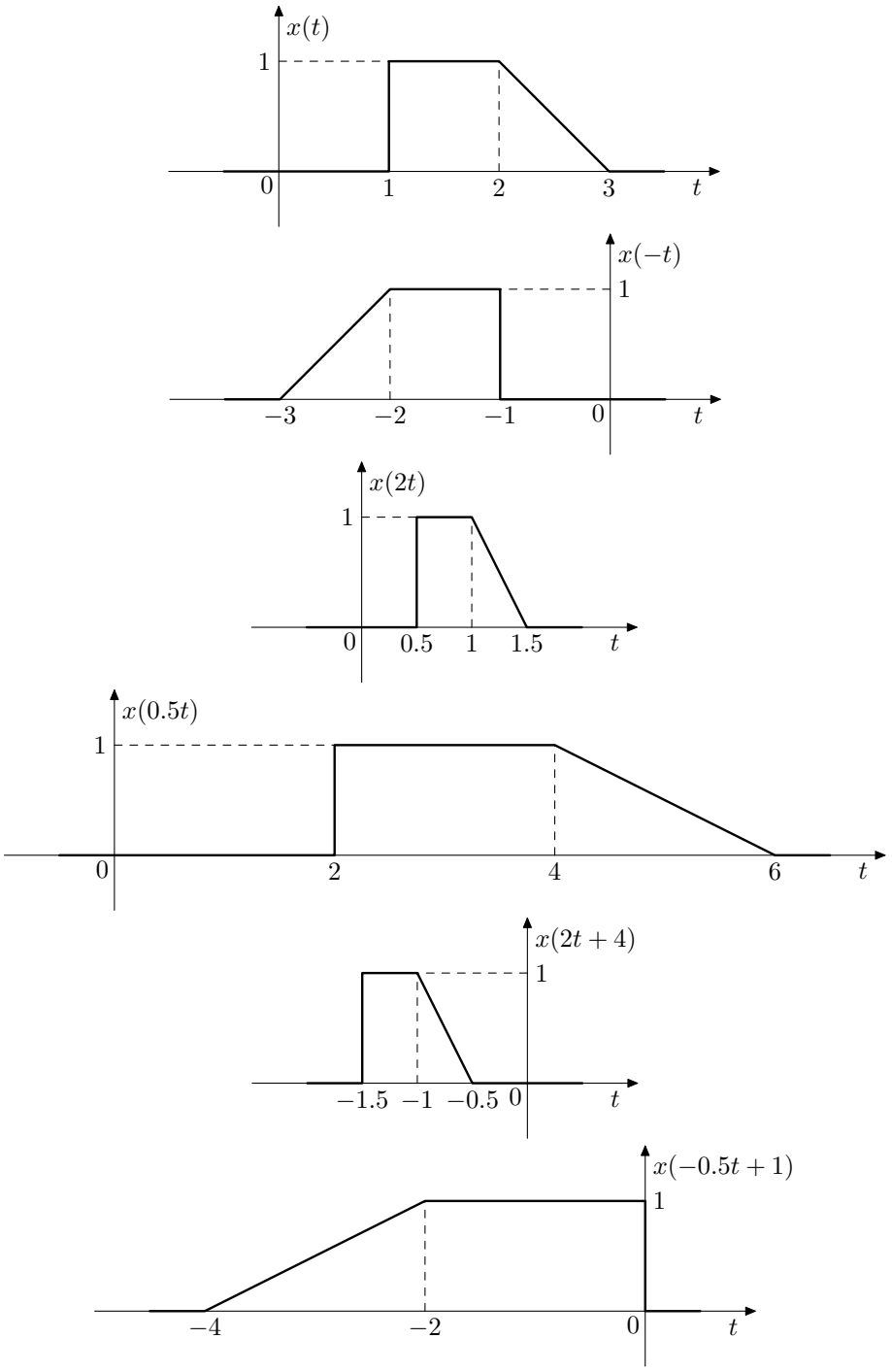


Figure 1.14: Example of the various operations on the independent variable.

as well as the **ideal full-wave rectifier**,

$$y(n) = H[x(n)] = |x(n)|. \tag{1.23}$$

Fig. 1.17 shows the relation between the input and output of these two systems. Fig. 1.18 shows

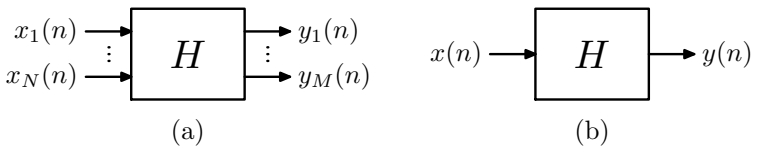


Figure 1.15: System: (a) with N inputs and M outputs; (b) with an input and an output.

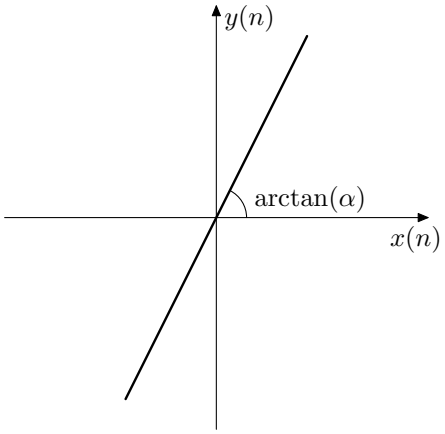


Figure 1.16: Relation between the input and output of an ideal amplifier with gain α.

an example of half-wave and full-wave rectification.

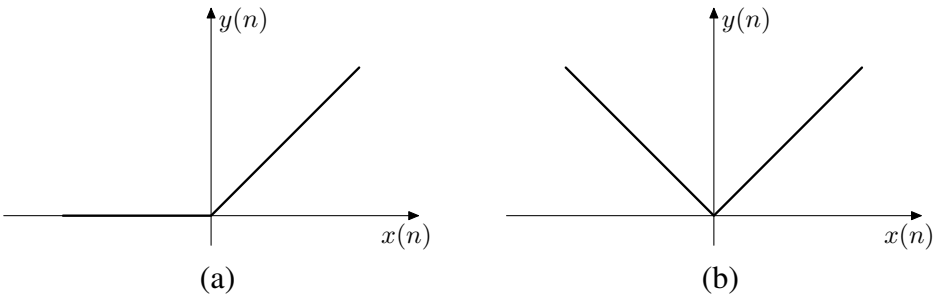


Figure 1.17: Relation between input and output of: (a) an ideal rectifier half-wave; (b) an ideal full-wave rectifier.

Another example of a system that is simple to describe mathematically, although of great importance, is the **variable gain amplifier**,

$$y(n) = H[x(n)] = \alpha(n)x(n). \tag{1.24}$$

1.2.1 Properties of systems: Stability

A system is stable if, for any amplitude-limited input signal, it produces an amplitude-limited output signal, that is, if

$$|y(n)| \leq M_y < \infty, \forall x(n) : |x(n)| \leq M_x < \infty.$$

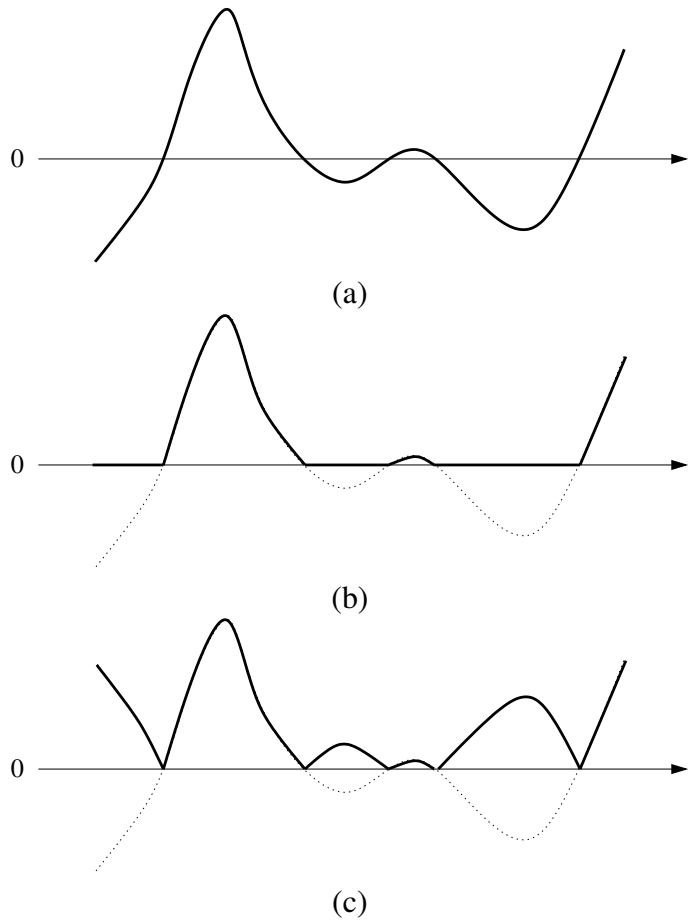


Figure 1.18: Example of rectification of a signal: (a) original signal; (b) half-wave rectification; (c) full-wave rectification.

For example, the ideal amplifier is a stable system, because if

$$|x(n)| \leq M_x < \infty,$$

then

$$|y(n)| = |\alpha x(n)| = |\alpha||x(n)| \leq |\alpha| M_x < \infty, \quad \text{for} \quad |\alpha| < \infty.$$

On the other hand, the system given by

$$y(n) = 2y(n-1) + x(n)$$

is not stable. In fact, if we consider an impulse as the input of the system, i.e., making $x(n) =$

$\delta(n)$, we will have the following sequence of output values (we assume $y(n) = 0, n < 0$):

$$\begin{aligned} &y(0) = 2y(-1) + \delta(0) = 0 + 1 = 1 \\ &y(1) = 2y(0) + \delta(1) = 2 + 0 = 2 \\ &y(2) = 2y(1) + \delta(2) = 4 + 0 = 4 \\ &\dots \\ &y(k) = 2^k \\ &\dots \end{aligned}$$

So, for an amplitude-limited input, we get a response that grows without limit, so the system is unstable.

1.2.2 Properties of systems: Memory

A system has memory when, to determine its output at a given instant n, other input values than $x(n)$ are needed. For example, the ideal amplifier has no memory, since to obtain the output value at time n, i.e., $y(n)$, it is only necessary to know the input value at time n, i.e., $x(n)$.

On the other hand, the system defined by the relation $y(n) = 2y(n-1) + x(n)$ has memory. In fact, to determine the value of $y(n)$, in addition to the value of $x(n)$, the output value at the previous instant is also necessary, i.e., $y(n-1)$.

1.2.3 Properties of systems: Causality

A system is causal if, to calculate its output at a given instant n, only current or past values are needed. Therefore, a causal system does not need to know the future to determine the output at a given moment, which, physically, seems quite obvious. However, under certain conditions, we can actually have systems that are not causal. Let us start by considering the causal system defined through

$$y(n) = x(n) + x(n-1) - 0.5y(n-1). \tag{1.25}$$

As we can see, the output of this system only depends on values from the present, $x(n)$, and from the past, $x(n-1)$ and $y(n-1)$.

Consider now the system

$$y(n) = x(n) + x(n+1) - 0.5y(n-1). \tag{1.26}$$

In this case, in addition to present and past values, a future value is also required, i.e., $x(n+1)$. However, we can transform this non-causal system into a causal system by changing the variable using $m = n + 1$,

$$y(m-1) = x(m-1) + x(m) - 0.5y(m-2). \tag{1.27}$$

Note that, in this case, the transformation of the non-causal system into a causal system was made at the cost of introducing a delay of one time unit into the system. In general, this is only

possible if there are no real-time requirements, that is, if it is not necessary to have the output of the system in the shortest possible time (or within a pre-defined maximum limit) after the input was provided.

1.2.4 Properties of systems: Invertibility

A system is invertible if it is possible to determine the input signal from the output signal. Therefore, for this to be possible, each distinct input signal must produce a distinct output signal. Hence, if the system H, such that $y(n) = H[x(n)]$, is invertible, there will be an inverse system, H^{-1}, such that

$$x(n) = H^{-1}[y(n)], \quad \forall x(n). \tag{1.28}$$

For example, the system

$$y(n) = H[x(n)] = x^3(n) \tag{1.29}$$

is invertible, but the system

$$y(n) = H[x(n)] = x^2(n) \tag{1.30}$$

is not (why?).

1.2.5 Properties of systems: Linearity

Linearity is one of the two most important properties of a system, because, when verified, it allows the use of a wide range of tools for system analysis and synthesis. The other important property is the temporal invariance, which is discussed in the next section. To be linear, a system has to be **additive** and **homogeneous**.

A system is additive if

$$H[x_1(n) + x_2(n)] = H[x_1(n)] + H[x_2(n)], \quad \forall x_1(n), x_2(n). \tag{1.31}$$

This property is represented in terms of a block diagram in Fig. 1.19.

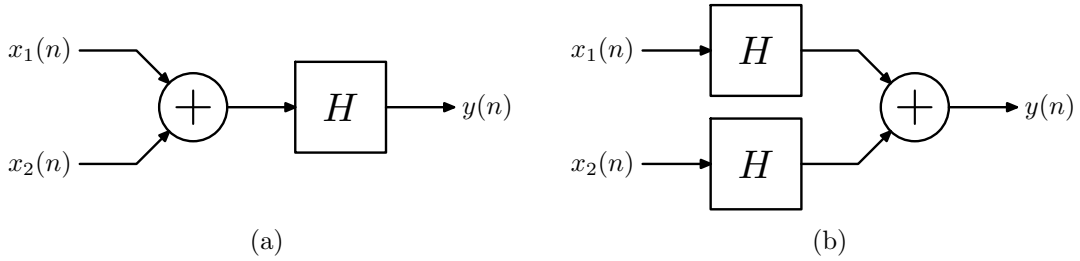


Figure 1.19: The additivity property allows permuting the sum block with the system, maintaining the same functionality.

A system is homogeneous if

$$H[\alpha x(n)] = \alpha H[x(n)], \quad \forall x(n), \alpha. \tag{1.32}$$

Fig. 1.20 illustrates this property, showing that in this case it is possible to permute the system with the gain block.

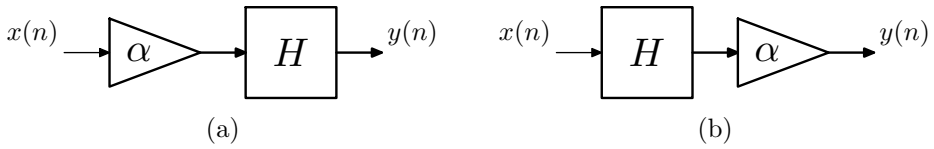


Figure 1.20: The homogeneity property allows permuting the gain block with the system, while keeping the same functionality.

1.2.6 Properties of systems: Time invariance

A system is time-invariant if a time shift of the input signal (lag or lead) produces only an equal time shift of the output signal. Therefore, a system $y(n) = H[x(n)]$ is time-invariant if

$$y(n-k) = H[x(n-k)], \quad \forall x(n), k. \tag{1.33}$$

As can be seen in Fig. 1.21, this means that one can switch a delay block with the system, obtaining an equivalent result.

For example, the ideal full-wave rectifier is a time-invariant system, because

$$H[x(n-k)] = |x(n-k)| = y(n-k). \tag{1.34}$$

On the other hand, the variable-gain amplifier is not time-invariant, because

$$H[x(n-k)] = \alpha(n)x(n-k) \neq y(n-k) = \alpha(n-k)x(n-k). \tag{1.35}$$

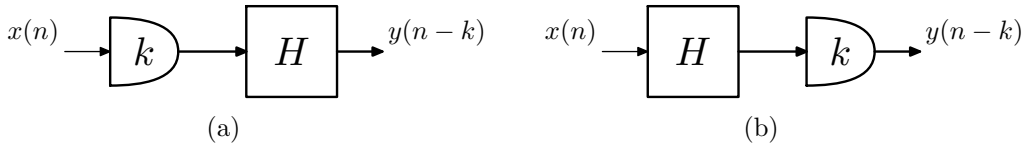


Figure 1.21: The time invariance property allows to permute the block of delay with the system.

1.3 Linear and time-invariant systems

1.3.1 Signal representation by superposition of impulses

As can be seen in Fig. 1.22, multiplying a unit impulse at $n = k$ by any discrete-time signal $x(n)$ results in a new impulse also at $n = k$ and with amplitude equal to $x(k)$, i.e.,

$$x(n)\delta(n-k) = x(k)\delta(n-k), \quad \forall k \in \mathbb{Z}.$$

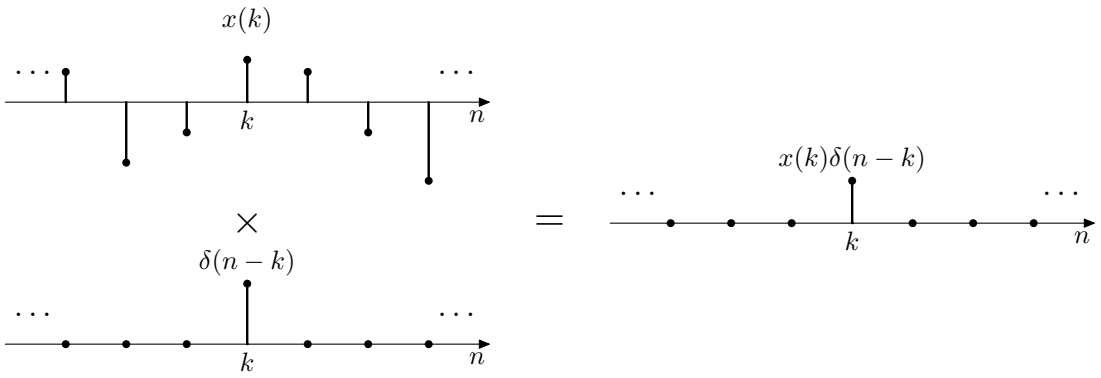


Figure 1.22: Result of multiplying a unit impulse at $n = k$ by a discrete signal $x(n)$.

Based on this observation, it is possible to represent any discrete-time signal, $x(n)$, through a sum of impulses, as exemplified in Fig. 1.23,

$$x(n) = \sum_{k=-\infty}^{\infty} x(k)\delta(n-k).$$

1.3.2 Impulse response

Let us now consider a linear, time-invariant discrete-time system, H, and its response to the impulse $\delta(n)$, which we will represent by $h(n)$ and call **impulse response** of the system (see Fig. 1.24). The response of system H to an arbitrary input signal, $x(n)$, can be obtained by

$$y(n) = H[x(n)] = H\left[\sum_{k=-\infty}^{\infty} x(k)\delta(n-k)\right]. \tag{1.36}$$

As the system is linear and therefore additive, we can write

$$H\left[\sum_{k=-\infty}^{\infty} x(k)\delta(n-k)\right] = \sum_{k=-\infty}^{\infty} H[x(k)\delta(n-k)], \tag{1.37}$$

and also, using the property of homogeneity,

$$\sum_{k=-\infty}^{\infty} H[x(k)\delta(n-k)] = \sum_{k=-\infty}^{\infty} x(k)H[\delta(n-k)], \tag{1.38}$$

and finally, making use of the time-invariance property, we have

$$\sum_{k=-\infty}^{\infty} x(k)H[\delta(n-k)] = \sum_{k=-\infty}^{\infty} x(k)h(n-k). \tag{1.39}$$

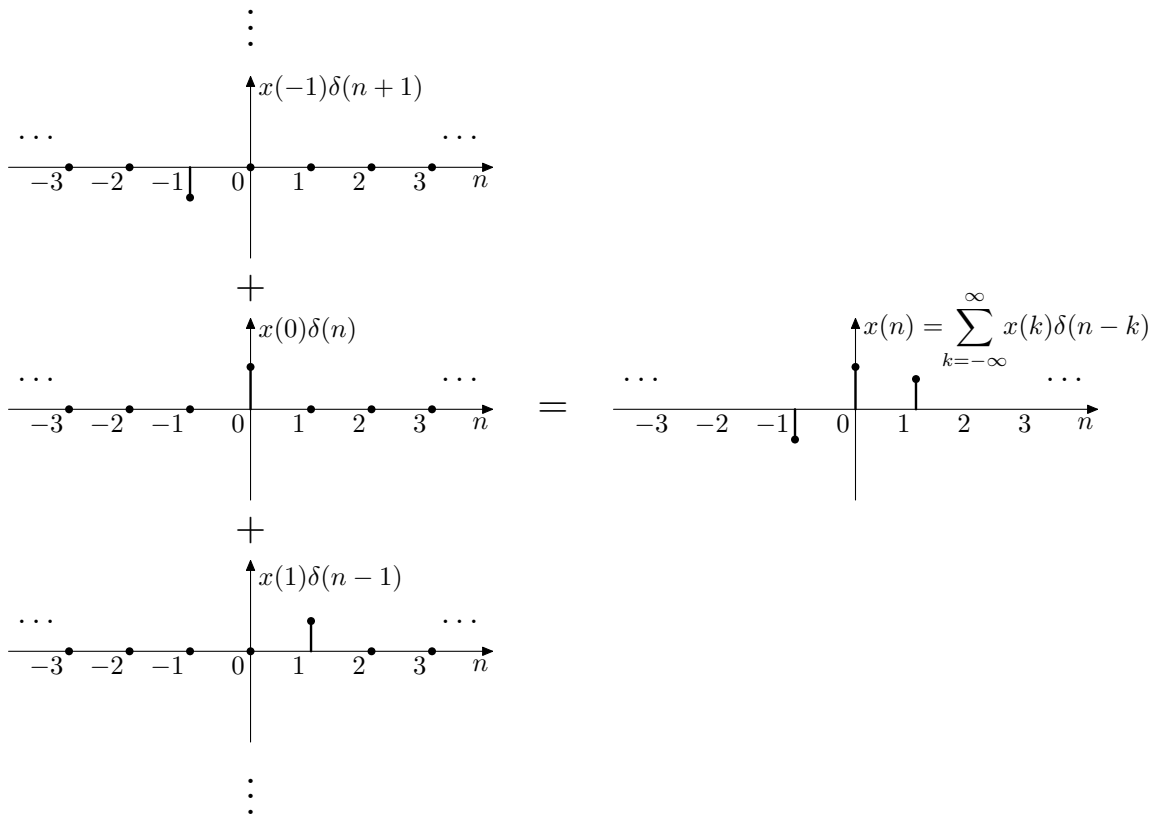


Figure 1.23: Representation of a discrete signal, $x(n)$, through a sum of impulses.

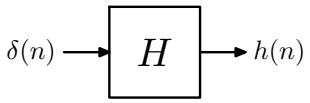


Figure 1.24: Impulse response of a system H.

This last expression shows us a way to calculate the response of any linear and time-invariant discrete-time system, with impulse response $h(n)$, to any input signal, $x(n)$. This operation is called **discrete linear convolution**, and is represented by the "$*$" operator,

$$x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k). \tag{1.40}$$

Example 1.3
Consider a system with impulse response $h(n) = 3\delta(n) + 2\delta(n-1) + \delta(n-2)$. *We want to determine the response of this system to the signal* $x(n) = \delta(n) - \delta(n-1)$.

The response can be calculated through

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k)h(n-k) = \tag{1.41a}$$

$$= x(0)h(n-0) + x(1)h(n-1) = h(n) - h(n-1) = \tag{1.41b}$$

$$= 3\delta(n) + 2\delta(n-1) + \delta(n-2) - 3\delta(n-1) - 2\delta(n-2) - \delta(n-3) = \tag{1.41c}$$

$$3\delta(n) - \delta(n-1) - \delta(n-2) - \delta(n-3). \tag{1.41d}$$

1.3.3 Properties of the convolution operation

We saw above that the convolution operation plays a very important role in the context of linear and time-invariant systems. Let us now look at some of its properties and their implications.

One of the most obvious properties is **commutativity**, that is,

$$a(n) * b(n) = \sum_{k=-\infty}^{\infty} a(k)b(n-k) = \sum_{k=-\infty}^{\infty} a(n-k)b(k) = b(n) * a(n), \tag{1.42}$$

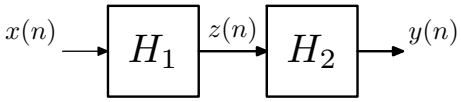
relation that can be verified through a simple change of variable.

The convolution operation is also **associative**, that is,

$$a(n) * b(n) * c(n) = a(n) * \big(b(n) * c(n)\big) = \big(a(n) * b(n)\big) * c(n). \tag{1.43}$$

Example 1.4

We intend to determine the system equivalent to the following ***series (or cascade) association*** *of the systems H_1 and H_2,*



Suppose that you put at the input of this cascade a delta signal, $\delta(n)$. Determine the output signal and then get the equivalent impulse response.

The convolution operation is also **distributive with respect to addition**, that is,

$$a(n) * \big(b(n) + c(n)\big) = a(n) * b(n) + a(n) * c(n). \tag{1.44}$$

(Note: use this property to determine the equivalent system in a **parallel association** of systems).

1.3.4 The impulse response and some properties of systems

Memory: A system has no memory if it only uses the actual input value to calculate the output. Therefore, since the output of a linear and time-invariant system can be calculated using

$$y(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k), \tag{1.45}$$

in a memoryless system we have $y(n) = cx(n)$, where c is an arbitrary constant, that is, $h(n) = c\delta(n)$.

Causality: A system is causal if, to calculate its output at a given instant n, only current or past input values are needed. Therefore, the output of a causal system at the instant n can only be calculated based on $x(n-k), k \geq 0$. This implies $h(n) = 0, n < 0$, i.e., the output of a linear and time-invariant causal system can be calculated through

$$y(n) = \sum_{k=0}^{\infty} h(k)x(n-k). \tag{1.46}$$

Stability: As we have seen, a system is stable if it responds to amplitude-limited inputs with amplitude-limited outputs. Therefore, a linear, time-invariant discrete-time system is stable if,

$$\forall x(n) \quad \text{and} \quad |x(n)| \leq M_x < \infty,$$

we have

$$|y(n)| \leq M_y < \infty.$$

Then,

$$|y(n)| = \left| \sum_{k=-\infty}^{\infty} h(k)x(n-k) \right| \leq \sum_{k=-\infty}^{\infty} |h(k)x(n-k)| = \tag{1.47a}$$

$$\sum_{k=-\infty}^{\infty} |h(k)||x(n-k)| \leq M_x \sum_{k=-\infty}^{\infty} |h(k)|. \tag{1.47b}$$

Therefore, if the system verifies the condition

$$\sum_{k=-\infty}^{\infty} |h(k)| < \infty, \tag{1.48}$$

then it is stable.

Example 1.5
Consider a linear and time-invariant system whose impulse response is given by

$$h(n) = a^n u(n),$$

where $a \in \mathbb{R}$. We want to determine for which values of the parameter a this system is stable. As we have seen, the impulse response has to obey

$$\sum_{k=-\infty}^{\infty} |h(k)| < \infty,$$

whereby

$$\sum_{k=-\infty}^{\infty} |a^k u(k)| = \sum_{k=0}^{\infty} |a^k| < \infty,$$

i.e., $|a| < 1$.

Invertibility: A system is invertible if its input can be obtained from its output. Let us call the impulse response of the inverse system (when such a system exists) $h^{-1}(n)$. As we have seen, the response of a linear, time-invariant system can be calculated using the convolution operation. For example, in the case of discrete-time we have $y(n) = x(n) * h(n)$. The recovery of $x(n)$ from $y(n)$ is called **deconvolution**. Therefore, $h^{-1}(n)$ must be such that $x(n) = y(n) * h^{-1}(n)$, that is,

$$x(n) = y(n) * h^{-1}(n) = [x(n) * h(n)] * h^{-1}(n) = x(n) * [h(n) * h^{-1}(n)], \tag{1.49}$$

implying

$$h(n) * h^{-1}(n) = \delta(n). \tag{1.50}$$

Example 1.6
Let us consider a linear, time-invariant, discrete-time system that reproduces the input signal added to an input replica delayed by one time unit (echo), i.e.,

$$y(n) = x(n) + ax(n-1).$$

We want to determine (in case it exists) the inverse system that will eliminate the echo in the receiver.

We start by writing the impulse response of this system, which is given by

$$h(n) = \delta(n) + a\delta(n-1).$$

Then,

$$h(n) * h^{-1}(n) = [\delta(n) + a\delta(n-1)] * h^{-1}(n) = h^{-1}(n) + ah^{-1}(n-1) = \delta(n).$$

Considering that the inverse system is causal, we have $h^{-1}(n) = 0, n < 0$*. So, for* $n = 0$*, we can write*

$$h^{-1}(0) + ah^{-1}(-1) = \delta(0),$$

i.e., $h^{-1}(0) = 1$*. For* $n = 1$ *we have*

$$h^{-1}(1) + ah^{-1}(0) = \delta(1),$$

i.e., $h^{-1}(1) = -a$*. For* $n = k \geq 0$ *we have*

$$h^{-1}(k) = (-a)^k.$$

Therefore, the impulse response of the inverse system is $h^{-1}(n) = (-a)^n u(n)$*.*

Note that the inverse system is only stable if

$$\sum_{n=-\infty}^{\infty} |h^{-1}(n)| = \sum_{n=0}^{\infty} |(-a)^n| < \infty,$$

implying, in this case, that $|a| < 1$*.*

1.3.5 Response to complex exponentials

Let us consider a linear, time-invariant, discrete-time system with impulse response $h(n)$. The response of this system to the signal $x(n) = e^{\mathrm{j}\Omega n}$ is given by

$$y(n) = h(n) * x(n) = \sum_{k=-\infty}^{\infty} h(k)e^{\mathrm{j}\Omega(n-k)} = e^{\mathrm{j}\Omega n} \sum_{k=-\infty}^{\infty} h(k)e^{-\mathrm{j}\Omega k} = H(e^{\mathrm{j}\Omega})e^{\mathrm{j}\Omega n}, \quad (1.51)$$

with

$$H(e^{\mathrm{j}\Omega}) = \sum_{k=-\infty}^{\infty} h(k)e^{-\mathrm{j}\Omega k}, \quad (1.52)$$

that we call the **frequency response** of the system. We recall that, according to Euler's relation,

$$e^{\mathrm{j}\Omega n} = \cos(\Omega n) + \mathrm{j}\sin(\Omega n) \quad (1.53)$$

and, therefore, this signal represents a pure (normalized) frequency of Ω radians per sample.

This result shows a very important characteristic of the linear and time-invariant systems, which will serve as the basis for other results that we will present later: the response to the signal $e^{\mathrm{j}\Omega n}$

is the signal $H(e^{\mathrm{j}\Omega})e^{\mathrm{j}\Omega n}$, where $H(e^{\mathrm{j}\Omega}) \in \mathbb{C}$ is given by (1.52). For this reason, we say that complex exponentials are **eigensignals** of the linear and time-invariant systems.

As we have seen, $H(e^{\mathrm{j}\Omega})$ depends, on the one hand, on the system itself and, on the other hand, on the frequency, Ω, of the complex exponential at the input of the system. Therefore, for a given system, the value of $H(e^{\mathrm{j}\Omega})$ only depends on the frequency of the signal.

Example 1.7
Consider a system with impulse response

$$h(n) = \begin{cases} 0.5, & n = 0, 1 \\ 0, & \textit{others} \end{cases}.$$

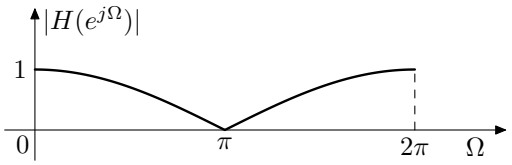
The frequency response of this system is given by

$$H(e^{\mathrm{j}\Omega}) = \sum_{n=-\infty}^{\infty} h(n)e^{-\mathrm{j}\Omega n} = \frac{1}{2} + \frac{1}{2}e^{-\mathrm{j}\Omega} = \frac{1}{2}e^{-\mathrm{j}\Omega/2}(e^{\mathrm{j}\Omega/2} + e^{-\mathrm{j}\Omega/2}) = e^{-\mathrm{j}\Omega/2}\cos\frac{\Omega}{2}.$$

Since, in general, $H(e^{\mathrm{j}\Omega}) \in \mathbb{C}$, it is usual to calculate its modulus and phase. In this case, the modulus is given by

$$|H(e^{\mathrm{j}\Omega})| = \left|e^{-\mathrm{j}\Omega/2}\cos\frac{\Omega}{2}\right| = \left|\cos\frac{\Omega}{2}\right|,$$

which has the following shape:



Example 1.8
Consider now a system with impulse response

$$h(n) = \begin{cases} 0.5, & n = 0 \\ -0.5, & n = 1 \\ 0, & \textit{others} \end{cases}.$$

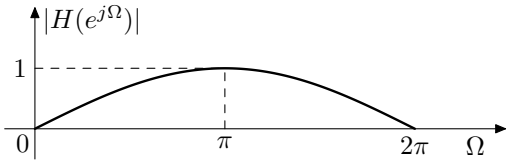
The frequency response of this system is given by

$$H(e^{\mathrm{j}\Omega}) = \sum_{n=-\infty}^{\infty} h(n)e^{-\mathrm{j}\Omega n} = \frac{1}{2} - \frac{1}{2}e^{-\mathrm{j}\Omega} = \frac{1}{2}e^{-\mathrm{j}\Omega/2}(e^{\mathrm{j}\Omega/2} - e^{-\mathrm{j}\Omega/2}) = \mathrm{j}e^{-\mathrm{j}\Omega/2}\sin\frac{\Omega}{2}.$$

Its modulus is

$$|H(e^{\mathrm{j}\Omega})| = \left|\mathrm{j}e^{-\mathrm{j}\Omega/2}\sin\frac{\Omega}{2}\right| = \left|\sin\frac{\Omega}{2}\right|,$$

which has the following shape:



1.4 The discrete Fourier transform (DFT) and the frequency domain

Let us consider a discrete-time and periodic signal, $x(n)$, with period N. We intend to determine whether it is possible to represent (or at least approximate) $x(n)$ through the overlapping of complex exponentials, i.e.,

$$x(n) \approx \tilde{x}(n) = \sum_k X(k) e^{\mathrm{j}\Omega_k n}, \tag{1.54}$$

where $\tilde{x}(n)$ is an approximated version of $x(n)$.

Before proceeding further, let us see that such representation is advantageous. In fact, as we saw above, the response of a linear and time-invariant system, whose impulse response is $h(n)$, to a signal $x(n)$, can be obtained through the convolution sum, i.e.,

$$y(n) = x(n) * h(n) = \sum_m h(m)x(n-m) = \sum_m h(m) \left[\sum_k X(k) e^{\mathrm{j}\Omega_k (n-m)} \right] = \tag{1.55a}$$

$$\sum_k X(k) e^{\mathrm{j}\Omega_k n} \left[\sum_m h(m) e^{-\mathrm{j}\Omega_k m} \right] = \sum_k X(k) H(e^{\mathrm{j}\Omega_k}) e^{\mathrm{j}\Omega_k n} = \sum_k Y(k) e^{\mathrm{j}\Omega_k n}, \tag{1.55b}$$

where $H(e^{\mathrm{j}\Omega_k})$ denotes the value of the frequency response of the system at frequency Ω_k radians. Therefore, if it is possible to represent $x(n)$ as a sum of complex exponentials, the output of any linear and time-invariant system will also be a sum of complex exponentials, whose coefficients, $Y(k)$, are simply obtained by calculating the products $X(k)H(e^{\mathrm{j}\Omega_k})$. In short, the convolution operation is transformed into the multiplication operation.

First, let us see what restrictions must be applied to the frequencies Ω_k so that the signal resulting from (1.54) is also periodic with period N. Because, to verify this condition, we have to have

$$\tilde{x}(n) = \tilde{x}(n+N) = \sum_k X(k) e^{\mathrm{j}\Omega_k (n+N)} = \sum_k X(k) e^{\mathrm{j}\Omega_k n} e^{\mathrm{j}\Omega_k N}, \tag{1.56}$$

then

$$\Omega_k N = 2\pi k, \quad k \in \mathbb{Z}, \tag{1.57}$$

i.e.,

$$\Omega_k = \frac{2\pi k}{N}, \quad k \in \mathbb{Z}. \tag{1.58}$$

Therefore, all frequencies Ω_k must be multiples of the fundamental frequency, that is, multiples of $2\pi/N$.

On the other hand, since

$$e^{\mathrm{j}\frac{2\pi}{N}kn} = e^{\mathrm{j}\frac{2\pi}{N}(k+N)n}, \tag{1.59}$$

then there are only N distinct exponentials, so the sum in (1.54) can be limited to N terms, i.e.,

$$x(n) \approx \tilde{x}(n) = \sum_{k=\langle N\rangle} X(k)e^{\mathrm{j}\frac{2\pi}{N}kn}, \tag{1.60}$$

where the notation $\langle N\rangle$ means any set of N consecutive integers[6].

Let us now see that, in fact, it is possible to exactly represent any periodic signal, $x(n)$, by a sum of complex exponentials. For this, we will first show that the N exponentials are all orthogonal to each other, that is, that they form a basis in the space of discrete-time periodic signals of dimension N.

The inner product between two discrete signals, $a(n)$ and $b(n)$, with duration N, is defined as

$$\langle a(n), b(n)\rangle = \sum_{n=\langle N\rangle} a(n)b^*(n). \tag{1.61}$$

Two signals are orthogonal if their inner product is zero. Since

$$\left\langle e^{\mathrm{j}\frac{2\pi}{N}kn}, e^{\mathrm{j}\frac{2\pi}{N}ln}\right\rangle = \sum_{n=\langle N\rangle} e^{\mathrm{j}\frac{2\pi}{N}kn}e^{-\mathrm{j}\frac{2\pi}{N}ln} = \tag{1.62a}$$

$$= \sum_{n=0}^{N-1} e^{\mathrm{j}\frac{2\pi}{N}(k-l)n} = \begin{cases} N, & k=l \\ \dfrac{1-e^{\mathrm{j}\frac{2\pi}{N}(k-l)N}}{1-e^{\mathrm{j}\frac{2\pi}{N}(k-l)}} = 0, & k\neq l \end{cases}, \tag{1.62b}$$

we then have that, in fact, the complex exponential N are all orthogonal to each other. Therefore, any periodic signal, $x(n)$, with period N, can be represented by

$$x(n) = \sum_{k=\langle N\rangle} X(k)e^{\mathrm{j}\frac{2\pi}{N}kn}. \tag{1.63}$$

The coefficients $X(k)$ can be obtained by calculating the inner product between the signal and the base signals, that is,

$$\left\langle x(n), e^{\mathrm{j}\frac{2\pi}{N}kn}\right\rangle = \sum_{n=\langle N\rangle} x(n)e^{-\mathrm{j}\frac{2\pi}{N}kn} = \sum_{n=\langle N\rangle}\sum_{l=\langle N\rangle} X(l)e^{\mathrm{j}\frac{2\pi}{N}ln}e^{-\mathrm{j}\frac{2\pi}{N}kn} = \tag{1.64a}$$

$$\sum_{l=\langle N\rangle} X(l)\sum_{n=\langle N\rangle} e^{\mathrm{j}\frac{2\pi}{N}(l-k)n} = NX(k). \tag{1.64b}$$

[6]Note that, due to the periodicity of the exponentials, it does not matter where this set of N consecutive integers begins. However, sometimes, motivated by the convenience of mathematical manipulation, a particular set is used, for example, $\{0, 1, \ldots, N-1\}$.

Hence,

$$X(k) = \frac{1}{N} \sum_{n=\langle N \rangle} x(n) e^{-\mathtt{j} \frac{2\pi}{N} kn}. \tag{1.65}$$

The operation that transforms the signal $x(n)$ into the coefficients $X(k)$ is called **direct discrete Fourier transform (or analysis)**. The operation that transforms the coefficients $X(k)$ into the signal $x(n)$ is called **inverse discrete Fourier transform (or synthesis)**. Sometimes, we will use the notation

$$x(n) \overset{\text{DFT}}{\longleftrightarrow} X(k) \tag{1.66}$$

to indicate that $x(n)$ and $X(k)$ are related through the discrete Fourier transform (DFT).

This pair of transformations allow us to go from a **time domain representation** of a discrete-time periodic (or finite-duration) signal to its **frequency domain representation** and vice-versa.

1.4.1 Some properties of the discrete Fourier transform

There are several properties associated with the discrete Fourier transform that can be very useful, both in simplifying some calculations and helping to better understand its meaning.

The **linearity** is one of the most obvious and easiest to show properties. In fact, considering two signals, $a(n)$ and $b(n)$, with duration N, and the respective discrete Fourier transforms, $A(k)$ and $B(k)$,

$$a(n) \overset{\text{DFT}}{\longleftrightarrow} A(k) \tag{1.67a}$$

$$b(n) \overset{\text{DFT}}{\longleftrightarrow} B(k), \tag{1.67b}$$

we can show that

$$c_1 a(n) + c_2 b(n) \overset{\text{DFT}}{\longleftrightarrow} c_1 A(k) + c_2 B(k), \tag{1.68}$$

with $c_1, c_2 \in \mathbb{C}$.

Some interesting properties have to do with various types of **symmetries**. For example, if the signal $x(n) \in \mathbb{R}$, then

$$X(k) = X^*(-k), \tag{1.69}$$

i.e., in this case the modulus of $X(k)$ has even symmetry and the phase has odd symmetry. It can also be verified that, if, in addition to being real, the signal has even symmetry, then $X(k) \in \mathbb{R}$.

If a signal, $x(n)$, with discrete Fourier transform $X(k)$, is **delayed** n_0 time units, then

$$\frac{1}{N} \sum_{n=\langle N \rangle} x(n - n_0) e^{-\mathtt{j} \frac{2\pi}{N} kn} = \frac{1}{N} \sum_{m=\langle N \rangle} x(m) e^{-\mathtt{j} \frac{2\pi}{N} k(m+n_0)} = e^{-\mathtt{j} \frac{2\pi}{N} kn_0} X(k). \tag{1.70}$$

Therefore, if

$$x(n) \overset{\text{DFT}}{\longleftrightarrow} X(k), \tag{1.71}$$

then

$$x(n-n_0) \overset{\text{DFT}}{\longleftrightarrow} e^{-\mathrm{j}\frac{2\pi}{N}kn_0}X(k). \tag{1.72}$$

Note that a **time shift** only causes a change in the phase of the discrete Fourier transform.

On the other hand, if we introduce a **frequency shift** of k_0 units, we will have

$$\sum_{k=\langle N\rangle} X(k-k_0)e^{\mathrm{j}\frac{2\pi}{N}kn} = \sum_{l=\langle N\rangle} X(l)e^{\mathrm{j}\frac{2\pi}{N}(l+k_0)n} = e^{\mathrm{j}\frac{2\pi}{N}k_0n}x(n). \tag{1.73}$$

Therefore, if

$$x(n) \overset{\text{DFT}}{\longleftrightarrow} X(k), \tag{1.74}$$

then

$$e^{\mathrm{j}\frac{2\pi}{N}k_0n}x(n) \overset{\text{DFT}}{\longleftrightarrow} X(k-k_0). \tag{1.75}$$

Example 1.9

Consider a signal $x(n)$ of duration N, with N even, such that $x(n) \overset{\text{DFT}}{\longleftrightarrow} X(k)$. If a shift in the frequency of $N/2$ positions is performed, we will have $e^{\mathrm{j}\pi n}x(n) \overset{\text{DFT}}{\longleftrightarrow} X(kN/2)$, i.e., $(-1)^n x(n) \overset{\text{DFT}}{\longleftrightarrow} X(kN/2)$. Therefore, if a given low-pass type signal is multiplied by $(-1)^n$, it will have a high-pass characteristic and vice-versa.

One of the most important properties of the discrete Fourier transform has to do with the **conservation of energy** (ignoring a possible multiplicative constant), when moving from one domain to the other (time to frequency or vice versa). The energy of a signal $x(n)$ with duration N is given by

$$E_x = \sum_{n=\langle N\rangle} |x(n)|^2 = \sum_{n=\langle N\rangle} x(n)x^*(n) = \sum_{n=\langle N\rangle} x(n) \sum_{k=\langle N\rangle} X^*(k)e^{-\mathrm{j}\frac{2\pi}{N}kn} = \tag{1.76a}$$

$$\sum_{k=\langle N\rangle} X^*(k) \sum_{n=\langle N\rangle} x(n)e^{-\mathrm{j}\frac{2\pi}{N}kn} = \sum_{k=\langle N\rangle} X^*(k)NX(k) = N\sum_{k=\langle N\rangle} |X(k)|^2. \tag{1.76b}$$

This result is known as the **Parseval relation**.

Let us now consider two signals, $a(n)$ and $b(n)$, with duration N, and the respective discrete Fourier transforms, $A(k)$ and $B(k)$. We intend to determine the discrete Fourier transform of the signal $a(n)b(n)$, i.e.,

$$\frac{1}{N}\sum_{n=\langle N\rangle} a(n)b(n)e^{-\mathrm{j}\frac{2\pi}{N}kn} = \frac{1}{N}\sum_{n=\langle N\rangle} \left[\sum_{l=\langle N\rangle} A(l)e^{\mathrm{j}\frac{2\pi}{N}ln}\right] b(n)e^{-\mathrm{j}\frac{2\pi}{N}kn} = \tag{1.77a}$$

$$\sum_{l=\langle N\rangle} A(l)\left[\frac{1}{N}\sum_{n=\langle N\rangle} b(n)e^{-\mathrm{j}\frac{2\pi}{N}(k-l)n}\right] = \sum_{l=\langle N\rangle} A(l)B(k-l) = A(k)\circledast B(k). \tag{1.77b}$$

Note that the last sum presented is in the form of a convolution. However, since the signals are periodic and this sum is only defined in consecutive integers N, then the result of this convolution is also a periodic signal with period N. That is why we use the symbol "$\circledast$" to denote this **periodic (or circular) convolution**, instead of the symbol "$*$" that we had introduced earlier to denote the linear convolution operation.

When calculating the inverse discrete Fourier transform of the product $A(k)B(k)$ a similar result is also obtained. In fact,

$$\sum_{k=\langle N\rangle} A(k)B(k)e^{\mathrm{j}\frac{2\pi}{N}kn} = \sum_{k=\langle N\rangle}\left[\frac{1}{N}\sum_{m=\langle N\rangle} a(m)e^{-\mathrm{j}\frac{2\pi}{N}km}\right]B(k)e^{\mathrm{j}\frac{2\pi}{N}kn} = \tag{1.78a}$$

$$\frac{1}{N}\sum_{m=\langle N\rangle} a(m)\left[\sum_{k=\langle N\rangle} B(k)e^{\mathrm{j}\frac{2\pi}{N}k(n-m)}\right] = \frac{1}{N}\sum_{m=\langle N\rangle} a(m)b(n-m) = \frac{1}{N}a(n)\circledast b(n). \tag{1.78b}$$

In short, if

$$a(n) \overset{\text{DFT}}{\longleftrightarrow} A(k) \tag{1.79a}$$

$$b(n) \overset{\text{DFT}}{\longleftrightarrow} B(k), \tag{1.79b}$$

then

$$a(n)b(n) \overset{\text{DFT}}{\longleftrightarrow} A(k)\circledast B(k) \tag{1.80a}$$

$$a(n)\circledast b(n) \overset{\text{DFT}}{\longleftrightarrow} NA(k)B(k), \tag{1.80b}$$

i.e., time domain signal multiplications are transformed into frequency domain convolutions and vice versa.

1.4.2 The discrete cosine transform

We mentioned that if a signal $x(n)$, of length N, is real and has even symmetry, then the coefficients of its DFT, $X(k)$, are real numbers. This is a convenient situation in many cases. Besides, because of the intrinsic periodic extention that $x(n)$ suffers when we calculate its DFT, discontinuity issues usually occur, implying high frequency components with significant values. For ataining compression, it is desirable to have the energy of the signal packed in as few as possible coefficients of the transform, hence the DFT is not usually a good choice.

The **discrete cosine transform (DCT)** addresses these two problems. On the one hand, it provides transform coefficients that are real. On the other hand, it provides high energy packing. There are several versions of the DCT. The one used for compression is known as DCT-II and has the direct relation

$$X(k) = \sqrt{\frac{2}{N}}\,\alpha(k)\sum_{n=0}^{N-1} x(n)\cos\left(\frac{\pi k(2n+1)}{2N}\right), \quad k = 0, 1, \ldots, N-1 \tag{1.81}$$

and the inverse relation

$$x(n) = \sqrt{\frac{2}{N}} \sum_{k=0}^{N-1} \alpha(k) X(k) \cos\left(\frac{\pi k(2n+1)}{2N}\right), \quad n = 0, 1, \ldots, N-1, \tag{1.82}$$

with

$$\alpha(0) = \frac{1}{\sqrt{2}} \quad \text{and} \quad \alpha(k) = 1, k \neq 0.$$

This version of the DCT can be calculated using a $2N$ point DFT (or FFT[7]), by mirrowing $x(n)$ (for example, "abcd" is transformed into "dcbaabcd") and performing an appropriate time-shift adjustment.

1.5 The Z transform and digital filters

Let us consider a discrete system, H, linear and time-invariant, with impulse response $h(n)$, and the signal

$$x(n) = z^n = \left(re^{\mathrm{j}\Omega}\right)^n.$$

The response of H to $x(n)$ can be obtained through the convolution sum, i.e.,

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k) = \sum_{k=-\infty}^{\infty} h(k)z^{n-k} = z^n \sum_{k=-\infty}^{\infty} h(k)z^{-k} = z^n H(z).$$

The function

$$H(z) = \sum_{n=-\infty}^{\infty} h(n)z^{-n} \tag{1.83}$$

is the Z **transform** of $h(n)$.

Example 1.10
Calculate the Z transform, $X(z)$, of the signal $x(n) = a^n u(n)$.

We have

$$X(z) = \sum_{n=-\infty}^{\infty} x(n)z^{-n} = \sum_{n=0}^{\infty} (az^{-1})^n = \frac{1}{1-az^{-1}}, \quad |z| > |a|.$$

Note that a convergence region is associated with $X(z)$, that is, in this case the function $X(z)$ obtained is only valid for values of z such that $|z| > |a|$.

[7]The Fast Fourier Transform (FFT) is an efficient algorithm for computing the coefficients of the DFT. Instead of requiring $\mathcal{O}(N^2)$ operations, the FFT needs only $\mathcal{O}(N \log N)$ operations, for a signal of length N.

Example 1.11
Calculate the Z transform of the signal $x(n) = a^n \cos(\Omega n) u(n)$.

In this case $X(z)$ *is given by*

$$X(z) = \sum_{n=-\infty}^{\infty} x(n) z^{-n} = \frac{1}{2} \sum_{n=0}^{\infty} (a e^{\mathrm{j}\Omega} z^{-1})^n + (a e^{-\mathrm{j}\Omega} z^{-1})^n = \tag{1.84a}$$

$$\frac{1}{2} \left(\frac{1}{1 - a e^{\mathrm{j}\Omega} z^{-1}} + \frac{1}{1 - a e^{-\mathrm{j}\Omega} z^{-1}} \right), \quad |z| > |a|, \tag{1.84b}$$

which can be simplified to

$$X(z) = \frac{1 - a \cos \Omega z^{-1}}{1 - 2a \cos \Omega z^{-1} + a^2 z^{-2}}. \tag{1.85}$$

1.5.1 The time-shift property

Let us consider a signal, $x(n)$, with Z transform $X(z)$. We want to determine the Z transform of the signal $x(n-k)$, that is,

$$\sum_{n=-\infty}^{\infty} x(n-k) z^{-n} = \sum_{n=-\infty}^{\infty} x(n) z^{-n-k} = z^{-k} \sum_{n=-\infty}^{\infty} x(n) z^{-n} = z^{-k} X(z).$$

Therefore, if

$$x(n) \overset{\mathcal{Z}}{\longleftrightarrow} X(z),$$

then

$$x(n-k) \overset{\mathcal{Z}}{\longleftrightarrow} z^{-k} X(z).$$

This property allows an easy transformation to the Z domain of linear difference equations of constant coefficients, that is, of the type

$$\sum_{k=0}^{N} a_k y(n-k) = \sum_{k=0}^{M} b_k x(n-k),$$

since, because

$$y(n-k) \overset{\mathcal{Z}}{\longleftrightarrow} z^{-k} Y(z) \quad \text{and} \quad x(n-k) \overset{\mathcal{Z}}{\longleftrightarrow} z^{-k} X(z),$$

we have

$$\sum_{k=0}^{N} a_k y(n-k) \overset{\mathcal{Z}}{\longleftrightarrow} \sum_{k=0}^{N} a_k z^{-k} Y(z) \quad \text{and} \quad \sum_{k=0}^{M} b_k x(n-k) \overset{\mathcal{Z}}{\longleftrightarrow} \sum_{k=0}^{M} b_k z^{-k} X(z)$$

and

$$\sum_{k=0}^{N} a_k z^{-k} Y(z) = \sum_{k=0}^{M} b_k z^{-k} X(z).$$

We can also write

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^{M} b_k z^{-k}}{\sum_{k=0}^{N} a_k z^{-k}}.$$

If $x(n)$ is the input signal and $y(n)$ is the respective output signal of a certain linear, time-invariant system, then the function $H(z)$ is the **transfer function** of that system.

Example 1.12

Determine the transfer function of a system described by the equation

$$y(n) = x(n) - x(n-1) + 0.5y(n-1).$$

Calculating the Z transform term by term, we obtain

$$Y(z) = X(z) - z^{-1}X(z) + 0.5z^{-1}Y(z)$$

and, hence,

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1 - z^{-1}}{1 - 0.5z^{-1}}.$$

The roots of the numerator and denominator of $H(z)$ are called, respectively, **zeros** and **poles** of the transfer function. A system that does not have poles (i.e., the denominator of $H(z)$ is a constant) is called a **FIR (finite impulse response)** system, otherwise it is called an **IIR (infinite impulse response)** system.

Example 1.13

Determine the poles and zeros of the transfer function of the system shown in Example 1.12.

The poles of a system are the roots of the denominator of the transfer function, so, in this case, we have

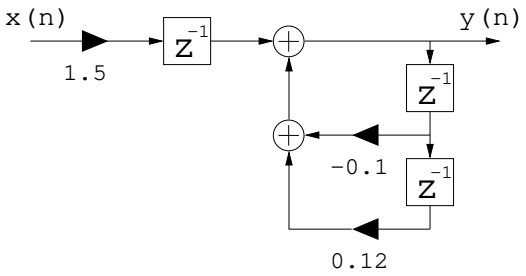
$$1 - 0.5z^{-1} = 0 \Leftrightarrow z = 0.5.$$

The zeros of a system are the roots of the numerator of the transfer function, so, in this case, we have

$$1 - z^{-1} = 0 \Leftrightarrow z = 1.$$

Therefore, the system has a pole at $z = 0.5$ and a zero at $z = 1$.

Example 1.14
Determine the transfer function of the system described by the following block diagram



The difference equation that defines this system is

$$y(n) = 1.5x(n-1) - 0.1y(n-1) + 0.12y(n-2).$$

Transforming into the Z domain,

$$Y(z) = 1.5X(z)z^{-1} - 0.1Y(z)z^{-1} + 0.12Y(z)z^{-2},$$

and, hence, the transfer function is

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1.5z^{-1}}{1 + 0.1z^{-1} - 0.12z^{-2}}.$$

The transfer function, $H(z)$, of a certain system, corresponds to the Z transform of the impulse response, $h(n)$, of that same system, that is,

$$h(n) \overset{\mathcal{Z}}{\longleftrightarrow} H(z). \tag{1.86}$$

In fact, because

$$y(n) = h(n) * x(n) = \sum_{k=-\infty}^{\infty} h(k)x(n-k),$$

then

$$Y(z) = \sum_{n=-\infty}^{\infty} y(n)z^{-n} = \sum_{n=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} h(k)x(n-k)z^{-n} = \tag{1.87a}$$

$$\sum_{k=-\infty}^{\infty} h(k) \sum_{n=-\infty}^{\infty} x(n-k)z^{-n} = \sum_{k=-\infty}^{\infty} h(k) \sum_{n=-\infty}^{\infty} x(n)z^{-(n+k)} = \tag{1.87b}$$

$$X(z) \sum_{k=-\infty}^{\infty} h(k)z^{-k} = X(z)H(z). \tag{1.87c}$$

This result shows the relationship between the multiplication operation in one domain and the convolution operation in the other domain.

1.5.2 System stability

Under certain conditions (that represent an important class of systems), the transfer function can be written as

$$H(z) = \sum_{k=1}^{N} \frac{A_k}{1 - p_k z^{-1}},$$

where the p_k represent the (simple[8]) poles of the system and the A_k are constants. Since the impulse response of the system, $h(n)$, corresponds to the inverse Z transform of $H(z)$, then (see Example 1.10)

$$h(n) = \sum_{k=1}^{N} A_k p_k^n u(n).$$

In general, $h(n)$ only converges if all the terms of the sum converge, for which it is necessary that $|p_k| < 1, \forall k$. In fact, as we saw earlier when we discussed the properties of the impulse response, for a system to be stable we must have

$$\sum_{n=-\infty}^{\infty} |h(n)| < \infty,$$

so for this condition to be verified we have to ensure that $|p_k| < 1, \forall k$, that is, all poles of the system must have a modulus smaller than one[9].

1.5.3 Frequency response

Since, for systems described by linear difference equations of constant coefficients, we have

$$H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{\sum_{k=0}^{N} a_k z^{-k}} = \frac{z^{-M} \sum_{k=0}^{M} b_k z^{M-k}}{z^{-N} \sum_{k=0}^{N} a_k z^{N-k}} = z^{N-M} \frac{b_0 \prod_{k=1}^{M} (z - z_k)}{a_0 \prod_{k=1}^{N} (z - p_k)},$$

[8] This means that the denominator of the transfer function has only simple roots.

[9] Sometimes we refer to this stability condition by saying that all poles of a discrete-time system must be located inside the unit circle of the complex plane.

then, the modulus of its frequency response (or gain of the system) can be written as[10]

$$|H(e^{\mathrm{j}\Omega})| = \left|\frac{b_0}{a_0}\right| \frac{\prod_{k=1}^{M} |e^{\mathrm{j}\Omega} - z_k|}{\prod_{k=1}^{N} |e^{\mathrm{j}\Omega} - p_k|}.$$

Note that this last expression can be interpreted as the quotient between the product of the distances from the point $e^{\mathrm{j}\Omega}$ to the zeros of $H(z)$ and the product of the distances from the point $e^{\mathrm{j}\Omega}$ to the poles of $H(z)$.

Example 1.15

We intend to calculate the modulus of the frequency response of the system whose transfer function is

$$H(z) = \frac{1}{1 - az^{-1}}.$$

We have

$$|H(e^{\mathrm{j}\Omega})| = \frac{1}{|1 - ae^{-\mathrm{j}\Omega}|}.$$

For example, the gain of this system for $f = 0$ is

$$|H(e^{0})| = |H(1)| = \frac{1}{|1 - a|},$$

and for $f = f_s/2$ is

$$|H(e^{\mathrm{j}\pi})| = |H(-1)| = \frac{1}{|1 + a|}.$$

For $f = f_s/4$ we have

$$|H(e^{\mathrm{j}\pi/2})| = |H(\mathrm{j})| = \frac{1}{|1 + \mathrm{j}a|} = \frac{1}{\sqrt{1 + a^2}}.$$

Note that, if $0 < a < 1$, the system has a low-pass behavior (see Fig. 1.25), whereas if $-1 < a < 0$, then the system has a high-pass characteristic (see Fig. 1.26).

---

[10] Compare with the results obtained in Section 1.3.5. Note that $e^{\mathrm{j}\Omega}$ are values of the unit circumference of the complex plane and are the same as those of Equation (1.52). Therefore, the frequency response of a system can be calculated using its transfer function calculated for $z = e^{\mathrm{j}\Omega}$, i.e., over the unit circumference of the complex plane.

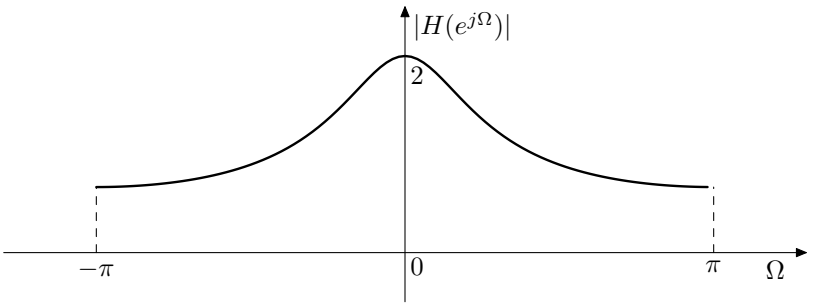


Figure 1.25: Frequency response of an IIR digital low-pass filter with a pole at $z = 0.5$.

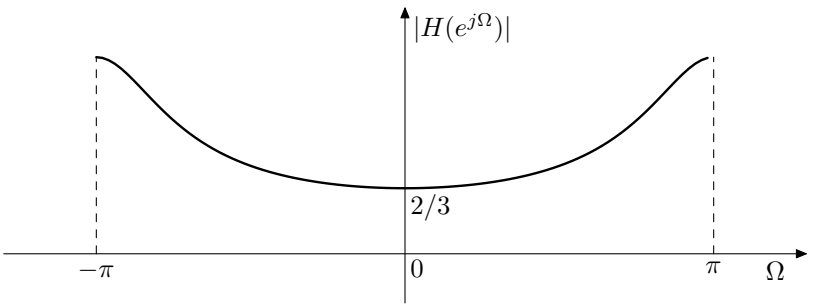


Figure 1.26: Frequency response of an IIR digital high-pass filter with a pole at $z = -0.5$.

2 Data, information and some related problems

What is "**data**"? What is "**information**"?

This section addresses some problems that, in some way, have to do with these questions. Although it is not expected that you have answers for all of them right now, you should nevertheless start thinking about them.

Discussion topic 2.1
Consider the following questions and try to sketch answers for them:

- *What is data?*
- *How to measure the quantity of data?*
- *What is information?*
- *How to measure the quantity of information?*
- *What is a computation?*
- *Are there problems that computers cannot solve?*

Discussion topic 2.2
Consider the following statement:

> *One bit of **data** contains, at most, one bit of **information**.*

Do you think this is true? Why?

Discussion topic 2.3
*Alice told Bob that she found an image compression software that is able to reduce the size of **every** image in, at least, 50% of its original size—for example, if originally the image occupies* $1\,000\,000$ *bytes, after compression it will require at most* $500\,000$ *bytes.*

*Bob was not very impressed by Alice's statement. However, when Alice added that she would also be able to **always** recover the original image from its compressed version, Bob immediately replied:*

> *"THAT IS IMPOSSIBLE! Every lossless (i.e., reversible) compression method is limited, i.e., it cannot compress all messages!"*

Is Bob correct? Why?

Discussion topic 2.4

Consider the following game, which is a case of a Post canonical system[11]*. The objective of the game is to discover the sequence of rules (among a previously specified set) that allows transforming a certain word into another word.*

Consider, for example, the case where the words can be formed using only the letters M, I and U, and that the allowed transformation rules are:

1. $xI \rightarrow xIU$
2. $Mx \rightarrow Mxx$
3. $xIIIy \rightarrow xUy$
4. $xUUy \rightarrow xy$

Let us try some transformations:

1. *Give the steps to transform MI into MUI.*
2. *Can you transform UIM into MIU? Why?*
3. *And MIU into UIM?*
4. *What are the steps to transform MI into MU?*

Can you think of an algorithm that answers questions of the type "Is it possible to transform the word w into the word z, using a certain set of rules"?

Discussion topic 2.5

Consider the set of all functions $f : \mathbb{N} \rightarrow \{0, 1\}$. Alice told Bob that there are functions in that set that cannot be calculated by any finite program, regardless of the programming language used to implement it. Bob thinks Alice is wrong. Is she wrong? Why?

Discussion topic 2.6

Alice wants to play a game with Bob, where a coin has to be tossed. To show that she is using a fair coin, she tosses it twenty times. However, in all trials, heads comes up. Can Bob trust this coin? Why?

Would it be different if the sequence of outcomes had been "00101001110101010110"? Why?

[11] This problem can be reformulated as a string rewriting system, also known as a semi-Thue system, which is related to the notion of unrestricted grammar.

Discussion topic 2.7
Consider the following procedure, suggested by John von Neumann (1903–1957):

1. *Toss a coin twice*
2. *If the results match, start over, forgetting both results*
3. *If the results differ, use the first result, forgetting the second,*

with which he claimed to be possible to obtain the equivalent to a sequence generated by a fair coin, even if the coin is unfair.

Do you agree with him? Why?

Discussion topic 2.8
You work at a company that sells large sequences of random numbers. Your boss thinks he needs to have some way of testing the "randomness" of the sequences before they are sent to the clients (let us consider this as a kind of quality control of the production...). Because you are the best informatics engineer of the company, he asks you to develop a program for testing the randomness of arbitrary sequences.

What should you answer him? Have you any ideas of how to do it? Can you define "randomness"?

Discussion topic 2.9
John von Neumann once said:

> *"Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin."*

What do you think he meant to say with this statement?

Discussion topic 2.10
Consider the following definition of a certain number:

"The smallest positive integer not definable in fewer than twelve words".

Does this number exit? Why?

(This is known as Berry's paradox.)

Discussion topic 2.11
You are at a party with friends. After some drinks, one of them says:

> *"I've had a great idea! I'll write a program that will be capable of analysing other programs and tell us if they are bug-free or not! I wonder why no one have done this before!"*

What would you say to your friend?

Discussion topic 2.12
Consider the following segment of pseudo-code:

```
F(uint x) {
   while(x > 1) {
      if(x % 2 == 0)
         x = x / 2
      else
         x = 3 * x + 1
   }

   return
}
```

Does function F return for all x?

(This is known as Collatz's conjecture.)

In what follows, several ways of measuring information will be discussed, namely, a **combinatorial** approach, a **probabilistic** approach and an **algorithmic** approach. Also, several techniques for representing a certain amount of information in the smallest as possible set of data, will be presented. These are known as **data compression algorithms**.

3 A combinatorial measure of information

Problem 3.1
Alice is tossing a coin repeatedly. She wants to send the outcomes to Bob. To send the ***message****, Alice is only able to use a flashlight, by turning it on and off once per second.*

1. *Suppose that Alice and Bob agreed on the size of the message, n, beforehand, i.e., on how many coin tossing outcomes will be sent in each message. Propose a procedure for this transmission scheme.*
2. *Suppose now that Alice intends to send several messages of lengths, n_1, n_2, ..., not known in advance by Bob. Can you suggest a transmission procedure that works in this case?*

Problem 3.2
Alice wants to send a letter to Bob, using the same transmission scheme. Consider an ***alphabet*** *of 27* ***symbols*** *(26 letters plus the space). How can this be done?*

Problem 3.3
Now Alice wants to play a game with Bob (they are finally face to face!). She thinks of a number between 1 and 100. Bob is allowed to ask every question he wants, but only gets a yes or no answer.

1. *What is the minimum number of questions that Bob needs to sequentially pose to Alice (the next question is posed only after knowing the answer to the previous question), in order to discover the number Alice thought of? What should those questions be?*
2. *Now Bob is only allowed to pose all the questions at once. Is the minimum number of required questions the same? In this case, what should be the questions?*

The minimum number of sequential questions is attained, for example, by successively halving the set of possibilities (a kind of binary search...). Taking as example the game of guessing the number, we have

$$2^b = 100,$$

where b is the number of questions. Hence,

$$b = \log_2 100 \approx 6.644,$$

meaning that, on average, about 6.644 questions are needed to find the number—in some cases six questions are enough, whereas in other cases seven questions are needed.

Example 3.1
Guess a number between 1 *and* 100*:*

$$\begin{gathered}\textit{Is } x \geq 50? \rightarrow 1\\ \textit{Is } x \geq 75? \rightarrow 0\\ \textit{Is } x \geq 62? \rightarrow 1\\ \textit{Is } x \geq 68? \rightarrow 0\\ \textit{Is } x \geq 65? \rightarrow 0\\ \textit{Is } x \geq 63? \rightarrow 1\\ \textit{Is } x = 63? \rightarrow 0\end{gathered}$$

Hence, x *has to be* 64 *(required seven yes/no questions).*

Example 3.2
Guess a number between 1 *and* 100*:*

$$\begin{gathered}\textit{Is } x \geq 50? \rightarrow 1\\ \textit{Is } x \geq 75? \rightarrow 0\\ \textit{Is } x \geq 62? \rightarrow 1\\ \textit{Is } x \geq 68? \rightarrow 0\\ \textit{Is } x \geq 65? \rightarrow 1\\ \textit{Is } x \geq 66? \rightarrow 0\end{gathered}$$

Hence, x *has to be* 65 *(in this case, six yes/no questions were enough).*

Problem 3.4
Using this searching scheme, the average number of questions needed is 6.72*. Verify this value (by calculating how many of the 100 numbers require the seven questions to be defined). Why is this value larger than the theoretical bound of* $\log_2 100 \approx 6.644$ *bits?*

Now imagine that x *is represented in binary. What should be the "natural" questions to pose in this case? Using this approach, can you attain an average number of questions smaller than* 6.72*?*

Notice that, if all the questions had to be provided at once, then in the example of guessing a number between 1 and 100 we had to ask always a minimum of $\lceil \log_2 100 \rceil = 7$ questions.

Problem 3.5
Suppose now that the number to be guessed is between 1 *and* 3*. Compare and discuss the impact on the average number of questions that have to be posed if:*

(a) Questions are posed sequentially, after knowing the previous answers;

(b) The questions are provided in batch.

Discuss also aspects such as the case where the numbers are not equally probable and also the difference between $\log_2 3 \approx 1.585$ *and the average number of questions estimated for the several scenarios addressed.*

Programming 3.1
Write a program that generates the minimal sequence of yes/no answers that allows discovering a certain number between 1 *and* n*.*

Problem 3.6
Using the ideas above, show that, in general, sorting has an average case lower bound of $\Omega(n \log n)$ *comparison operations.*

The problems that we have been addressing consider a measure of information that is usually called **combinatorial**, because it is only concerned with the number of possible objects involved, assuming that they occur with equal probability. Hence, for m distinct objects (a size-m alphabet), the amount of **combinatorial information** needed to specify each one is

$$\log_2 m \quad \text{bits.}$$

This approach for measuring information can be traced back at least to the works of Nyquist and Hartley:

- Harry Nyquist (1889–1976), *Certain Factors Affecting Telegraph Speed*, Bell System Technical Journal (Nyquist, 1924):

  > *"This paper considers two fundamental factors entering into the maximum speed of* ***transmission of intelligence*** *by telegraph. These factors are signal shaping and choice of codes."*

- Ralph Hartley (1888–1970), *Transmission of Information*, Bell System Technical Journal (Hartley, 1928):

  > *"A quantitative measure of "information" is developed which is based on physical as contrasted with psychological considerations."*

In his paper, Hartley proposes measuring the information content, H, of a message as

$$H = n \log m = \log m^n,$$

where m denotes the number of possible symbols (the size of the alphabet) and n the number of symbols in the message. So, Hartley defined the amount of information in a message as the logarithm of the number of possible messages (message space).

The base of the logarithm determines the unit of information used: **hartley** for base 10, **nat** for base e, **bit** for base 2.

Problem 3.7

Alice has a deck of playing cards (52 *cards). After shuffling the deck, Alice wants to send enough information to Bob in order for him to put his deck in the same order. How many bits does Alice need to send to Bob to communicate this information? Can you propose a method for doing it?*

4 A probabilistic measure of information

Let us return to Alice and Bob and consider the following new problem:

Problem 4.1
Using the flashlight communication system, Alice wants to communicate to Bob the results of throwing a pair of dice (i.e., a number from 2 to 12). However, Alice is running out of batteries for the flashlight and, therefore, she wants to preserve them as much as possible. So, she wonders if, in this case, it is possible to use, on average, less than $\log_2 11 \approx 3.46$ *bits for sending to Bob the outcome of each trial... Can you help her solving this problem?*

The first aspect to take into consideration is that the possible outcomes of rolling two dice do not occur with the same frequency. Let us denote by Σ the alphabet representing those outcomes, i.e., $\Sigma = \{2, 3, \ldots, 12\}$. For example, there is only one combination for getting a twelve, whereas for getting a seven there are six possible combinations. Hence, Alice will have to transmit symbol "7" about six times more often than the symbol "12" of the alphabet.

The key idea is to use shorter representations, i.e., less bits, for the most frequent symbols.

The Morse code (19th century) is a good example of this principle, recognized well before the foundations of the theory of information were established by Claude Shannon (1916–2001) in the mid of the 20th century:

- Claude Shannon, *A Mathematical Theory of Communication*, Bell System Technical Journal (Shannon, 1948):

  > *"The recent development of various methods of modulation such as PCM and PPM which exchange bandwidth for signal-to-noise ratio has intensified the interest in a general theory of communication. A basis for such a theory is contained in the important papers of Nyquist and Hartley on this subject. In the present paper we will extend the theory to include a number of new factors, in particular the effect of noise in the channel, and the savings possible due to the* ***statistical structure of the original message*** *and due to the nature of the final destination of the information."*

Discussion topic 4.1
The Morse code is composed of sequences of dots and dashes of various lengths, according to the convention shown in Fig. 4.1. In this code, the characters of a word need to be separated by additional space (three units of time, where one unit of time is equivalente to the length of a dot). For example, the letters "ET" originate the sequence "· —", to distinguish from the letter "A" that is encoded as "· —". Do you think this explicit separation could be avoided? If you think so, how should the code be constructed?

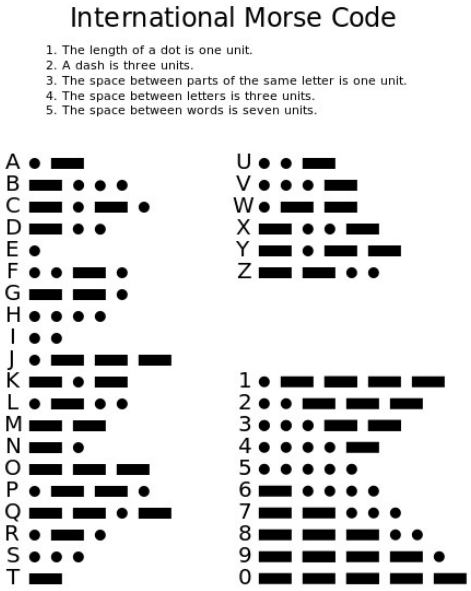


Figure 4.1: Morse code table.

4.1 About codes

Consider an alphabet with m symbols, $\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_m\}$, and the corresponding **estimated frequencies of occurrence** of each symbol, p_i. Of course,

$$\sum_{i=1}^{m} p_i = 1.$$

If symbol $\sigma_i \in \Sigma$ is represented by a codeword w_i, then the objective is to **minimize**

$$\sum_{i=1}^{m} p_i l_i, \tag{4.1}$$

where $l_i \in \mathbb{N}^+$ is the length of the codeword w_i, i.e., its number of bits. Often, we will represent the length of a binary string w_i by $|w_i|$, i.e., $l_i = |w_i|$.

Obviously, we need to impose additional constraints to the minimization of (4.1), otherwise we could arrive at completely useless solutions, such as $l_i = 1, \forall_i \ldots$

Discussion topic 4.2
Can you suggest restrictions that should be imposed in the minimization of (4.1)?

Of course, one of the restrictions should be that $w_i \neq w_j, \forall_{i \neq j}$, otherwise the code is not reversible, i.e., it is not possible to recover the original message. But this condition is not

enough for reversibility (can you see why?). In fact, we need a stronger condition: The code should be **uniquely decodable**. This implies that two different messages will always have two different encodings. Recall the encoding of "ET" and "A" using the Morse code of Fig. 4.1 and the need for additional space to separate the letters. Without it, two different messages would originate the same encoding, i.e., "· —".

Note that, for our purposes, an **encoding** is the assignment of binary strings to elements of the alphabet. Some encodings are of **fixed length** (the ASCII encoding is an obvious example), whereas other encodings are of **variable length**. To attain compression, we need the latter ones.

Problem 4.2
Find codes for which $w_i \neq w_j, \forall_{i \neq j}$, but that are not uniquely decodable.

A sufficient (but not necessary) condition for unique decodability is **immediate decodability**. This means that symbol σ_i is immediately determined as soon as the last (i.e., the rightmost) bit of w_i is read during decoding. For this to happen, the codewords need to be **prefix-free**, i.e., a shorter codeword cannot be a prefix of a longer codeword (why do we need this requirement?).

Problem 4.3
*Show that the Morse code is not a **prefix-free code** (Suggestion: try to build a decoding tree). What are the practical implications of this limitation of the Morse code?*

Letters	Probability	Code 1	Code 2	Code 3	Code 4
a_1	0.5	0	0	0	0
a_2	0.25	0	1	10	01
a_3	0.125	1	00	110	011
a_4	0.125	10	11	111	0111
Average length		1.125	1.25	1.75	1.875

(Sayood 2012)

Table 1: Example of several codes, with different properties.

Table 1 shows four different codes for a 4-symbol alphabet (although not all of them are useful...). They have different properties, namely:

- Code 1 is not uniquely decodable.
- Code 2 is also not uniquely decodable.
- Code 3 is uniquely decodable and instantaneous.

- Code 4 is uniquely decodable, but not instantaneous.

Problem 4.4
Consider now the following uniquely decodable, but not instantaneous, code:

Symbol	*Codeword*
σ_1	*0*
σ_2	*01*
σ_3	*11*

How to decode the string "01111111111111111"?
(Note that the first symbol can be either σ_1 or σ_2...)

Problem 4.5
Consider now the following code:

Symbol	*Codeword*
σ_1	*0*
σ_2	*01*
σ_3	*10*

How to decode the string "01010101010101010"?
(This code is neither instantaneous nor uniquely decodable)

Algorithm 4.1 (Unique decodability)
Suppose that we have two binary codewords a and b, with $k = |a|$, $n = |b|$, with $k < n$, and where a is a prefix of b. We refer to the last $n - k$ bits of b as the dangling suffix.

The algorithm for finding if a code is uniquely decodable is the following:

- *Construct a set with all the codewords.*
- *For every pair of codewords, generate the dangling suffix (if it exists) and add it to the set.*
- *Repeat until:*
  (a) You get a dangling suffix that is a codeword;
  (b) There are no more unique dangling suffixes.

If the algorithm terminates with condition (a), i.e., if it finds a dangling suffix that is a codeword, then the code is not uniquely decodable.

Example: $\{0, 01, 10\} \rightarrow \{0, 01, 10, 1\} \rightarrow \{0, 01, 10, 1, 0\}$.

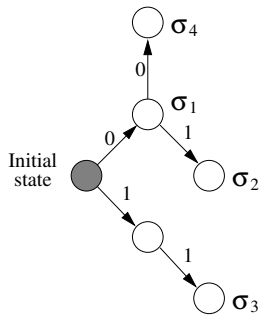


Figure 4.2: Example of a decoding tree of a non instantaneous (and also non uniquely decodable) code.

If the algorithm terminates with condition (b), i.e., if there are no more unique dangling suffixes, then the code is uniquely decodable.

Example: $\{0, 01, 11\} \rightarrow \{0, 01, 11, 1\}$.

As mentioned, if, in a certain encoding, none of the codewords is a prefix of another codeword, then the code is necessarily uniquely decodable. An easy way of checking if a code is prefix-free is to build the corresponding rooted binary tree. In a prefix-free code, the codewords are only associated with the external nodes of the tree.

Example 4.1

Consider an alphabet with symbols σ_1, σ_2, σ_3 and σ_4, and the following codeword assignment: $\sigma_1 \rightarrow 0, \quad \sigma_2 \rightarrow 01, \quad \sigma_3 \rightarrow 11, \quad \sigma_4 \rightarrow 00$.

If the decoder receives "0001", then it cannot determine if the encoder has sent the string "$\sigma_1\sigma_1\sigma_2$" or "$\sigma_4\sigma_2$". As can be seen using the corresponding decoding tree (Fig. 4.2), this code has several problems:

- *The path to node σ_3 is done by an intermediate node that does not contain a bifurcation: the code is inefficient.*
- *Node σ_1 is not a terminal node. The paths to nodes σ_2 and σ_4 include node σ_1: the code is not instantaneous (in fact, it is also not uniquely decodable, as can be seen using Algorithm 4.1).*

Example 4.2

Let us now consider that each branch of the tree connects to a terminal node or to a decision node that leads to a terminal node. Based on this restriction, we arrive at the following codeword assignment: $\sigma_1 \rightarrow 0, \quad \sigma_2 \rightarrow 10, \quad \sigma_3 \rightarrow 110, \quad \sigma_4 \rightarrow 111$. *As can be confirmed (see Fig. 4.3), this code is uniquely decodable (and also instantaneous).*

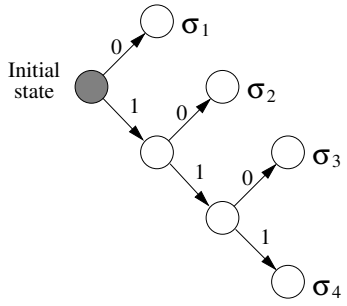


Figure 4.3: Example of a decoding tree of a uniquely decodable (an instantaneous) code. This code is prefix-free.

Another way of verifying if a code is prefix-free is to map the codewords in the unit interval, where each codeword occupies a non-overlapping fraction of length 2^{-l_i} of the unit interval (note that when a codeword of length k is used, a 2^{-k} fraction of the associated binary tree is "taken", because of the prefix-free condition).

Problem 4.6

Suppose that Alice wants to build a prefix-free code such that $l_1 = l_4 = 2$, $l_2 = l_3 = 3$ and $l_5 = 1$. Can you show to Alice that this is not possible? (Try both analogies: the binary tree and the unit interval covering)

Discussion topic 4.3

Given a set of lengths, l_i, in increasing order, discuss strategies for covering the unit interval as efficiently as possible. Can the covering be done as efficiently as previously even if the lengths are given in an arbitrary order?

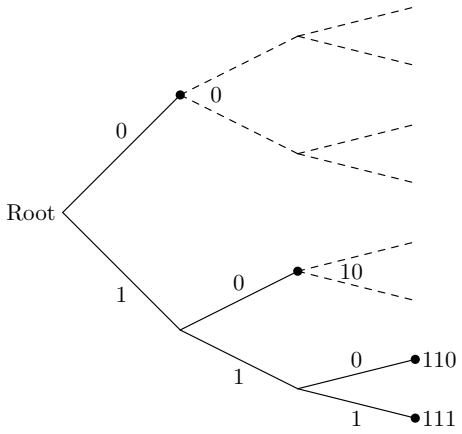


Figure 4.4: Tree associated to a prefix-free code.

Theorem 4.1 (Kraft inequality)
For any instantaneous (i.e., prefix-free) binary code, the codeword lengths $l_1, l_2, \ldots, l_m$ must satisfy the inequality

$$\sum_{i=1}^{m} 2^{-l_i} \leq 1.$$

Proof. Referring to the code tree of Fig. 4.4, consider all nodes at level $l_{\max}$ (the length of the longest codeword). A codeword at level l_i has $2^{l_{\max}-l_i}$ descendants at level $l_{\max}$. Because of the prefix-free condition, each of these descendant sets must be disjoint. Also, the total number of nodes in these sets must be less than or equal to $2^{l_{\max}}$. Therefore, summing over all the codewords, we have $\sum 2^{l_{\max}-l_i} \leq 2^{l_{\max}}$ or $\sum 2^{-l_i} \leq 1$. □

Theorem 4.2 (Counterpart of the Kraft inequality)
If a set of lengths $l_1, l_2, \ldots, l_m$ satisfy the Kraft inequality, then there exists an instantaneous code with these word lengths.

Proof. Construct a tree of depth $l_{\max}$. Label the first node (lexicographically) of depth l_1 as codeword 1 and remove its descendants from the tree. Then label the first remaining node of depth l_2 as codeword 2, and so on. Proceeding this way, we construct a prefix code with the specified $l_1, l_2, \ldots, l_m$. □

4.2 Definition of information according to Shannon

The definition of **information** according to Shannon is associated with the probability of certain events (the symbols). Consider E an event, i.e., a set of outcomes of some random experiment. If $P(E)$ is the probability of event E to occur, then the information (also called by Shannon "self-information") associated with the occurrence of E is given by

$$i(E) = \log_b \frac{1}{P(E)} = -\log_b P(E).$$

Recall that $\log(1) = 0$ and that $-\log(x)$ increases as x decreases from one to zero. Hence, an event with probability one does not carry any information. On the other hand, as the probability of the event approaches zero, the information that can be associated with that event approaches infinity.

Moreover, the information associated with the occurrence of two **independent** events, E_1 and

E_2,[12] is $i(E_1, E_2) = i(E_1) + i(E_2)$, because

$$i(E_1, E_2) = -\log_b P(E_1, E_2) = -\log_b P(E_1)P(E_2) =$$
$$= -\log_b P(E_1) - \log_b P(E_2) = i(E_1) + i(E_2).$$

As mentioned before, the unit of information depends on the base of the logarithm, b: if $b = 2$, we measure information in bits; if $b = e$, in nats; if $b = 10$, in hartleys. From now on, when the base of the logarithm is not specified, we assume $b = 2$.

For a certain partition of the sample space, S, in E_j sets, i.e., if

$$\bigcup_j E_j = S \quad \text{and} \quad E_j \cap E_k = \emptyset, \forall j \neq k,$$

the associated **average self-information** is given by

$$H = \sum_j P(E_j) i(E_j) = -\sum_j P(E_j) \log P(E_j),$$

more usually called **entropy**. Note that the entropy is completely defined by a probability distribution—hence the name "**probabilistic**" in this definition of information.

Example 4.3
Consider tossing one fair coin. In this case, we have:

- *Two symbols (i.e.,* $\Sigma = \{heads, tails\}$*),* $P_i = 0.5$
- $H = -(0.5 \log 0.5 + 0.5 \log 0.5) = 1$
  $\Longrightarrow$ *One bit for each trial*
- $R = \log |\Sigma| - H = \log 2 - 1 = 0$
  This is called ***redundancy*** *and measures the* ***difference between the combinatorial and probabilistic information measures****.*

Example 4.4
The case of two fair and independent coins:

- *Four symbols,* $P_i = 0.25$
- $H = -\sum_{i=1}^{4} P_i \log P_i = 2$ *bits*

[12] Recall that if E_1 and E_2 are independent, then $P(E_1, E_2) = P(E_1)P(E_2)$.

- $R = \log|\Sigma| - H = \log 4 - 2 = 0$

Example 4.5
Two independent, but biased, coins:

- *Four symbols,* $P_1 = 0.5625$*,* $P_2 = P_3 = 0.1875$ *and* $P_4 = 0.0625$
- $H \approx 1.62$ *bits*
- $R = \log|\Sigma| - H = \log 4 - 1.62 = 0.38$
- *In this case, representing each symbol, on average requires only* 1.62 *bits.*
- *This means that, for example,* 1000 *outcomes of the process can be represented by approximately* 1620 *bits, instead of* 2000 *bits, as suggested by the combinatorial definition of information.*
- *As already mentioned, this data reduction can be obtained using* ***variable-length codes****.*

Shannon demonstrated that, if an **information source** produces events with probabilities $P(E_i)$, where $\{E_i\}$ is a partition of S, then, **on average**, it is not possible to use less than H bits to represent each event.

Usually, and for simplicity, we label the events E_i with symbols from an alphabet Σ which is often a subset of the integers, for example, $\Sigma = \{1, 2, \ldots, m\}$. However, for the sake of generality, we will use the more generic $\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_m\}$ alphabet.

Let us assume that an information source produces sequences of symbols $\mathbf{X} = \{X_1, X_2, \ldots\}$,[13] with $X_i \in \Sigma$. The nth-order entropy, H_n, is defined as

$$H_n = -\frac{1}{n} \sum_{x_1=\sigma_1}^{\sigma_m} \cdots \sum_{x_n=\sigma_1}^{\sigma_m} P(X_{k+1} = x_1, \ldots, X_{k+n} = x_n) \log P(X_{k+1} = x_1, \ldots, X_{k+n} = x_n),$$

where $\{X_{k+1}, X_{k+2}, \ldots, X_{k+n}\}$ are sub-sequences of length n generated by the information source. Shannon showed that, for a **stationary source**,

$$H = \lim_{n\to\infty} H_n,$$

where H is the **entropy of the source**. Notice that, if the events are **independent and identically distributed** (i.i.d.), then

$$H = H_1 = H_n = -\sum_{x=\sigma_1}^{\sigma_m} P(X_k = x) \log P(X_k = x), \quad \forall_k, \tag{4.2}$$

i.e., the entropy and the 1st-order entropy of the source are the same.

[13] $\mathbf{X}$ is a discrete-time random process and, therefore, the X_i are random variables.

Problem 4.7
Show that (4.2) holds $\forall_n$, if the events are independent and identically distributed.

Generally, the entropy of an information source is not known. Therefore, we have to compute **estimates** for it.

4.3 Source modeling

Consider the following sequence of symbols, produced by a certain information source:

```
1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10
```

If we take it as a "good" representation of the statistics of the information source, then we can estimate the probabilities of the symbols as

$$P(1) = P(6) = P(7) = P(10) = 1/16$$
$$P(2) = P(3) = P(4) = P(5) = P(8) = P(9) = 2/16.$$

Assuming i.i.d., the entropy of this source is 3.25 bits.

However, if we transform the original sequence,

```
1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10
```

into

```
1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1 1
```

we now have only two different symbols (-1 and 1), for which $P(1) = 13/16$ and $P(-1) = 3/16$. Now, the entropy is only 0.7 bits...

Discussion topic 4.4
We have seen that the entropy is a measure of the average of information produced by a source. According to the example above, apparently it is possible to change the amount of information produced by a source, using a reversible transformation (show that the transformation used above is, in fact, reversible).

Do you think this is indeed possible? [14]

Let us denote by $x_1^n = x_1 x_2 \dots x_n,\ x_i \in \Sigma$, the sequence of outputs (symbols from the source alphabet Σ) that an information source has generated until instant n. Often, we refer to x_1^n (or

[14] In fact, it is not! So, what is wrong in this reasoning?

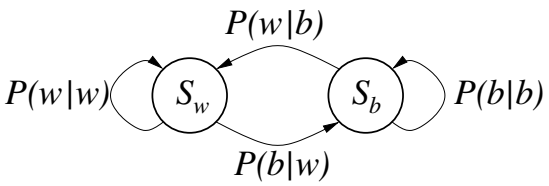


Figure 4.5: Example of a transition diagram of a Markov model for a binary image.

simply x, if size can be omitted) as a string over the alphabet Σ. The transformation that was performed in the example above corresponds to the observation that the relation

$$x_n = x_{n-1} + r_n$$

provides a "satisfactory" **model** for this information source. In that case, only the values of r_n need to be encoded. This model is called **static**, because it does not change with n. Otherwise, the model is called **adaptive**.

Discussion topic 4.5
How do we know that a certain model is "satisfactory"? For example, why is it resonable to say that the model above is "satisfactory"?

Correctly modeling an information source (and, therefore, **finding the underlying structure of the data**) is one of the most important aspects in data compression.

One of the most used aproaches for representing data dependencies relies on the use of **Markov models**. In lossless data compression, we use a specific type, called discrete time Markov chain or **finite-context model**.

A k-order Markov model verifies

$$P(x_n|x_{n-1}\dots x_{n-k}) = P(x_n|x_{n-1}\dots x_{n-k}\dots),$$

where the string $c = x_{n-1}\dots x_{n-k}$ is called the state or **context** of the process.

A 1st-order Markov model reduces to

$$P(x_n|x_{n-1}) = P(x_n|x_{n-1}x_{n-2}\dots).$$

As an example, consider that we want to model a binary image. We have two states, S_w (white pixel) and S_b (black pixel), and four possible transitions, namely $S_w \to S_w$, $S_w \to S_b$, $S_b \to S_w$, $S_b \to S_b$. The state transition diagram of this 1st-order model is shown in Fig. 4.5.

The entropy of a process with N states S_i is simply the average value of the entropy of each state, i.e.,

$$H = \sum_{i=1}^{N} P(S_i)H(S_i),$$

where $H(S_i)$ denotes the entropy of state S_i and P_i is the probability of occurrence of state S_i. Therefore, in our example,

$$H(S_w) = -P(b|w)\log P(b|w) - P(w|w)\log P(w|w).$$

Example 4.6
Consider a 1st-order model with

$$P(w|b) = 0.3 \quad \text{and} \quad P(b|w) = 0.01.$$

The state probabilities can be calculated according to

$$P(S_w) = \frac{P(w|b)}{P(w|b) + P(b|w)} \approx 0.968$$

and

$$P(S_b) = \frac{P(b|w)}{P(w|b) + P(b|w)} \approx 0.032.$$

Therefore, considering independence in the occurrence of symbols, we have,

$$H = -0.968\log_2 0.968 - 0.032\log_2 0.032 \approx 0.204 \textit{ bps}$$

However, considering the 1st-order Markov model, we obtain an estimate of the entropy of the source that is

$$H(S_b) = -0.3\log_2 0.3 - 0.7\log_2 0.7 \approx 0.881 \textit{ bps}$$
$$H(S_w) = -0.01\log_2 0.01 - 0.99\log_2 0.99 \approx 0.081 \textit{ bps}$$
$$H = 0.968 \times 0.081 + 0.032 \times 0.881 \approx 0.107 \textit{ bps}$$

Markov models are particularly useful in text compression, because the next letter in a word is generally heavily influenced by the preceding letters. In fact, the use of Markov models for written English appeared in the original work of Shannon. In 1951, he estimated the entropy of English to be in between about 0.6 and 1.3 bits per letter.

For simplicity, consider only 26 letters and the space character. The frequency of letters in English is far from uniform: the most common, "E", occurs about 13%, whereas the least common, "Q" and "Z", occur about 0.1% of the time. The frequency of pairs of letters is also nonuniform. For example, the letter "Q" is always followed by a "U". The most frequent pair is "TH" (occurs about 3.7%).

We can use the frequency of the pairs to estimate the probability that a letter follows any other letter. This reasoning can also be used for constructing higher-order models. However, the size of the model grows exponentially. For example, to build a third-order Markov model we need to estimate the values of $p(x_n|x_{n-1}x_{n-2}x_{n-3})$. This requires a table with $27^4 = 531\,441$ entries and enough text to correctly estimate the probabilities.

The following examples have been constructed using empirical distributions collected from samples of text (Shannon, 1948, 1951).

Example 4.7 (Zero-order approximation (Independent and equiprobable symbols))

XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGXYD QPAAMKBZAACIBZLHJQD

Entropy: $\log 27 = 4.76$ *bits per letter.*

Example 4.8 (First-order approximation (Independent, but with the correct $p(x)$))

OCRO HLI RGWR NMIELWIS EU LL NBNESEBYA TH EEI ALHENHTTPA OOBTTVA NAH BRL

Estimated entropy: ≈ 4.03 *bits per letter.*

Example 4.9 (Second-order approximation (1st-order Markov model, i.e., $p(x_n|x_{n-1})$))

ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TUCOOWE AT TEASONARE FUSO TIZIN ANDY TOBE SEACE CTISBE

Estimated entropy: ≈ 3.32 *bits per letter.*

Example 4.10 (Third-order approximation (2nd-order Markov model, i.e., $p(x_n|x_{n-1}x_{n-2})$))

IN NO IST LAT WHEY CRATICT FROURE BERS GROCID PONDENOME OF DEMONSTURES OF THE REPTAGIN IS REGOACTIONA OF CRE

Estimated entropy: ≈ 3.1 *bits per letter (excluding spaces).*

Example 4.11 (First-order word approximation)

REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE

Example 4.12 (Second-order word approximation)

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED

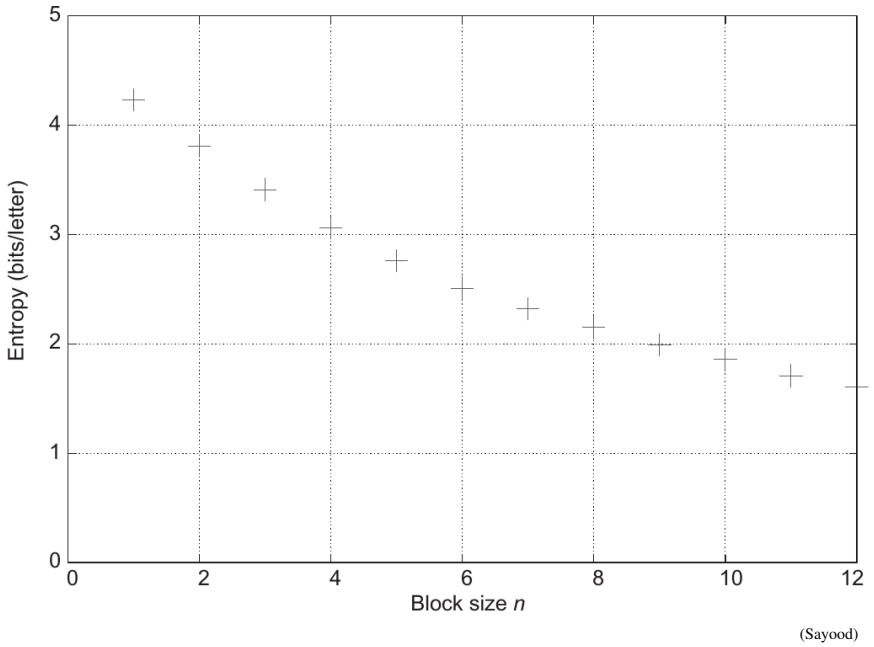


Figure 4.6: Estimated value of the entropy of a text using serveral orders.

Figure 4.6 shows the evolution of the estimate of the entropy of a book when the order of the model increaces.

The number of different contexts (or states of the Markov process) is given by $|\Sigma|^k$, where k is the order of the model, which clearly grows exponentially with the size of the alphabet. For example, in a 2-symbol alphabet, a finite-context model of depth 4 originates only $2^4 = 16$ states or different contexts. However, if the alphabet is of size 256 (for example, if we consider a gray-level image), then the number of contexts jumps to $256^4 = 4\,294\,967\,296$!

Often, the finite-context models are "learned" online, i.e., as the sequence of symbols is processed (note that in Example 4.6 the model is static, i.e., the probabilities are considered fixed during the operation of the model). In the case of online operation, since the probabilistic models are continuously adapting, after processing the first n symbols of x, the average number of bits associated with an order-k finite-context model is given by

$$H_n = -\frac{1}{n}\sum_{i=1}^{n} \log P(x_i|x_{i-k}^{i-1}). \tag{4.3}$$

Problem 4.8
Verify (4.3) and discuss the practical implications of x_{n-k}^{n-1} when $n < k$ (note that the convention is that $x_n^n = x_n$) and propose solutions to handle it.

In the case of online learning of the finite-context model, a table (or some other more sophisticated data structure, such as an hash-table) is used to collect counts that represent the number

of times that each symbol occurs in each context. For example, suppose that, in a certain moment, a binary ($|\Sigma| = 2$) source is modeled by an order-3 finite-context model represented by the table

x_{i-3}	x_{i-2}	x_{i-1}	n_0	n_1
0	0	0	10	25
0	0	1	4	12
0	1	0	15	2
0	1	1	3	4
1	0	0	34	78
1	0	1	21	5
1	1	0	17	9
1	1	1	0	22

where n_i indicates how many times symbol "i" occurred following that context. Therefore, in this case, we may estimate the probability that a symbol "0" follows the string "100" as being $34/(34 + 78) \approx 0.30$.

This way of estimating probabilities, based only on the relative frequencies of previously occurred events, suffers from the problem of assigning probability zero to events that were not (yet) seen. That would be the case, in this example, if we used the same approach to estimate the probability of having a "0" following a string of three or more "1s".

Problem 4.9
Discuss the potential implications of the "zero-probability" problem and give suggestions to avoid it.

In 1774, Laplace (1749–1827) proposed a rule for estimating the probability that a given event succeeds, after occurring n_1 successes and n_2 failures in $n = n_1 + n_2$ trials, as

$$\frac{n_1 + 1}{n + 2}.$$

This formula is in fact correct if the trials are independent and if an uniform prior over the estimated probability is considered.

For the more general case of a beta prior, $B(\alpha_1, \alpha_2)$, the estimator adopts the form (G. F. Hardy, 1889)

$$\frac{n_1 + \alpha_1}{n + \alpha_1 + \alpha_2}.$$

The multinomial generalization of the formula leads to

$$\frac{n_i + \alpha}{n + \alpha|\Sigma|},$$

where, for simplicity, we consider $\alpha = \alpha_1 = \cdots = \alpha_{|\Sigma|}$, and the α_i are the parameters of the Dirichlet family of distributions.

A worth noting aspect of this estimator is that, defining

$$\mu = \frac{n}{n + \alpha|\Sigma|},$$

it can be written as

$$\mu \frac{n_i}{n} + (1 - \mu)\frac{1}{|\Sigma|},$$

showing how it evolves from a uniform estimator to a frequency estimator, as n increases.

4.4 Some more on Shannon entropy

4.4.1 Entropy, joint entropy and conditional entropy

As already shown, given a probability distribution, we may define a quantity called **entropy**, that has many properties that agree with our intuitive notion of what a measure of information should be. We have seen that the entropy expresses the **amount of uncertainty** about a certain random variable. It measures the **amount of information** required **on average** to describe the random variable.

This notion can be extended to define **mutual information**, which is a measure of the amount of information that one random variable contains about another. In fact, the mutual information is a special case of a more general quantity called **relative entropy**, which gives a measure of the "distance" between two probability distributions.

Consider X a discrete random variable that takes values in a finite alphabet $\Sigma = \{\sigma_1, \ldots, \sigma_m\}$ and has a probability mass function $p(x) = P(X = x)$, $x \in \Sigma$.

The entropy of X can be interpreted as the **expected value** of the random variable $\log \dfrac{1}{p(X)}$, where X is drawn according to $p(x)$. Therefore,

$$H(X) = E\left[\log \frac{1}{p(X)}\right] = E[-\log p(X)],$$

where $E[g(X)]$ denotes the expected value of $g(X)$, defined as

$$E[g(X)] = \sum_{x \in \Sigma} g(x)p(x).$$

Example 4.13
Let X be a binary random variable with alphabet $\Sigma = \{0, 1\}$ and probability mass function $P(X = 1) = p$ and $P(X = 0) = 1 - p$. Then, the entropy of X is $H(X) = -p \log p - (1 - p) \log(1 - p)$ (see Fig. 4.7).

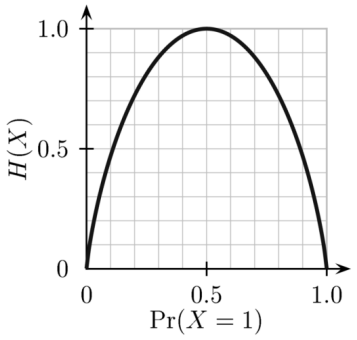


Figure 4.7: Entropy curve for a binary alphabet.

Theorem 4.3
$0 \leq H(X) \leq \log|\Sigma|$, *where* $|\Sigma|$ *denotes the cardinality of* Σ. *Also,* $H(X) = 0$ *iff* $p(x) = 1$ *for some* $x \in \Sigma$, *and* $H(X) = \log|\Sigma|$ *iff* $p(x) = 1/|\Sigma|$.

Proof of $H(X) \geq 0$. Since each $p(x) \leq 1$, then each term $p(x)\log p(x) \leq 0$, implying $H(X) \geq 0$. Also, $H(X) = 0$ requires that $p(x)\log p(x) = 0, \forall x \in \Sigma$, implying $p(x) = 1$ or $p(x) = 0$.

Therefore, $H(X) = 0$ iff $p(x) = 1$ for some $x \in \Sigma$ (and, obviously, $p(x) = 0$ for the other x). □

Proof of $H(X) \leq \log|\Sigma|$. Consider the inequality $\ln x \leq x - 1$ and the subset $\Sigma' \subset \Sigma$ such that $p(x) > 0, \forall x \in \Sigma'$. Then

$$H(X) - \log|\Sigma'| = \sum_{x\in\Sigma'} p(x)\log\frac{1}{p(x)} - \sum_{x\in\Sigma'} p(x)\log|\Sigma'| =$$

$$= \frac{1}{\ln 2}\sum_{x\in\Sigma'} p(x)\ln\frac{1}{|\Sigma'|p(x)} \leq \frac{1}{\ln 2}\sum_{x\in\Sigma'} p(x)\left[\frac{1}{|\Sigma'|p(x)} - 1\right] =$$

$$= \frac{1}{\ln 2}\left[\sum_{x\in\Sigma'}\frac{1}{|\Sigma'|} - \sum_{x\in\Sigma'} p(x)\right] = 0.$$

The rest of the proof is left as exercise. □

We can extend the definition of entropy to a pair of random variables (X, Y) with joint distribution $p(x, y) = P(X = x, Y = y)$. For simplicity, we will assume that both random variables are drawn from the same alphabet Σ.

Definition 4.1 (Joint entropy)
The joint entropy $H(X,Y)$ of a pair of discrete random variables (X,Y), with a joint distribution $p(x,y)$, is defined as

$$H(X,Y) = -\sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x,y) = E[-\log p(X,Y)].$$

Note that if $p(x,y) = p(x)p(y)$ (i.e., if X and Y are independent random variables), then $H(X,Y) = H(X) + H(Y)$. On the other hand, if X and Y are dependent, then $H(X,Y) < H(X) + H(Y)$.

Definition 4.2 (Conditional entropy)
For a pair of discrete random variables (X,Y), having a joint distribution $p(x,y)$, we define conditional entropy $H(Y|X)$ as

$$H(Y|X) = \sum_{x\in\Sigma} p(x)H(Y|X=x) = -\sum_{x\in\Sigma} p(x)\sum_{y\in\Sigma} p(y|x)\log p(y|x) =$$

$$= -\sum_{(x,y)\in\Sigma^2} p(x,y)\log p(y|x) = E[-\log p(Y|X)].$$

Theorem 4.4 (Chain rule for the entropy of a pair of random variables)

$$H(X,Y) = H(X) + H(Y|X).$$

Proof.

$$H(X,Y) = -\sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x,y) = -\sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x)p(y|x) =$$

$$= -\sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x) - \sum_{(x,y)\in\Sigma^2} p(x,y)\log p(y|x) =$$

$$= -\sum_{x\in\Sigma} p(x)\log p(x) + H(Y|X) = H(X) + H(Y|X).$$

□

Example 4.14

Consider the following joint distribution of (X, Y)*:*

$Y \backslash X$	1	2	3	4
1	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$	$\frac{1}{32}$
2	$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{32}$	$\frac{1}{32}$
3	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$
4	$\frac{1}{4}$	0	0	0

The marginal distributions of X *and* Y *are, respectively,* $(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$ *and* $(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4})$*. Hence,* $H(X) = 1.75$ *bits and* $H(Y) = 2$ *bits. Also,*

$$H(X|Y) = \sum_{i=1}^{4} P(Y=i)H(X|Y=i) = \frac{1}{4}H\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}\right) +$$

$$\frac{1}{4}H\left(\frac{1}{4}, \frac{1}{2}, \frac{1}{8}, \frac{1}{8}\right) + \frac{1}{4}H\left(\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right) + \frac{1}{4}H(1,0,0,0) = 1.375 \text{ bits.}$$

Similarly, $H(Y|X) = 1.625$ *bits and* $H(X,Y) = 3.375$ *bits.*

Note that, generally, $H(Y|X) \neq H(X|Y)$, but that $H(X) - H(X|Y) = H(Y) - H(Y|X)$.

4.4.2 Relative entropy

The relative entropy is a measure of the "distance" between two distributions. It expresses the inefficiency of assuming that the distribution is $q(x)$ when the true distribution is $p(x)$.

Definition 4.3 (Relative entropy)
The relative entropy or Kullback-Leibler distance between two probability mass functions, $p(x)$ *and* $q(x)$*, is defined as*

$$D(p\|q) = \sum_{x\in\Sigma} p(x) \log \frac{p(x)}{q(x)} = E_p\left[\log \frac{p(X)}{q(X)}\right].$$

Note that we consider the conventions: $0 \log \frac{0}{0} = 0$, $0 \log \frac{0}{q} = 0$ and $p \log \frac{p}{0} = \infty$.

Example 4.15
Let $\Sigma = \{0, 1\}$ *and consider two distributions,* $p(x)$ *and* $q(x)$*, on the elements of* Σ*. Let* $p(0) = 1 - r$*,* $p(1) = r$*,* $q(0) = 1 - s$ *and* $q(1) = s$*. Then*

$$D(p\|q) = (1-r) \log \frac{1-r}{1-s} + r \log \frac{r}{s}$$

and

$$D(q\|p) = (1-s)\log\frac{1-s}{1-r} + s\log\frac{s}{r}$$

If $r = 1/2$ *and* $s = 1/4$*, then* $D(p\|q) = 0.208$ *bits,* $D(q\|p) = 0.189$ *bits.*

Note that, in general, $D(p\|q) \neq D(q\|p)$, and, if $p(x) = q(x)$, then $D(p\|q) = D(q\|p) = 0$.

4.4.3 Mutual information

The mutual information expresses the amount of information that one random variable contains about another rv.

Definition 4.4 (Mutual information)
Consider two random variables X *and* Y*, with joint probability mass function* $p(x,y)$ *and marginal functions* $p(x)$ *and* $p(y)$*.*

The mutual information, $I(X;Y)$*, can be defined as the relative entropy between the joint distribution and the product* $p(x)p(y)$*, i.e.,*

$$I(X;Y) = D(p(x,y)\|p(x)p(y)) =$$

$$= \sum_{(x,y)\in\Sigma^2} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} = E_{p(x,y)}\left[\log\frac{p(X,Y)}{p(X)p(Y)}\right].$$

Also, the mutual information indicates the reduction in the uncertainty of one random variable due to the knowledge of the other. In fact, we can write

$$I(X;Y) = \sum_{(x,y)\in\Sigma^2} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} = \sum_{(x,y)\in\Sigma^2} p(x,y)\log\frac{p(x|y)}{p(x)} =$$

$$= -\sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x) + \sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x|y) =$$

$$= -\sum_{x\in\Sigma} p(x)\log p(x) + \sum_{(x,y)\in\Sigma^2} p(x,y)\log p(x|y) =$$

$$= H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y).$$

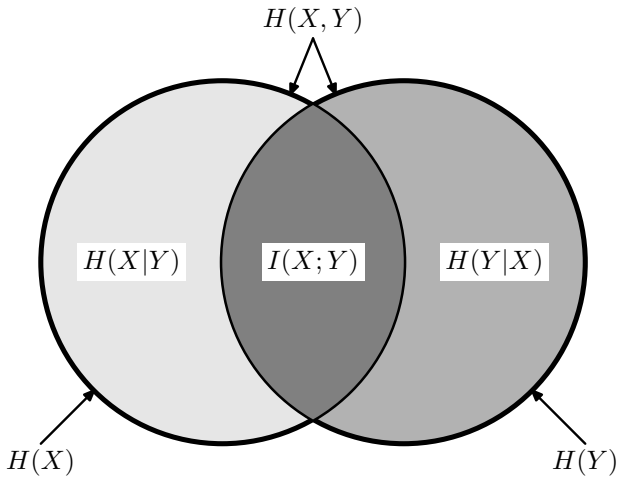


Figure 4.8: Some relations between entropy and mutual information.

Example 4.16
Considering again the joint distribution of (X, Y)*,*

$Y \backslash X$	1	2	3	4
1	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$	$\frac{1}{32}$
2	$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{32}$	$\frac{1}{32}$
3	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$
4	$\frac{1}{4}$	0	0	0

where $H(X) = 1.75$ *bits,* $H(Y) = 2$ *bits,* $H(X|Y) = 1.375$ *bits and* $H(Y|X) = 1.625$*, we obtain a mutual information between* X *and* Y *of*

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = 0.375 \quad \text{bits.}$$

Theorem 4.5 (Relations between the mutual information and the entropy)
The following relations between the mutual information of a pair of random variables, X *and* Y*, and the corresponding entropies hold (see also Fig. 4.8):*

$$\begin{aligned}
I(X;Y) &= H(X) - H(X|Y) \\
I(X;Y) &= H(Y) - H(Y|X) \\
I(X;Y) &= H(X) + H(Y) - H(X,Y) \\
I(X;Y) &= I(Y;X) \\
I(X;X) &= H(X) - H(X|X) = H(X).
\end{aligned}$$

The mutual information of a random variable with itself is the entropy of the random variable (verify this). This is why the entropy is sometimes referred to as **self-information**.

Theorem 4.6 (Chain rule for the entropy)
Let $X_1, X_2, \ldots, X_n$ be a set of random variables drawn according to $p(x_1, x_2, \ldots, x_n)$. Then

$$H(X_1, X_2, \ldots, X_n) = \sum_{i=1}^{n} H(X_i|X_{i-1}, \ldots, X_1).$$

Proof.

$$H(X_1, X_2) = H(X_1) + H(X_2|X_1),$$

$$H(X_1, X_2, X_3) = H(X_1) + H(X_2, X_3|X_1) = H(X_1) + H(X_2|X_1) + H(X_3|X_2, X_1),$$

$$\vdots$$

$$H(X_1, X_2, \ldots, X_n) = H(X_1) + H(X_2|X_1) + \cdots + H(X_n|X_{n-1}, \ldots, X_1).$$

□

Definition 4.5 (Conditional mutual information)
The conditional mutual information of the random variables X and Y given Z is defined as

$$I(X;Y|Z) = H(X|Z) - H(X|Y,Z) = E_{p(x,y,z)}\left[\log \frac{p(X,Y|Z)}{p(X|Z)p(Y|Z)}\right].$$

(Remember that $I(X;Y) = H(X) - H(X|Y)$)

Theorem 4.7 (Chain rule for the information)

$$I(X_1, X_2, \ldots, X_n; Y) = \sum_{i=1}^{n} I(X_i; Y|X_{i-1}, X_{i-2}, \ldots, X_1).$$

Proof.

$$\begin{aligned} I(X_1, X_2, \ldots, X_n; Y) &= H(X_1, X_2, \ldots, X_n) - H(X_1, X_2, \ldots, X_n|Y) = \\ &= \sum_{i=1}^{n} H(X_i|X_{i-1}, \ldots, X_1) - \sum_{i=1}^{n} H(X_i|X_{i-1}, \ldots, X_1, Y) = \\ &= \sum_{i=1}^{n} I(X_i; Y|X_{i-1}, X_{i-2}, \ldots, X_1). \end{aligned}$$

□

Theorem 4.8 (Jensen's inequality)
If $f(x)$ is a convex function and X is a random variable,

$$E[f(X)] \geq f(E[X]).$$

Moreover, if $f(x)$ is strictly convex, the equality implies that $X = E[X]$ with probability 1, i.e., X is constant.

Theorem 4.9 (Information inequality)
Let $p(x)$ and $q(x)$ be two probability mass functions, with $x \in \Sigma$. Then, with equality iff $p(x) = q(x), \forall x$,

$$D(p\|q) \geq 0.$$

Proof of the non-negativity. Let $\Sigma' = \{x|p(x) > 0\}$ be the support set of $p(x)$. Then

$$-D(p\|q) = \sum_{x\in\Sigma'} p(x) \log \frac{q(x)}{p(x)} \leq \quad \text{(by Jensen inequality)}$$

$$\leq \log \sum_{x\in\Sigma'} p(x)\frac{q(x)}{p(x)} = \log \sum_{x\in\Sigma'} q(x) \leq \log \sum_{x\in\Sigma} q(x) = \log 1 = 0,$$

i.e., $D(p\|q) > 0$. □

Proof that $D(p\|q) = 0$ iff $p = q$. Since $\log x$ is a strictly concave function of x, we have

$$\sum_{x\in\Sigma'} p(x) \log \frac{q(x)}{p(x)} = \log \sum_{x\in\Sigma'} p(x)\frac{q(x)}{p(x)}$$

iff $q(x)/p(x)$ is constant everywhere (i.e., if $q(x) = cp(x), \forall x$). Therefore,

$$\sum_{x\in\Sigma'} q(x) = c \sum_{x\in\Sigma'} p(x) = c.$$

Moreover, it is required that $\sum_{x\in\Sigma'} q(x) = 1$ for having

$$\log \sum_{x\in\Sigma'} q(x) = \log \sum_{x\in\Sigma} q(x),$$

implying $c = 1$. Hence, we have $D(p\|q) = 0$ iff $p(x) = q(x), \forall x$. □

Corollary 4.9.1 (Nonnegativity of mutual information)
For any two random variables, X and Y,

$$I(X;Y) \geq 0,$$

with equality iff X and Y are independent.

Proof.

$$I(X;Y) = D(p(x,y)\|p(x)p(y)) \geq 0,$$

with equality iff $p(x,y) = p(x)p(y)$, i.e., when X and Y are independent. □

Theorem 4.10 (Upper bound on the entropy)

$$H(X) \leq \log |\mathcal{X}|,$$

with equality iff X has uniform distribution over $\mathcal{X}$.

Proof. Let $u(x) = 1/|\Sigma|$ be the uniform probability mass function over Σ, and let $p(x)$ be the probability mass function for X. Then,

$$D(p\|u) = \sum_{x\in\Sigma} p(x) \log \frac{p(x)}{u(x)} = \log |\Sigma| - H(X) \geq 0.$$

Therefore, $H(X) \leq \log |\Sigma|$, with equality iff $p(x) = u(x)$. □

Theorem 4.11 (Conditioning reduces entropy)

$$H(X|Y) \leq H(X)$$

with equality iff X and Y are independent.

Proof.

$$0 \leq I(X;Y) = H(X) - H(X|Y).$$

□

This theorem states that knowing another random variables, Y, can only reduce the uncertainty in X. However, this is only true **on average**. Specifically, $H(X|Y = y)$ may be greater than or less than or equal to $H(X)$, but, on average, $H(X|Y) = \sum_y p(y)H(X|Y = y) \leq H(X)$.

Example 4.17
Let (X, Y) have the following joint distribution,

$Y\backslash X$	1	2
1	0	$\frac{3}{4}$
2	$\frac{1}{8}$	$\frac{1}{8}$

Then, $H(X) = H(\frac{1}{8}, \frac{7}{8}) \approx 0.544$ *bits,* $H(X|Y=1) = 0$ *and* $H(X|Y=2) = 1$ *bit. However,*

$$H(X|Y) = \frac{3}{4}H(X|Y=1) + \frac{1}{4}H(X|Y=2) = 0.25 \quad \text{bits.}$$

Thus, the uncertainty in X *increases if* $Y=2$ *is observed and decreases if* $Y=1$*. Nevertheless, on average, the uncertainty decreases.*

Theorem 4.12 (Independence bound on the entropy)
Let $X_1, X_2, \ldots, X_n$ *be drawn according to* $p(x_1, x_2, \ldots, x_n)$*. Then,*

$$H(X_1, X_2, \ldots, X_n) \leq \sum_{i=1}^{n} H(X_i),$$

with equality iff the X_i *random variables are independent.*

Proof. By the chain rule for entropies and using the fact that conditioning reduces the entropy,

$$H(X_1, X_2, \ldots, X_n) = \sum_{i=1}^{n} H(X_i|X_{i-1}, \ldots, X_1) \leq \sum_{i=1}^{n} H(X_i),$$

with equality iff the X_i are independent. □

---

5 Data compression

The main purpose of a data compression algorithm is to reduce the size of the data necessary to represent a certain amount of information. In this chapter, we address some of the most important approaches to data compression, characterized by allowing the total recovery of the original data, i.e., without losing any information. These are know as **lossless data compression algorithms**.

5.1 Variable-length coding

5.1.1 Optimal codes

For a given source distribution p_i, we want to find the prefix-free code with the minimum expected length, i.e., we want to minimize

$$L = \sum p_i l_i,$$

over all integers $l_1, l_2, \ldots, l_m$ satisfying the Kraft inequality, i.e.,

$$\sum 2^{-l_i} \leq 1.$$

By relaxing the integer constraint on l_i, we obtain the optimal code lengths, l_i^*, using an optimization method[15], that gives

$$l_i^* = -\log p_i.$$

This noninteger choice of codeword lengths yields an average codeword length

$$L^* = \sum p_i l_i^* = -\sum p_i \log p_i = H.$$

Theorem 5.1
The average length L of any instantaneous binary code for a source distribution p_i is greater than or equal to the entropy H, i.e.,

$$L \geq H,$$

with equality iff $2^{-l_i} = p_i$.

Proof.

$$H - L = -\sum p_i \log p_i - \sum p_i l_i = \sum p_i \log 2^{-l_i} - \sum p_i \log p_i =$$

$$= \sum p_i \log \frac{2^{-l_i}}{p_i} \leq \log \left(\sum p_i \frac{2^{-l_i}}{p_i} \right) = 0,$$

where the inequality results from the use of Jensen's inequality. □

[15]Consider, for example, that $l_i = -\log q_i$. Using Lagrange multipliers to solve this contrained optimization problem, we have to find the gradient of $-\sum p_i \log q_i + \lambda(\sum q_i - 1)$.

As shown, the choice of word lengths $l_i = -\log p_i$ yields $L = H$. However, in practice, these values may not be integers. Therefore, we might want to construct a code by rounding up the $-\log p_i$ lengths, i.e.,

$$l_i = \lceil -\log p_i \rceil.$$

Note that these lengths satisfy the Kraft inequality, because

$$\sum 2^{-\lceil -\log p_i \rceil} \leq \sum 2^{-(-\log p_i)} = \sum p_i = 1.$$

Moreover, we can write

$$-\log p_i \leq l_i < -\log p_i + 1 \;\Rightarrow\; H \leq L < H + 1.$$

Note that the optimal code can only be better than this code.

Theorem 5.2
Let $l_1^*, l_2^*, \ldots, l_m^*$ be the optimal codeword lengths for a source distribution p_i and let L^* be the associated expected length of an optimal code, i.e., $L^* = \sum p_i l_i^*$. Then

$$H \leq L^* < H + 1.$$

Proof. We have seen that if $l_i = \lceil -\log p_i \rceil$ then

$$H \leq L = \sum p_i l_i < H + 1.$$

Since L^* is less than L and since $L^* \geq H$, we have the theorem. □

In this last theorem, we have an overhead of at most one bit, due to the fact that $-\log p_i$ is not always an integer. We can reduce the overhead per symbol by spreading it out over many symbols. Therefore, let us consider a system in which we send strings of n symbols at once. Consider that L_n is the expected codeword length per input symbol, i.e., if $l(x_1 x_2 \cdots x_n)$ is the length of the codeword associated with string $x_1 x_2 \cdots x_n$, where $x_i \in \Sigma$, then

$$L_n = \frac{1}{n} \sum p(x_1 x_2 \cdots x_n) l(x_1 x_2 \cdots x_n) = \frac{1}{n} E[l(X_1, X_2, \ldots, X_n)].$$

Theorem 5.3
The minimum expected codeword length per symbol satisfies

$$\frac{H(X_1, X_2, \ldots, X_n)}{n} \leq L_n^* < \frac{H(X_1, X_2, \ldots, X_n)}{n} + \frac{1}{n}.$$

Moreover, if $X_1, X_2, \ldots, X_n$ is a stationary stochastic process,

$$L_n^* \to H(\mathcal{X}),$$

where $H(\mathcal{X})$ is the entropy rate of the process.

Particularly, if the process is i.i.d., then

$$H \leq L_n^* < H + \frac{1}{n}.$$

5.1.2 A counting argument

Let us consider that there exists an algorithm, $\mathcal{C}$, that is able to compress all messages of n bits (or more). There are 2^n messages of n bits. By hypothesis, $\mathcal{C}$ is able to reduce the size of each of the 2^n messages to less than n bits.

How many codewords can be formed with less than n bits? There are 2^{n-1} with $n-1$ bits, 2^{n-2} with $n-2$ bits, $2^{n-(n-1)} = 1$ bit, ..., $2^{n-n} = 1$ with $n-n = 0$(!) bits.

Therefore, the total number of codewords with less than n bits is

$$\sum_{i=0}^{n-1} 2^i = \frac{1-2^n}{1-2} = 2^n - 1.$$

Hence, at least, two of the messages should share the same codeword, meaning that it is impossible to have a one-to-one codeword assignment. This implies that it is impossible to have a (reversible) compression algorithm that is able to compress all messages. For some of them, it will not compress or even it will expand its length.

5.1.3 Huffman codes

Huffman codes are variable-length codes developed by David A. Huffman (1925–1999) (Huffman, 1952). These codes are optimum, in the sense that they minimize the average length of the encoded messages, among all possible variable-length codes.

They are prefix-free codes, i.e., none of the codewords has a prefix made of a shorter codeword. Recall that, for example, if a given codeword is represented by the binary string "100", then the binary string "10001" cannot belong to that code. As shown before, this property ensures unique and instantaneous decoding.

The Huffman procedure for building the codes is based on two observations regarding optimum prefix codes:

- In an optimum code, symbols that occur more frequently should have shorter codewords than symbols that occur less frequently.
- In and optimum code, the two symbols that occur less frequently should have the same length.

The Huffman procedure contains the additional restriction that:

- The codewords of the two lowest probability symbols differ only in the last bit.

Therefore, for constructing the Huffman codes, the symbols (tree nodes) with the smallest probabilities are combined first. The new node gets the sum of the probabilities of the two combined nodes. This procedure is repeated until having a single node with a probability equal to one. Finally, the codewords are obtained by associating 0's and 1's to the branches of the tree.

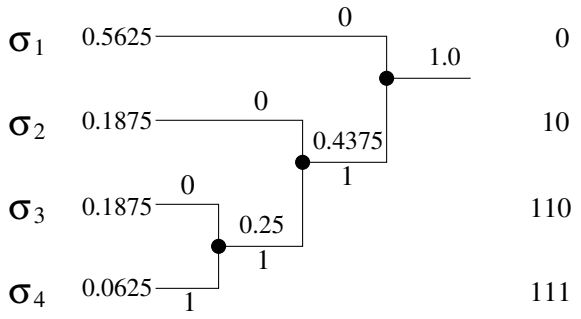


Figure 5.1: Example of tree associated with a 4-symbol Huffman code.

Example 5.1
In the example of Fig. 5.1, the ***mean number of bits/symbol*** *is*

$$1 \times 0.5625 + 2 \times 0.1875 + 3 \times 0.1875 + 3 \times 0.0625 \approx 1.69$$

The entropy of this information source is 1.62 *bits. Therefore, the* ***redundancy*** *of this code is* ≈ 0.07 *bits/symbol.*

The decoding process is straightforward. For example, suppose that the decoder received the binary string:

```
0100111101100
```

Then, the symbol sequence is

0	10	0	111	10	110	0
σ_1	σ_2	σ_1	σ_4	σ_2	σ_3	σ_1

When there are several nodes available for combining (i.e., with equal probabilities), it is possible to design different codes. In this case, which one shall we choose? For example, consider the following probability table, associated to a 5-symbol alphabet,

	σ_1	σ_2	σ_3	σ_4	σ_5
p_i	0.4	0.2	0.2	0.1	0.1

The first order entropy of this information source is 2.12 bits, and Fig. 5.2 shows two possible codes for this probability distribution. As can be verified, both codes have the same average number of bits/symbol:

$$2 \times 0.4 + 2 \times 0.2 + 2 \times 0.2 + 3 \times 0.1 + 3 \times 0.1 = 2.2$$

$$1 \times 0.4 + 3 \times 0.2 + 3 \times 0.2 + 3 \times 0.1 + 3 \times 0.1 = 2.2$$

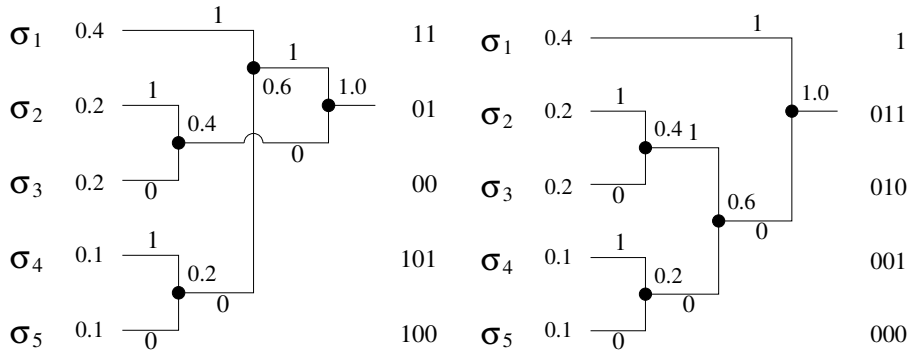


Figure 5.2: Two alternative Huffman codes for the same probability distribution.

After a sufficiently long transmission time, both codes are equivalent. However, those having higher variances might be less convenient, because they might require instantaneous higher bitrates and hence larger buffers. The first code has variance

$$0.4(2-2.2)^2 + 2 \times 0.2(2-2.2)^2 + 2 \times 0.1(3-2.2)^2 = 0.16,$$

whereas the second has variance 0.96.

Programming 5.1
Write a program that simulates the encoding process for a given probability distribution and corresponding Huffman code (for example, those shown in Fig. 5.2). Using that program, analyze the statistics of bitstream produced by the encoder with the aim of finding out if it is still compressible.

Programming 5.2
Using simulation, estimate the average time to overflow/underflow of the buffer, as a function of buffer size. As a possible example, verify that the leftmost code of Fig. 5.2 is less demanding than the code shown on the right, regarding buffer size.

5.1.4 Shannon-Fano codes

Shannon-Fano coding was proposed before Huffman coding and, although not optimum as Huffman coding, it nevertheless provides good variable-size codes. Shannon-Fano codes are like Huffman codes, in the sense that they also are variable-length and immediately decodable codes.

For constructing these codes, the symbols are first ordered by increasing or decreasing probability. Then, the symbols are split into two sets having as much as possible similar total probabilities. One of the sets gets a "0" label, whereas the other gets a "1" label. This is repeated until having sets with only one or two symbols.

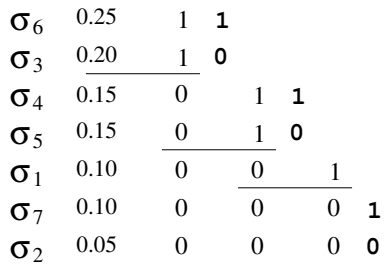


Figure 5.3: Example of construction of a Shannon-Fano code.

Example 5.2
Consider the following probability table:

σ_1	σ_2	σ_3	σ_4	σ_5	σ_6	σ_7
0.10	0.05	0.20	0.15	0.15	0.25	0.10

The Shannon-Fano code for this probability distribution is shown in Fig. 5.3.

Note that whereas the Shannon-Fano tree is created from the root to the leaves, the Huffman algorithm works from leaves to the root.

5.1.5 Alphabet extension

When

$$p_i \to 2^{-k}, \quad k \in \mathbb{N}, \quad \forall i,$$

the efficiency of the variable-length codes gets closer to the entropy of the information source. Particularly, when the probabilities are powers of $1/2$, the average length of the codes equals the entropy of the source. In this case,

$$-\log p_i \in \mathbb{N}, \quad \forall i,$$

i.e., the optimum lengths of the codewords are integers. However, for small alphabets and for highly skewed probability distributions, the corresponding variable-length codes can be far from optimal!

Example 5.3
Let us consider two symbols, σ_1 and σ_2. Varying the values of p_1 and p_2, we get the following entropy values:

p_1	0.9	0.8	0.7	0.6	0.5
p_2	0.1	0.2	0.3	0.4	0.5
H	0.47	0.72	0.88	0.97	1.00

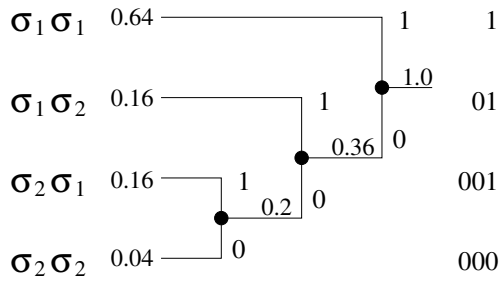


Figure 5.4: Huffman code resulting from the aggregation of pairs of symbols.

As expected, the larger the difference between these two probabilities, less bits, on average, are in theory needed for representing a message.

Consider, for example, the case where $p_1 = 0.8$ *and* $p_2 = 0.2$*:*

- *Using Huffman coding or Shannon-Fano coding, we get an average code length of one bit per symbol.*
- *This is 28% longer than what is theoretically necessary.*

Let us now consider groupings of pairs of symbols and assume ***independence****. The probabilities of the four combinations are*

Pair	Probability
$\sigma_1\sigma_1$	$0.8 \times 0.8 = 0.64$
$\sigma_1\sigma_2$	$0.8 \times 0.2 = 0.16$
$\sigma_2\sigma_1$	$0.2 \times 0.8 = 0.16$
$\sigma_2\sigma_2$	$0.2 \times 0.2 = 0.04$

Constructing a Huffman code for these new symbols (see Fig. 5.4), we get an average length of

$$1 \times 0.64 + 2 \times 0.16 + 3 \times 0.16 + 3 \times 0.04 = 1.56 \textit{ bits},$$

i.e., an average length of 0.78 *bits/original symbol.*

Consider now groupings of three symbols, with probabilities

Group	Probability
$\sigma_1\sigma_1\sigma_1$	$0.8 \times 0.8 \times 0.8 = 0.512$
$\sigma_1\sigma_1\sigma_2$	$0.8 \times 0.8 \times 0.2 = 0.128$
$\sigma_1\sigma_2\sigma_1$	$0.8 \times 0.2 \times 0.8 = 0.128$
$\sigma_1\sigma_2\sigma_2$	$0.8 \times 0.2 \times 0.2 = 0.032$
$\sigma_2\sigma_1\sigma_1$	$0.2 \times 0.8 \times 0.8 = 0.128$
$\sigma_2\sigma_1\sigma_2$	$0.2 \times 0.8 \times 0.2 = 0.032$
$\sigma_2\sigma_2\sigma_1$	$0.2 \times 0.2 \times 0.8 = 0.032$
$\sigma_2\sigma_2\sigma_2$	$0.2 \times 0.2 \times 0.2 = 0.008$

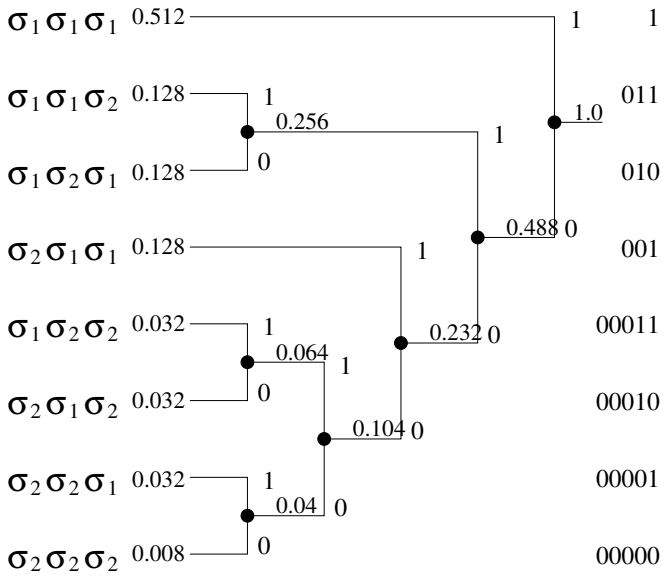


Figure 5.5: Huffman code for groups of three symbols.

The average length of the corresponding Huffman code is 2.184 *bits, i.e.,* 0.728 *bits/original symbol (see Fig. 5.5).*

5.1.6 Some other variable-length codes

Sometimes, it is necessary to design codes for representing, efficiently, integer numbers whose probabilities decrease with their values. In this case, it is possible to construct codes that are simpler than the Huffman codes, although sometimes with some loss of efficiency. On the other hand, these are "open" codes, i.e., they are able to accommodate rare, but possible, (large) numbers.

The **comma code** (also known as the **unary code**) is the simplest variable-length code. Each codeword is built using a string of 0's (1's), terminated by one 1 (0):

$$\begin{array}{lcl} 0 & - & 0 \\ 1 & - & 10 \\ 2 & - & 110 \\ 3 & - & 1110 \\ 4 & - & 11110 \\ \ldots & & \end{array}$$

Note that the decoder is just a counter.

Problem 5.1
Show that the unary code is optimum if the integers, i, occur with probabilities

$$p_i = \frac{1}{2^{(i+1)}}, \quad i = 0, 1, 2, \ldots$$

Shift-codes are generalizations of the comma code.

$B = 1$	$B = 2$	$B = 3$
0	00	000
10	01	001
110	10	010
1110	1100	011
11110	1101	100
111110	1110	101
1111110	111100	110
11111110	111101	111000
...	...	...

- These codes are organized by levels, based on a set of 2^B codewords of length B.
- One of those codewords is used to indicate the shift to the next level.
- For $B = 1$ we have the comma code.

Another type of shift code uses, besides the base, B, an increment, I. The first level is formed by all codewords of length B, except for the shift codeword. The next level (level 2) is formed by all codewords of $B + I$ bits, except for the shift codeword, concatenated with the (prefix) shift codeword of the previous level.

Generalizing, level L consists of all $B + I(L - 1)$ bit codewords, except for the shift sequence, concatenated with the prefix (shift codeword) of level $L - 1$.

Example 5.4

$B=1, I=1$		$B=2, I=1$		$B=1, I=2$	
L	*Codeword*	L	*Codeword*	L	*Codeword*
1	0	1	00	1	0
2	100	1	01	2	1000
2	101	1	10	2	1001
2	110	2	11000	2	1010
3	111000	2	11001	2	1011
3	111001	2	11010	2	1100
3	111010	2	11011	2	1101
3	111011	2	11100	2	1110
3	111100	2	11101	3	111100000
3	111101	2	11110	3	111100001
3	111110	3	111110000	3	111100010
	...		...		...

5.1.7 Golomb code

The **Golomb code** was proposed by Solomon W. Golomb (1932–2016) (Golomb, 1966). It relies on separating an integer into two parts:

- One of those parts is represented by a **unary code**.
- The other part is represented using a **binary code**.

In fact, the Golomb code is a family of codes that depend on an integer parameter, $m > 0$. The Golomb code is optimum for an information source following a distribution

$$p_i = \alpha^i(1-\alpha), \quad i = 0, 1, 2, \ldots$$

where

$$m = \left\lceil -\frac{1}{\log \alpha} \right\rceil .$$

An integer $i \geq 0$ is represented by two numbers, q and r, where

$$q = \left\lfloor \frac{i}{m} \right\rfloor \qquad \text{and} \qquad r = i - qm.$$

The quotient, q, can have the values $0, 1, 2, \ldots$, and is represented by the corresponding unary code. The remainder of the division, r, can have the values $0, 1, 2, \ldots, m-1$, and is represented by the corresponding binary code.

Example 5.5
Consider $i = 11$ *and* $m = 4$*. Then,*

$$q = \left\lfloor \frac{11}{4} \right\rfloor = 2, \quad r = 11 - 2 \times 4 = 3 \quad \rightarrow \quad 001\ 11$$

If m is not a power of 2, then the binary code (that represents the remainder of the division), is not efficient. In that case, the number of bits used can be reduced using a **truncated binary code**:

- First, define $b = \lceil \log m \rceil$.
- Encode the first $2^b - m$ values of r using the first $2^b - m$ binary codewords of $b - 1$ bits.
- Encode the remainder values of r by coding the number $r + 2^b - m$ in binary codewords of b bits.

Example 5.6
Consider $m = 5$*. In this case, we have* $b = \lceil \log 5 \rceil = 3$*. Therefore, values* $r = 0, 1, 2$ *are represented using 2 bits (respectively "00", "01" and "10"), whereas the values* $r = 3, 4$ *will be encoded using 3 bits (respectively "110" and "111").*

Example 5.7
Golomb code for $m = 5$ *and* $i = 0, 1, \ldots, 15$*:*

i	q	r	*Codeword*	i	q	r	*Codeword*
0	0	0	000	8	1	3	10110
1	0	1	001	9	1	4	10111
2	0	2	010	10	2	0	11000
3	0	3	0110	11	2	1	11001
4	0	4	0111	12	2	2	11010
5	1	0	1000	13	2	3	110110
6	1	1	1001	14	2	4	110111
7	1	2	1010	15	3	0	111000

5.2 Dictionary based compression

In many cases, the information source produces recurring patterns. A possible approach to take advantage of this behaviour is to keep a **dictionary** with the (most frequent) patterns. When

one of those patterns occurs, it is encoded using a reference to the dictionary. If it does not appear in the dictionary, it can be encoded using some other (usually less efficient) method. Therefore, the idea is to split the patterns into two classes: frequent and infrequent.

Example 5.8
Suppose we have a source alphabet with 32 symbols (for example, 26 letters plus some punctuation marks). For an i.i.d. source, we would need 5 bits/symbol. Treating all 32^4 *(*$1\,048\,576$*) 4-symbol patterns as equally likely, it would imply 20 bits for each 4-symbol pattern.*

Now, suppose we put the 256 most common patterns in a dictionary. If the pattern is in the dictionary, send '0' followed by the (8-bit) index of the pattern in the dictionary. Otherwise, send '1' followed by the 20 bits encoding the pattern.

Notice that, if p *is the probability of finding the pattern in the dictionary, then the average code-length is*

$$L = 9p + 21(1 - p) = 21 - 12p.$$

To be useful, this scheme should attain $L < 20$*. This happens for* $p \geq 0.084$*. However, for a uniform and i.i.d. source, the probability of a 4-symbol pattern over a 32-symbol alphabet is only* $0.00000095\ldots$

5.2.1 Tunstall codes

As we have seen, variable-length codes assign variable-length bit strings to the symbols of the alphabet. One of the problems with this approach is that errors in the encoded data propagate easily.

In the **Tunstall code**, all codewords are of equal length. Instead, each codeword represents variable-length groups of alphabet symbols.

Example 5.9
Consider the following 2-bit Tunstall code for the alphabet $\Sigma = \{A, B\}$*:*

Sequence	*Codeword*
AAA	*00*
AAB	*01*
AB	*10*
B	*11*

Then, the sequence "AAABAABAABAABAAA" is encoded as "001101010100".

The design of a code that has a fixed codeword length, but a variable number of symbols per codeword, needs to meet the following two conditions:

- We should be able to parse all source strings into substrings of symbols that appear in the codebook (dictionary). This property is usually known as "**generality**".
- We should maximize the average number of source symbols represented by each codeword.

Example 5.10
To understand the meaning of the first condition, consider now the following code:

Sequence	*Codeword*
AAA	*00*
ABA	*01*
AB	*10*
B	*11*

Let us try to encode the same sequence, "AAABAABAABAABAAA", but now using this new code:

- *First, we encode "AAA" with code "00".*
- *Then, we encode "B" with code "11".*
- *Next, we run into trouble, because there is no way to proceed...*

Lemma 5.1 (Dictionary generality)
A dictionary consisting of strings from an alphabet Σ satisfies the generality condition iff for every possible string of symbols x from Σ there exists at least one prefix of x in the dictionary.

Discussion topic 5.1
Discuss conditions that a dictionary must meet in order to obey the generality restriction.

A n-bit Tunstall code for an i.i.d. source over a size-m alphabet can be built using the following algorithm:

Algorithm 5.1 (Tunstall code)

- *Start with the m symbols of the alphabet in the codebook.*
- *Then, remove the entry with highest probability and add the m strings obtained by concatenating this symbol with every symbol of the alphabet. The new probabilities are the product (because independence is assumed) of the probabilities of the concatenated symbols—notice that this operation increases the size of the codebook from m to $m + (m - 1)$.*
- *Repeat this procedure k times, subject to the constraint $m + k(m - 1) \leq 2^n$, where k is the largest possible integer that verifies the condition.*

Example 5.11
Consider the design of a 3-bit Tunstall code for the alphabet

Symbol	*Probability*
A	*0.60*
B	*0.30*
C	*0.10*

Because "A" is the most probable symbol, we first obtain

Sequence	*Probability*
B	*0.30*
C	*0.10*
AA	*0.36*
AB	*0.18*
AC	*0.06*

Finally, we have

Sequence	*Probability*	*Code*
B	*0.300*	*000*
C	*0.100*	*001*
AB	*0.180*	*010*
AC	*0.060*	*011*
AAA	*0.216*	*100*
AAB	*0.108*	*101*
AAC	*0.036*	*110*

Discussion topic 5.2
Are the Tunstall codes uniquely decodable? Why?

5.2.2 String parsing

The ultimate objective of dictionary-based data compression is to parse the source string into substrings (also known as phrases or factors) whose corresponding codewords have the smallest possible total length. Parsing, i.e., creating a partition of the string, plays an important role in this process.

For an arbitrary dictionary, complying with the generality constraint, there are optimal (in the sense of producing the least number of phrases) ways of parsing the string to be compressed. Typically, they involve more than one pass over the string of data, using dynamic programming. More direct approaches, although possibly sub-optimal, rely on greedy strategies. In this case, assuming that processing is done left-to-right, the greedy algorithm chooses the longest possible prefix of the yet uncompressed string that exists in the dictionary.

For some special cases, the greedy strategy can be, in fact, optimal. This happens when the dictionaries are prefix-complete or suffix-complete. A dictionary is **prefix-complete** if for every string in the dictionary, all of its prefixes are also in the dictionary. A dictionary is **suffix-complete** if for every string in the dictionary, all of its suffixes are also in the dictionary.

Theorem 5.4 (Cohn and Khazan 1996)
With respect to a suffix-complete dictionary, greedy parsing left-to-right is optimal; dually, with respect to a prefix-complete dictionary, greedy parsing right-to-left is optimal.

5.2.3 LZ77 compression

In the 1970's, Jacob Ziv (1931–) and Abraham Lempel (1936–) proposed two different approaches for data compression, using dictionary-based principles. Contrarily to the Tunstall code that we have just seen, where the dictionary is built beforehand (i.e., it is **static**), these methods are **adaptive**—the dictionary is "learned" at the same time compression is performed.

The first algorithm, usually known as LZ77 (or LZ1), was presented in

- J. Ziv and A. Lempel, *A universal algorithm for sequential data compression*, IEEE Transactions on Information Theory (Ziv and Lempel, 1977).

Essentially, it relies on a separation of the data in two parts:

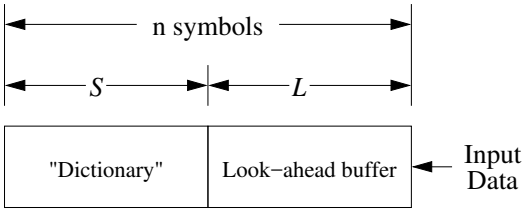


Figure 5.6: Separation between the "dictionary" and data to be encoded in LZ77.

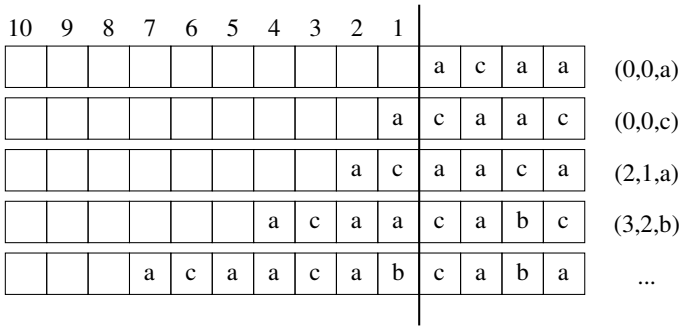


Figure 5.7: Simple example of LZ77 encoding.

- Data already encoded;
- Data that are to be encoded,

as shown in Fig. 5.6.

During operation, the data go first through a **look-ahead buffer** and then through a **search buffer** (the "dictionary"). The algorithm searches, in the "dictionary", for the largest possible prefix of the string in the look-ahead buffer. Obviously, the larger the prefix, the higher will be the coding efficiency.

The codeword that is generated in each coding step is composed of **three components**:

1. A pointer, that indicates the position of the matching string in the "dictionary".
2. The length of the string matched.
3. The first symbol that in the look-ahead buffer follows the matching string.

Figure 5.7 shows a simple example of encoding. The look-ahead buffer can also be used as a "dictionary" extension, as exemplified in Fig. 5.8.

In LZ77 encoding, a codeword requires

$$\lceil \log_2 S \rceil + \lceil \log_2 (L-1) \rceil + \lceil \log_2 m \rceil$$

bits, where S is the size of the "dictionary", L is the size of the look-ahead buffer and m is the size of the alphabet. Taking as example $S = 2048$, $L = 33$ and $m = 256$, a codeword would

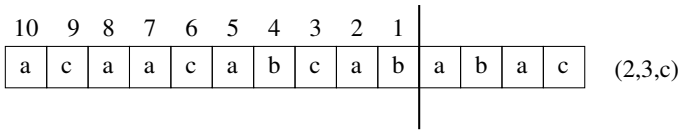


Figure 5.8: Example of the use of the look-ahead buffer for "dictionary" extension in LZ77 encoding.

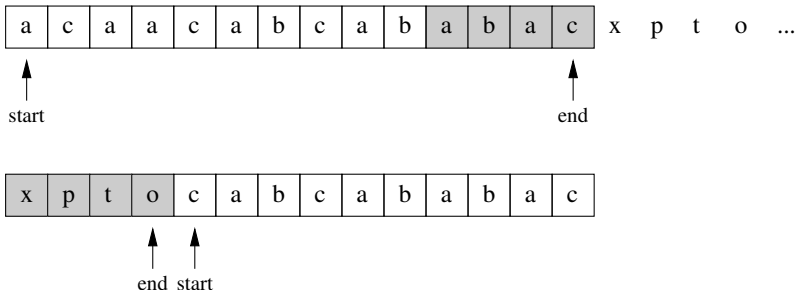


Figure 5.9: Look-ahead buffer implemented using a circular queue.

require $11 + 5 + 8 = 24$ bits, which is three times the number of bits required to represent a single symbol (8 bits).

Discussion topic 5.3
There is a number of practical questions that need to be addressed when implementing this compression algorithm, such as

- *The search time required by each coding step as a function of the dictionary size.*
- *The probability of finding matching sequences as a function of the dictionary size.*
- *The processing time required by each coding step as a function of the look-ahead buffer size.*
- *The probability of finding matching sequences as a function of the look-ahead size.*
- *The impact in codeword length when the sizes of the dictionary and/or look-ahead buffer are changed.*
- *The potential inefficiency of using three component codewords when having to encode isolated symbols.*

Discuss them.

The implementation of LZ77-type algorithms needs also to take into consideration the efficient use of data structures. For example, the use of circular buffers (see Fig. 5.9) and tree structures (see Fig. 5.10) may dramatically improve processing speed.

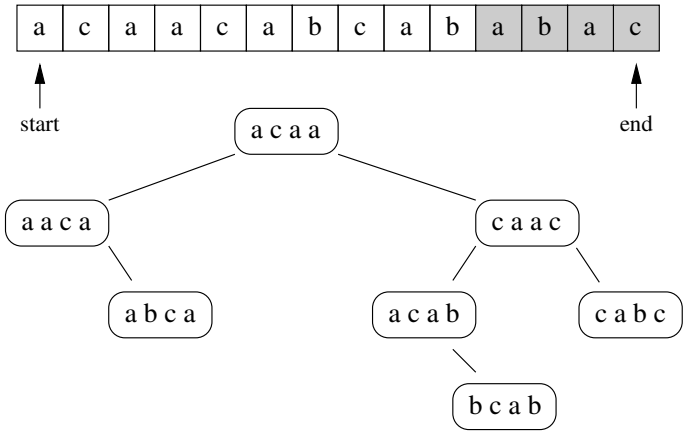


Figure 5.10: Dictionary implemented using a binary tree.

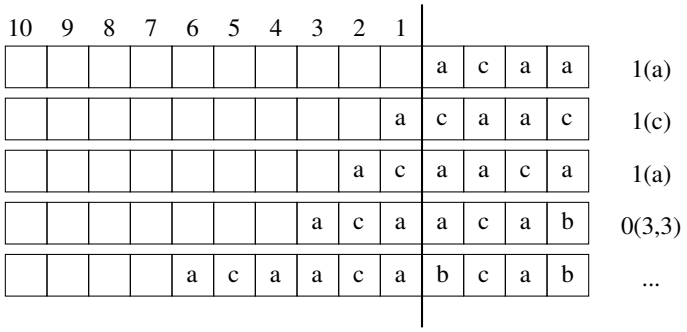


Figure 5.11: Example of LZSS encoding.

5.2.4 LZSS compression

LZSS is a variant of LZ77 proposed by James Storer (1953–) and Thomas Szymanski (Storer and Szymanski, 1982). As shown in the previous section, with LZ77 sometimes it is possible to produce codewords that use more bits than the original bits of the string it represents. With LZSS, this is avoided by enforcing codewords with only **two components**—one bit is used to indicate if the data are encoded or sent as uncoded, literal form. Therefore, when a sequence is not found in the dictionary, the symbol code is sent, instead of $(0, 0, \dots)$ as in LZ77. To distinguishing both cases, one bit is used. Figure 5.11 gives an example of an encoding using LZSS.

5.2.5 LZ78 compression

The second algorithm proposed by Jacob Ziv and Abraham Lempel, known as LZ78 (or LZ2), was published in

- J. Ziv and A. Lempel, *Compression of individual sequences via variable-rate coding*,

Dictionary:	a	aa	b	aaa	d	aab	aad	o
Index:	1	2	3	4	5	6	7	8
Codeword:	0,a	1,a	0,b	2,a	0,d	2,b	2,d	0,o

Figure 5.12: LZ78 encoding of string “aaabaaadaabaado”.

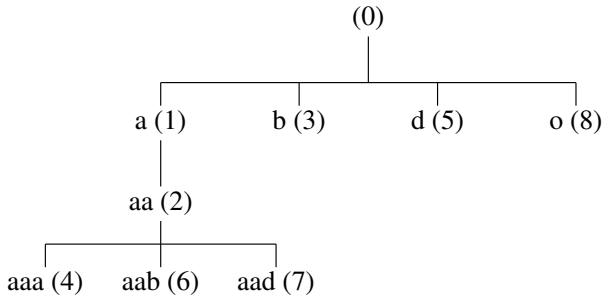


Figure 5.13: Tree data structure for fast access to the LZ78 dictionary.

IEEE Trans. on Information Theory (Ziv and Lempel, 1978).

LZ78 relies on a distinct approach from that of LZ77, where the use of an explicit dictionary is probability the most obvious. In LZ78 coding:

- The data are parsed into strings.
- Each string is built from the largest matching string found so far, plus the first non-matching symbol.
- The new string is represented using the index of the matching string (which is in the dictionary), followed by the code of the unmatched symbol.
- Finally, the new string is added to the dictionary.

Figure 5.12 shows how the string “aaabaaadaabaado” is represented by the LZ78 encoding algorithm. It is worth noting that, apparently, this new strategy seems not to put restrictions to the distance at which repeating patterns occur, whereas with LZ77, if this distance exceeds the size of the searching area (the “dictionary”), they are not taken into account. Nevertheless, it was shown that both techniques are asymptotically optimal for ergodic sources, i.e., asymptotically, its encoding efficiency approaches the entropy of the source.

As with LZ77-type algorithms, it is fundamental to use appropriate data structures to speed up processing. In the case of LZ78, a possibility is to represent the dictionary using tree structures, such as in the example of Fig. 5.13.

Discussion topic 5.4

In practice, the dictionary cannot grow without bound, because, for example,

- *The size of the codewords is related to the size of the dictionary.*
- *The amount of memory for storing the dictionary (both by the encoder and by the decoder) might be too large.*

Therefore, in practice, it is necessary to implement mechanisms for limiting the growth of the dictionary. Discuss advantages and disadvantages of possible mechanisms for doing so, such as,

- *Dictionary "freezing", when it reaches some predefined size.*
- *When the dictionary reaches some predefined size, it is deleted and started again (note that this is identical to block coding).*
- *When the dictionary is full, some entries are deleted, for example, those not used for a longer time or less used.*

Discussion topic 5.5
LZ77 relies on a suffix-complete dictionary, whereas LZ78 relies on a prefix-complete dictionary (explain why). Based on Theorem 5.4, discuss possible implications of this fact.

5.2.6 LZW compression

In 1984, Terry Welch (1939–1988) published a paper where solutions for some of the problems associated to LZ78 are proposed, namely (Welch, 1984):

- Symbols seen for the first time are encoded more efficiently (in LZ78, two-component codewords are generated in this case).
- The number of components of the codewords is reduced to just **one**.

Initially, all symbols of the alphabet are inserted in the dictionary. Then, the algorithm operates as follows:

- Symbols are read, one by one, from the string to encode.
- These symbols (σ) are concatenated to a string, x, while $x\sigma$ can be found in the dictionary.
- When it is not possible to add one more symbol, the index of x in the dictionary is sent, $x\sigma$ is inserted in the dictionary, and x is initialized with symbol σ.

Example 5.12

Consider the encoding of string "aaabaaadaabaado". Initially, the dictionary contains all symbols of the alphabet:

0	…	97	98	99	100	…	255
nul	…	a	b	c	d	…	

String: aa *abaaadaabaado; Codeword sent: 97*

…	97	98	99	100	…	256
…	a	b	c	d	…	aa

String: a aab *aaadaabaado; Codeword sent: 256*

…	97	98	99	100	…	256	257
…	a	b	c	d	…	aa	aab

String: aaa ba *adaabaado; Codeword sent: 98*

…	97	98	99	100	…	256	257	258
…	a	b	c	d	…	aa	aab	ba

String: aaab aaa *daabaado; Codeword sent: 256*

…	97	98	99	100	…	256	257	258	259
…	a	b	c	d	…	aa	aab	ba	aaa

String: aaabaa ad *aabaado; Codeword sent: 97*

…	97	98	99	100	…	256	257	258	259	260
…	a	b	c	d	…	aa	aab	ba	aaa	ad

String: aaabaaa da *abaado; Codeword sent: 100*

…	97	98	99	100	…	256	257	258	259	260	261
…	a	b	c	d	…	aa	aab	ba	aaa	ad	da

String: aaabaaad aaba *ado; Codeword sent: 257*

…	97	98	99	100	…	256	257	258	259	260	261	262
…	a	b	c	d	…	aa	aab	ba	aaa	ad	da	aaba

String: aaabaaadaab aad *o; Codeword sent: 256*

…	97	98	99	100	…	256	257	258	259	260	261	262	263
…	a	b	c	d	…	aa	aab	ba	aaa	ad	da	aaba	aad

...

Example 5.13

Decoding is performed as follows. Initially, the dictionary contains all symbols of the alphabet.

0 nul	... ...	97 a	98 b	99 c	100 d	... ...	255

Codeword received: 97 ; String: a?

... ...	97 a	98 b	99 c	100 d	... ...	256 a?

Codeword received: 256 ; String: a aa?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aa?

Codeword received: 98 ; String: aaa b?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aab	258 b?

Codeword received: 256 ; String: aaab aa?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aab	258 ba	259 aa?

Codeword received: 97 ; String: aaabaa a?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aab	258 ba	259 aaa	260 a?

Codeword received: 100 ; String: aaabaaa d?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aab	258 ba	259 aaa	260 ad	261 d?

Codeword received: 257 ; String: aaabaaad aab?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aab	258 ba	259 aaa	260 ad	261 da	262 aab?

Codeword received: 256 ; String: aaabaaadaab aa?

... ...	97 a	98 b	99 c	100 d	... ...	256 aa	257 aab	258 ba	259 aaa	260 ad	261 da	262 aaba	263 aa?

...

5.3 Arithmetic coding

5.3.1 Motivation

We have seen that one of the main limitations of variable length codes is that they are optimum only if the probabilities of the symbols are powers of $1/2$.

Example 5.14
Suppose we have

$$p_1 = \frac{1}{2}, \quad p_2 = \frac{1}{4}, \quad p_3 = \frac{1}{4}$$

The first-order entropy of this information source is 1.5 *bits/symbol. Using a Huffman code, we have, for example,*

$$\sigma_1 \to 0 \quad \sigma_2 \to 10 \quad \sigma_3 \to 11$$

The average length of this code is 1.5 *bits/symbol and, therefore, its redundancy is zero (i.e., this code is optimal for this source).*

Example 5.15
Consider now that the probability distribution of the source symbols is

$$p_1 = \frac{2}{3}, \quad p_2 = \frac{1}{6}, \quad p_3 = \frac{1}{6}.$$

In this case, the entropy of the source is ≈ 1.26*. However, as in the previous case, the Huffman encoding algorithm assigns 1 bit to* σ_1 *and 2 bits to the other two symbols. The average length of this code is* ≈ 1.33*, i.e.,* 0.07 *bits per symbol more than the source entropy. The difference between the average code length and the entropy increases when* $\max(p_i) > 0.5$ *and* $\max(p_i) \to 1$.

As with Huffman coding, arithmetic coding needs an estimate of the probabilities of the symbols of the alphabet. Arithmetic coding represents the entire message by a single codeword: a number in the $[0, 1)$ interval. This is why arithmetic coding is not affected by the skewness of the probability distribution, as variable length codes might be. Besides, arithmetic coding has the advantage of allowing a clear separation between the **coding** process and source **modeling**.

5.3.2 Encoding

For encoding, the $[0, 1)$ interval is partitioned according to the probability distribution of the symbols. The only restriction is that the decoder has to use the same partitioning.

Example 5.16
Consider the following alphabet and corresponding probability distribution:

Symbol	*Prob.*	*Interval*
d	0.1	$[0, 0.1)$
e	0.3	$[0.1, 0.4)$
f	0.1	$[0.4, 0.5)$
g	0.05	$[0.5, 0.55)$
n	0.1	$[0.55, 0.65)$
r	0.2	$[0.65, 0.85)$
u	0.05	$[0.85, 0.9)$
space	0.1	$[0.9, 1)$

Let us consider the encoding of the message "fred".

*Initially, the **coding interval** is* $[0, 1)$*. The first symbol to encode, "f", has the* $[0.4, 0.5)$ *interval assigned to it, according to the probability table. After encoding this symbol, the coding interval will be* $[0.4, 0.5)$*. The next symbol, "r", is associated with the* $[0.65, 0.85)$ *interval. Then, the **lower limit** of the new coding interval will be*

$$0.4 + 0.65(0.5 - 0.4) = 0.465$$

*and the **upper limit***

$$0.4 + 0.85(0.5 - 0.4) = 0.485.$$

The next symbol, "e", transforms the current coding interval, $[0.465, 0.485)$*, into* $[0.467, 0.473)$*, according to*

$$0.467 = 0.465 + 0.1(0.485 - 0.465)$$

and

$$0.473 = 0.465 + 0.4(0.485 - 0.465).$$

After encoding symbol "d", we get the interval $[0.467, 0.4676)$*.*

Therefore, if the current coding interval is $[L^n, H^n)$, and if the next symbol to encode is represented by the interval $[l, h)$, then

$$L^{n+1} = L^n + l(H^n - L^n)$$

and

$$H^{n+1} = L^n + h(H^n - L^n).$$

Example 5.17
Consider the encoding of the binary string `0110`*, using* $P(0) = 0.6$ *and* $P(1) = 0.4$*. The encoded message is a number in the interval* $[0.504, 0.5616)$ *(see Fig. 5.14). Another possible view of the encoding process can be seen in Fig. 5.15, in this case with renormalized intervals.*

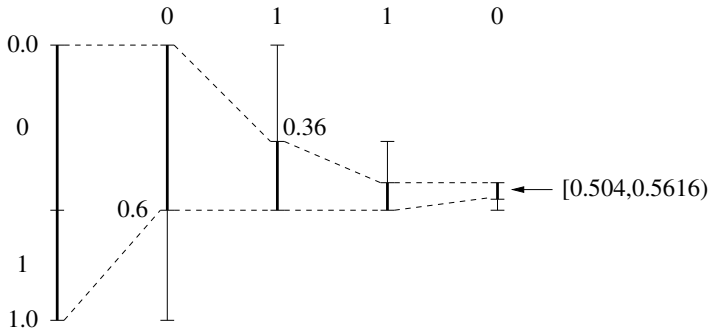


Figure 5.14: Example of the arithmetic encoding of a simple string.

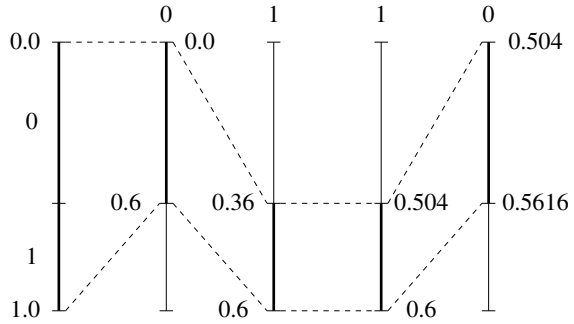


Figure 5.15: The same example as of Fig. 5.14, but with a different visualization of the intervals.

5.3.3 Decoding

For decoding the "fred" message, we proceed as follows. Let x be the value received by the decoder (i.e., the encoded message), such that $x \in [0.467, 0.4676)$. For example, consider $x = 0.467$. Since $x \in [0.4, 0.5)$, then the first symbol of the message is "f". The "effect" of this symbol is then eliminated from the encoded message, using the following procedure:

1. Subtract the lower limit of the "f" interval from x, i.e., $x - 0.4 = 0.467 - 0.4 = 0.067$.
2. Divide this new value by the range of the "f" interval (0.1, in this case): $0.067/0.1 = 0.67$.

The new number, $x = 0.67$, is in the $[0.65, 0.85)$ interval, meaning that "r" is the next symbol. Re-normalizing, we obtain $x = (0.67 - 0.65)/(0.85 - 0.65) = 0.1$. Because $x \in [0.1, 0.4)$, then the next symbol is "e". Re-normalizing again, $x = (0.1 - 0.1)/(0.4 - 0.1) = 0$. Since $x \in [0, 0.1)$, then the symbol is "d".

Notice that, there is no way to detect, automatically, the end of a message encoded using arithmetic coding. Therefore, it is necessary to indicate the length of the message (i.e., the number of symbols of the message) or to use a special "end-of-message" symbol.

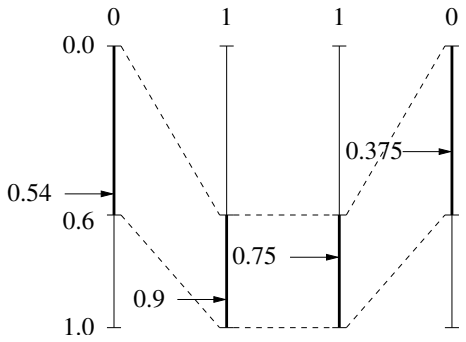


Figure 5.16: Decoding of the example depicted in Figs. 5.14 and 5.15.

Example 5.18
The decoding process of the previous example can be seen in Fig. 5.16, where $x = 0.54$ was considered.

5.3.4 Implementation issues

Obviously, it is not possible to obtain an useful encoder by direct implementation of the described process. In fact, the precision of the floating point representation of the computer would rapidly be exhausted. Nevertheless, even if we had access to a computer with infinite precision, it certainly would not be efficient to perform arithmetic operations with several thousands or millions of decimal places. Moreover, it is generally not acceptable to have to wait for the end of the message in order to start receiving the first bits of the encoded message.

As we saw earlier, as new symbols arrive at the encoder, the limits of the coding interval get closer to each other, implying the need of an increasing precision in order to distinguish them apart. However, it is reasonable to admit that, at a certain point, both limits will lay above 0.5 or below 0.5. When this happens, the most significant bit of L and H will stay permanently equal to 0, if $L < 0.5$ and $H < 0.5$, or equal to 1, if $L \geq 0.5$ and $H \geq 0.5$. Notice that

$$\begin{aligned} x \geq 0.5_{(10)} &\longrightarrow x = 0.1\ldots_{(2)} \\ x < 0.5_{(10)} &\longrightarrow x = 0.0\ldots_{(2)} \end{aligned}$$

Therefore, this bit can be sent out by the encoder.

On the other hand, we can expand the interval in the following way,

$$\begin{array}{lll} \text{Sent bit : } 0 & [0, 0.5) \longrightarrow [0, 1) & x \to 2x \\ \text{Sent bit : } 1 & [0.5, 1) \longrightarrow [0, 1) & x \to 2(x - 0.5), \end{array}$$

which, in principle, allow us to keep the precision under control.

But if $L \to 0.0111\ldots_{(2)}$ and $H \to 0.1000\ldots_{(2)}$?

To take into account this situation, whenever the coding interval belong to $[0.25, 0.75)$, we perform the expansion

$$[0.25, 0.75) \longrightarrow [0, 1) \qquad x \to 2(x - 0.25),$$

and a counter takes note of how many times these re-scalings occurred since the last bit has been sent by the encoder.

Notice that this operation is performed when $0.25 \leq L < 0.5$ ($L = 0.01\ldots_{(2)}$) and $0.5 \leq H < 0.75$ ($H = 0.10\ldots_{(2)}$). After this change of scale, we get $L = 0.0\ldots_{(2)}$ and $H = 0.1\ldots_{(2)}$. The result of this operation is that the second most significant bit is "deleted".

If, after incorporating the next symbol, $L \geq 0.5$ and $H \geq 0.5$, then the corresponding "1" is sent, followed by as many "0" bits as the value of the counter. On the other hand, if $L < 0.5$ and $H < 0.5$, it is sent the corresponding "0" bit, followed by as many "1" bits as the value of the counter.

6 Predictive coding

6.1 Principles

Let $x_1^n = x_1 x_2 \dots x_n$ be the sequence of values (scalars or vectors) produced by an information source until time n. **Predictive coding** is a technique that relies on encoding the sequence $r_1^n = r_1 r_2 \dots r_n$, instead of the original sequence x_1^n, where

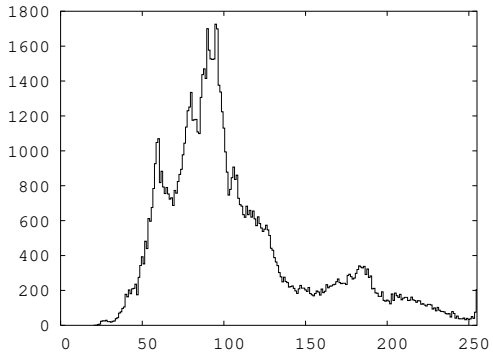
$$r_n = x_n - \hat{x}_n \tag{6.1}$$

and

$$\hat{x}_n = p(x_1^{n-1}) = p(x_1 x_2 \dots x_{n-1}). \tag{6.2}$$

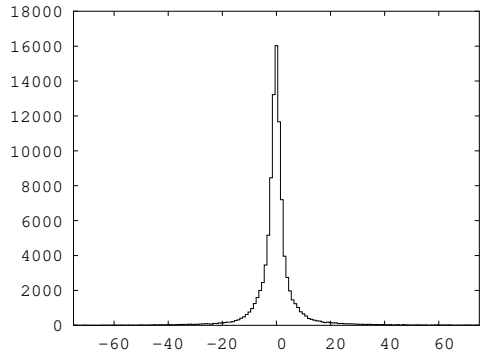
The $\hat{x}_n$ is the **estimate** at time n and the value of r_n is known as the **residual**. The $p()$ function is the **estimator** or **predictor**. The aim of predictive coding is to have $L(r_1^n) < L(x_1^n)$, where $L()$ denotes the code length attained by "adequate" compression methods to compress r_1^n and x_1^n. Figure 6.1 gives an example of this principle.





Original

$H = 7.26$ bits/symbol



Predictor 1 JPEG

$H = 4.49$ bits/symbol

Figure 6.1: Comparison of the first order entropy calculated over the original pixel values of the image and of the residuals resulting from a simple predictor. The plots show the histograms of x_1^n (on the left) and of r_1^n (on the right).

6.2 Predictive coding techniques

6.2.1 Simple 1D prediction

Some lossless audio encoders (for example, FLAC) use simple polynomial predictors, such as

$$\begin{cases} \hat{x}_n^{(0)} = 0 \\ \hat{x}_n^{(1)} = x_{n-1} \\ \hat{x}_n^{(2)} = 2x_{n-1} - x_{n-2} \\ \hat{x}_n^{(3)} = 3x_{n-1} - 3x_{n-2} + x_{n-3} \end{cases} \tag{6.3}$$

The first one, $\hat{x}_n^{(0)}$, provides an estimate that is always zero (this is useful, for example, when the encoder is starting, because no previous values are available yet). The second one, $\hat{x}_n^{(1)}$, assumes that the signal is constant, i.e., that the next value is equal to the present value. The third one uses an extrapolation by a straight line and the last one by a parabolic function.

Problem 6.1
Try to deduce the formulas for $\hat{x}_n^{(2)}$ and for $\hat{x}_n^{(3)}$, shown in (6.3).

In the particular case of the predictors given in (6.3), the corresponding residuals can be computed efficiently in a recursive way, using

$$\begin{cases} r_n^{(0)} = x_n \\ r_n^{(1)} = r_n^{(0)} - r_{n-1}^{(0)} \\ r_n^{(2)} = r_n^{(1)} - r_{n-1}^{(1)} \\ r_n^{(3)} = r_n^{(2)} - r_{n-1}^{(2)} \end{cases} . \tag{6.4}$$

6.2.2 Predictive image coding techniques

Typically, images are encoded from left to right, top to bottom, i.e., in **raster-scan** order, as illustrated in Fig. 6.2.

In this case, the sequence x_1^n is obtained by concatenating the first $\lfloor n/N_C \rfloor$ image rows, plus the $n \bmod N_C$ pixels from row number $\lfloor n/N_C \rfloor + 1$.

Other approaches use hierarchical decompositions (or **multi-resolution**). This is the case of the HINT method (Hierarchical INTerpolation), illustrated in Fig. 6.3.

6.2.3 Predictors

Obviously, good predictors are fundamental in predictive coding. For efficient encoding, the estimated values should be as close as possible to the real values, i.e., the r_1^n values should be as small as possible.

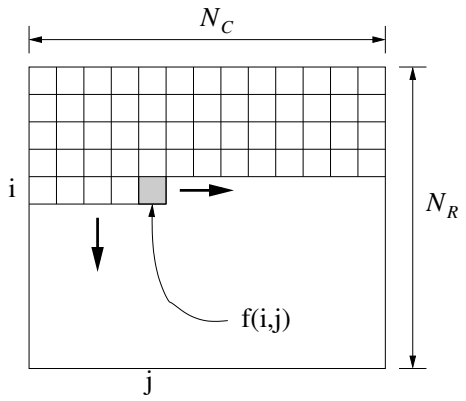


Figure 6.2: Raster-scan movement throughout an image with N_R rows and N_C columns, where $f(i, j)$ denotes the pixel value in row i and column j.

1	5	3	5	1	5	3	5	1	5	3	5	1
5	4	5	4	5	4	5	4	5	4	5	4	5
3	5	2	5	3	5	2	5	3	5	2	5	3
5	4	5	4	5	4	5	4	5	4	5	4	5
1	5	3	5	1	5	3	5	1	5	3	5	1
5	4	5	4	5	4	5	4	5	4	5	4	5
3	5	2	5	3	5	2	5	3	5	2	5	3
5	4	5	4	5	4	5	4	5	4	5	4	5
1	5	3	5	1	5	3	5	1	5	3	5	1

Figure 6.3: Scanning approach used by the HINT method. First, the pixels marked with "1" are encoded. Then, the pixels marked with "2" are predicted from those marked with "1" and the corresponding residuals are encoded. Pixels marked with "3" are encoded in a similar way, but in this case prediction is made from those marked with "1" and "2", and so on.

Note that the decoder must be able to generate the same sequence, $\hat{x}_1^n$, of estimated values. In other words, **the predictor cannot introduce any error** during encoding / decoding, otherwise the decoded signal will diverge from the original. Therefore, the predictor must be **causal**, and, in the case of lossy coding, the predictor at the encoder **must use the reconstructed values**, $\tilde{x}_1^{n-1}$, instead of the original values, x_1^{n-1}. Figure 6.4 shows a block diagram of a codec based on predictive coding, where the residuals suffer some form of quantization (i.e., the codec is lossy).

Problem 6.2

Try to find examples showing this divergence, i.e., when the residual calculated by the decoder is different from the one used by the encoder.

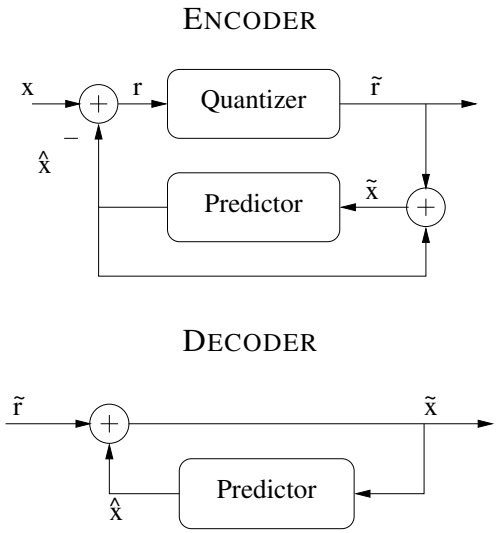


Figure 6.4: Block diagram of a prediction-based lossy codec.

Generally, the complexity of the predictor depends on two aspects:

- The number of values used for calculating the estimates (**the order of the predictor**).
- The spatial (or temporal) configuration of these values.

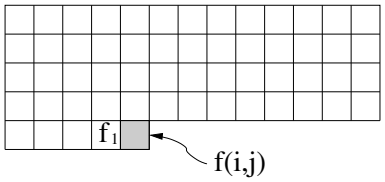


Figure 6.5: Example of an order-1 image predictor, where the prediction corresponds to the value of the previously encoded pixel.

Consider the example of a spatial predictor of order 1, where the estimated value is given by the immediately preceding value (as shown in Fig. 6.5). This type of predictor can be easily extended to higher orders, using the last k processed pixels of the image. However, for orders higher than 3 or 4, the efficiency does not increase significantly (in fact, it may even decrease). This happens because images are 2D signals, not 1D sequences of data. Therefore, generally, the spatial configurations used for predictive image coding have a 2D shape. Figure 6.6 shows one example of a 2D image predictor.

6.3 Lossless predictive coding

One of the main advantages of predictive coding is allowing a simple design of lossless encoders. In fact, most of the lossless encoders for audio and image rely on predictive coding

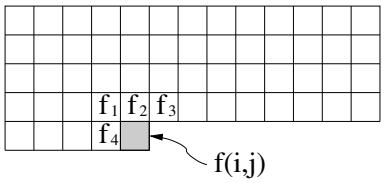


Figure 6.6: Example of a 2D image predictor. In this case, besides the previously encoded value (denoted here as f_4), the predictor also uses neighbouring values from the previously encoded row.

techniques. However, for lossless coding, there is an additional constraint regarding the predictor: the estimates generated must be platform independent. This constraint can be met, for example, using integer arithmetic. Figure 6.7 shows a block diagram of a prediction-based lossless codec.

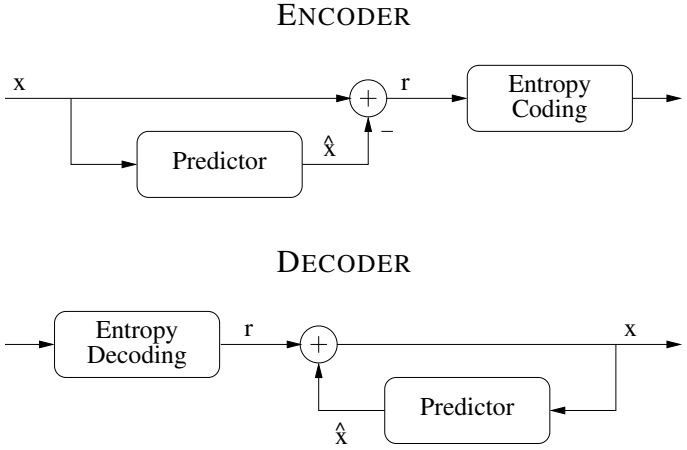
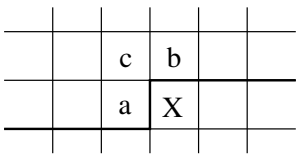


Figure 6.7: Block diagram of a prediction-based lossless codec.

6.3.1 Linear prediction: the lossless mode of JPEG

The lossless mode of JPEG (ISO/IEC 10918-1, ITU-T T.81, 1992) provides seven **linear predictors**, according to



Mode	Predictor
1	a
2	b
3	c
4	$a+b-c$
5	$a+(b-c)/2$
6	$b+(a-c)/2$
7	$(a+b)/2$

(6.5)



Predictor	1	2	3	4	5	6	7
Entropy	4.49	4.21	4.74	4.17	4.16	**4.04**	4.10

Figure 6.8: Example of the first order entropy of the prediction residuals provided by the seven linear predictors of the lossless mode of JPEG.

Generally, the performance of the several predictors may vary considerably from image to image. If encoding time is not a problem, then all of them can be tested and the one with the best compression rate chosen. Figure 6.8 shows an example of the performance attained by these different predictors, estimated using a first order entropy measure.

6.3.2 The nonlinear predictor of JPEG-LS

JPEG-LS (ISO/IEC 14495-1, ITU-T T.87, 1999) uses a predictor based on the same spatial configuration as that of JPEG, but, instead of a linear predictor, it uses the **nonlinear predictor**

$$\hat{x} = \begin{cases} \min(a,b) & \text{if} \quad c \geq \max(a,b) \\ \max(a,b) & \text{if} \quad c \leq \min(a,b) \\ a+b-c & \text{otherwise} \end{cases} \tag{6.6}$$

Note that the linear part of this predictor ($a+b-c$) is the same as predictor number 4 of JPEG. Figure 6.9 shows examples of the performance (estimated using first order entropy) attained by this predictor, as well as those of JPEG lossless.

6.4 Temporal prediction and motion compensation

Typically, the differences between one frame and the previous frame of a video sequence are due to the motion of the several elements of the visual scene (see Fig. 6.10). Exceptions occur when there are scene changes, zoom-in / zoom-out operations or camera translation.

To exploit this redundancy, it is frequent to use **temporal prediction**, which usually relies on **motion compensation**. This is one of the core techniques used to attain inter-frame compression.



(a)



(b)

Predictor	1	2	3	4	5	6	7	JLS
Entropy (a)	4.49	4.21	4.74	4.18	4.16	4.04	4.10	**3.98**
Entropy (b)	5.60	5.05	5.82	5.19	5.23	4.97	5.15	**4.93**

Figure 6.9: Example of the first order entropy of the prediction residuals provided by the non-linear predictor of JPEG-LS and also, for comparison, of the seven linear predictors of JPEG lossless.

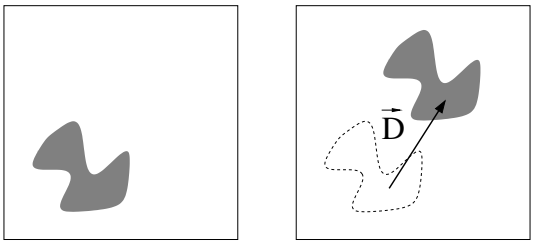


Figure 6.10: Illustration of the principle behind motion compensation, where the position of the object in frame $t + 1$ is estimated (predicted) from its position at time t.

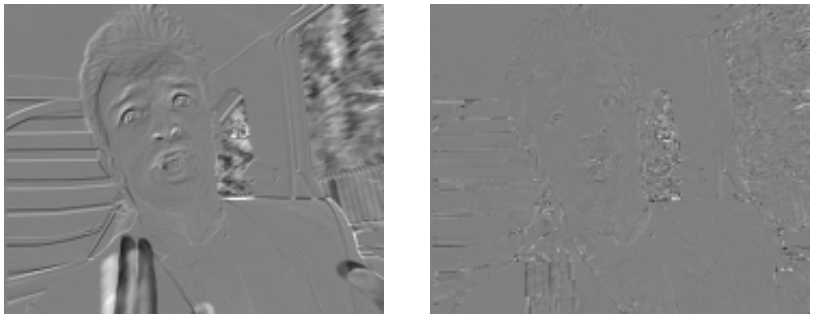
A simpler technique that also takes advantage of the temporal correlation between frames is **conditional replenishment** video coding. It finds zones in the video frame where there were changes with respect to the previous frame, and only those zones are encoded. This technique does not use motion compensation. It just performs a detection of temporal activity.

More generally, video coding based on **motion compensation** involves the following steps:

- Estimation of the motion vectors.
- Compensation, i.e., temporal prediction.
- Encoding of the motion vectors.
- Encoding of the prediction residuals.



Frame 200 Frame 201



Direct difference
$H = 5.23$ bpp

Motion compensation
$H = 4.38$ bpp

Figure 6.11: First order entropy of the residuals obtained by direct frame difference and after motion compensation.

There are a large number of techniques for motion detection, but one of them is clearly the most common approach for video coding. For each frame block (for example, of $N \times N$ pixels), it seeks the position where it minimizes some measure in relation to the previous frame (**the reference frame**). Note that this approach tries to find the position that minimizes a measure of interest, which might not correspond to the true motion in the scene.

Typically, we want to minimize some measure $D(i,j)$, such as

$$D(i,j) = \sum_{r=1}^{N} \sum_{c=1}^{N} d\big(g(r,c,t) - g(i+r, j+c, t+1)\big) \tag{6.7}$$

where $d(\cdot)$ is, for example, $(\cdot)^2$ or $|\cdot|$.

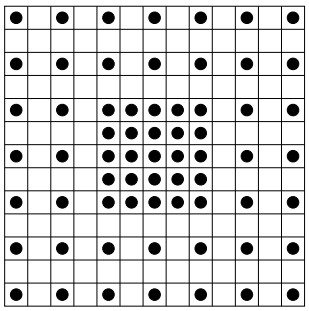
Due to complexity constraints, searching is limited to a neighborhood of $(N+2\Delta)\times(N+2\Delta)$ pixels around the block, i.e., $-\Delta \leq i,j \leq \Delta$. Table 2 shows examples of the first order entropy of the residuals and motion vectors for some values of N and Δ (see also Fig. 6.11).

If exhaustive search is used, it is guaranteed to find the minimum of $D(i,j)$. However, this approach is generally computationally too demanding, hence other sub-optimal techniques have been proposed.

Several of the sup-optimal approaches for finding the best reference block rely on spatial sub-sampling. For example, considering that the most probable zone for finding the reference block

Table 2: First order entropy of the prediction residuals and motion vectors (MV) obtained for several values of the block size, N, and search area, Δ.

Δ	N	Entropy			
		Residuals	Nominal MV	MV	Total
3	4	4.29	0.50	0.31	4.60
7	4	4.03	1.00	0.39	4.42
15	4	3.91	2.00	0.47	4.38
3	8	4.63	0.12	0.07	4.70
7	8	4.43	0.25	0.09	4.52
15	8	4.36	0.50	0.10	4.46
3	16	4.84	0.03	0.01	4.85
7	16	4.76	0.06	0.02	4.78
15	16	4.72	0.12	0.02	4.74



Total: 169 blocks Sub-optimal: 65 blocks

Figure 6.12: Example of a sub-optimal search for the reference block that uses a finer search around the original position of the block.

is in the neighborhood of the block, then we may use a scheme as the one shown in Fig. 6.12.

If we consider that after finding a reasonably good reference block it is probable that others better than itself can be found in the neighborhood, then we can use a greedy search, such as the one illustrated in Fig. 6.13.

Many other variants of local search have been proposed, such as, for example, logarithmic search (see Fig. 6.14).

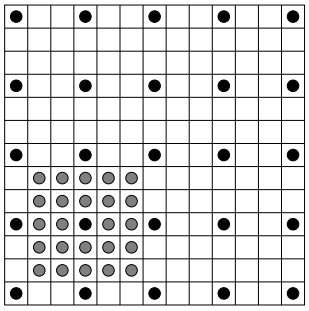
Total: 169 blocks  Sub-optimal: 49 blocks

Figure 6.13: Example of a sub-optimal search for the reference block that uses a finer search around a block found after an initial coarse search.

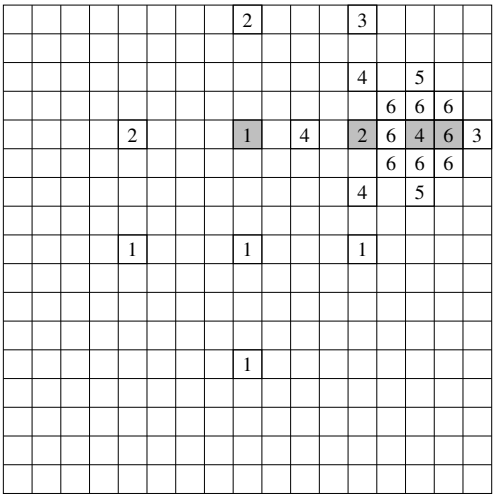


Figure 6.14: Example of a sub-optimal (logarithmic) search for the reference block.

7 An algorithmic measure of information

7.1 Motivation

As shown previously, the formulation of a probabilistic measure of information explores statistical regularities of the data, by means of probabilistic modeling of the information sources. However, there are regularities that cannot be captured by probabilistic models alone. For example, several approaches involving the statistical analysis of the initial digits of π have failed to reveal significant deviations from what is usually understood as statistical **randomness**. Nevertheless, the initial sequence of digits of π can be generated by simple programs, showing that the number π is far from being random! In fact, intuitively, we may say that a string is random if it cannot be **compressed**, i.e., if it is not possible to find a **shorter description** for it. This is precisely the core idea behind the algorithmic measure of information.

Discussion topic 7.1
In data compression, what is the role of the decompressor and of the string representing the compressed data? Does it make sense to consider the decompressor as an "interpreter" for a certain language, formed by all possible compressed strings?

Example 7.1
Consider the following three binary strings:

1. 0101010101010101010101010101010101010101010101010101010101010101

2. 0110101000001001111001100110011111111001110111100110010010000 1000

3. 1101111001110101111101101111101110101101111000101110010100111011

What should be the shortest descriptions for them? The first one seems to be easy to describe. Apparently, the second one looks random, but actually it is just the initial binary representation of $\sqrt{2}-1$. The third one seems random and probably is (however, in general, it is impossible to prove it!).

Now, consider the representation of a sequence generated by flipping a fair coin n times, such as

$$00100101110101010\ldots010$$

There are 2^n such sequences and they are equally probable. It is highly likely that such sequence cannot be compressed at all. Therefore, there might not be a better (shorter) program for generating this sequence as "print $00100101110101010\ldots010$". Thus, the descriptive complexity of a random binary sequence is as long as the sequence itself.

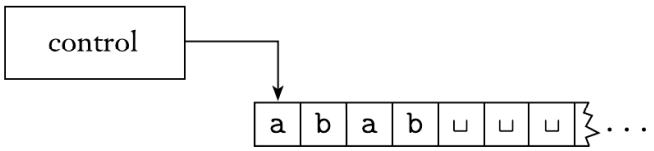


Figure 7.1: Schematic representation of a Turing machine.

7.2 Turing machines

One question that may arise when thinking about computers is: **Can a computer solve any type of problem?** And, if the answer to this question is "no": Which are the problems that cannot be solved by computers?

In fact, according to Nigel J. Cutland (1944–), in his book "*Computability: An introduction to recursive function theory*", Cambridge University Press (Cutland, 1980),

> *"We could describe computability theory, from the viewpoint of computer science, as beginning with the question What can computers do in principle (without restrictions of space, time or money)?—and, by implication—What are their inherent theoretical limitations?"*

Hence, one of the objectives of the field of computability theory is precisely to study which problems are solvable by computers and which are not. However, to achieve this goal, we need first to have a precise definition of what do we mean by a **digital computer**.

First, let us introduce some notation. We define an **alphabet** as a nonempty finite set. We refer to the members of the alphabet as **symbols**. Usually, and without loss of generality, we consider the alphabet $\Sigma = \{0, 1\}$. A **string** over an alphabet is a finite sequence of symbols from that alphabet. For example, if $\Sigma = \{0, 1\}$, then 011101 is a string over the alphabet Σ. If s is a string, then we represent by $|s|$ the length (i.e., number of symbols) of s. We denote by Σ^* the set of all finite-length strings, i.e.,

$$\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, \dots\},$$

where ϵ denotes the empty string, i.e., $|\epsilon| = 0$.

For convenience, we assume an ordering of the strings, such that they appear ordered by size and, for the same size, ordered lexicographically (this is the ordering used to display the set above). Note that one such ordering allows to put all finite-length strings into a one-to-one correspondence with the integers and, hence, to treat strings as integers.

A **language**, A, is a subset of Σ^*, i.e., $A \subseteq \Sigma^*$. A language is **prefix-free** if no string of the language is a proper prefix of another string of the language.

A **Turing machine** is composed of a control based on a finite, non-empty, set of states and access to an unlimited and unrestricted amount of memory (Fig. 7.1). It is a mathematical model (or abstraction) of a hypothetical computing device where:

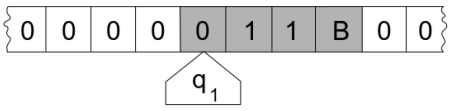


Figure 7.2: Example of a tape and head of a Turing machine. The q_1 written inside the head indicates the current internal state of the machine.

1. At each step, the machine is in one of a predefined finite (non-empty) set of states.
2. Symbols from a predefined finite (non-empty) set can be both read from and written on a tape (see Fig. 7.2, for an example).
3. The read-write head can move both to the left and to the right.
4. The tape is unlimited.
5. At start, the tape is blank, except for some finite number of squares.
6. The machine stops when a special state (the halting state) is encountered or when it fails to find a continuation state.
7. The special states for rejecting and accepting strings of a language, that in the case of Turing machines take effect immediately after halting, even without having exhausting the input. There might be a specific halting state for accepting and another one for rejecting, or just a single halting state and the interpretation of acceptance or rejection left written on the tape.

According to Marvin Minsky (1927-2016), in his book "*Computation: Finite and Infinite Machines*", Prentice-Hall, Inc. (Minsky, 1967):

> *"A Turing machine is a finite-state machine associated with an external storage or memory medium. This medium has the form of a sequence of squares, marked off on a linear tape. The machine is coupled to the tape through a head, which is situated, at each moment, on some square of the tape (Fig. 7.2). The head has three functions, all of which are exercised in each operation cycle of the finite-state machine. These functions are: reading the square of the tape being "scanned," writing on the scanned square, and moving the machine to an adjacent square (which becomes the scanned square in the next operation cycle)."*

A Turing machine equipped with accepting and rejecting states, started with a certain string written on the tape, may have one of three possible behaviours: it stops at an accepting state; it stops at a rejecting state; it loops forever.

Definition 7.1
*A language is **Turing-recognizable / recursively enumerable** if some Turing machine recognizes it.*

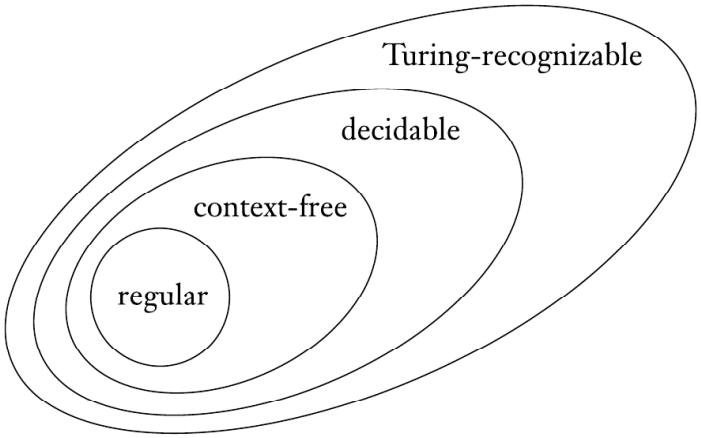


Figure 7.3: Hierarchy of the classes of languages (Sipser, 2012).

This means that a Turing machine recognizes a language if it accepts every string in the language and rejects or does not halt on the strings not in the language.

Definition 7.2
*A language is **Turing-decidable / decidable / recursive**, if some Turing machine decides it.*

This means that a Turing machine decides a language if it accepts every string in the language and rejects every string not in the language. Figure 7.3 shows the hierarchical relation among languages.

There are many variants of Turing machines, for example, using more than one tape/head, using one-way or two-way unlimited tapes, using alphabets of different sizes and with different symbols, and even using nondeterminism. However, all of them share the same computational power.

In fact, there are a number of remarkable facts about Turing machines, some of which are the following:

- A Turing machine can be restricted to a two-symbol alphabet, without losing any of its computational power. For example, suppose that a certain Turing machine T uses k different symbols and that $n = \lceil \log k \rceil$. Then, we may construct another Turing machine, T', replacing each symbol of T by its n-bit binary representation and regarding the tape of T' as a sequence of n-bit blocks. Of course, to attain equivalence between T and T' it is also necessary to change appropriately the finite-state machine of T' (which will necessarily have more states than machine T).

- It can also be shown that Turing machines which have tapes that are unlimited in both directions have the same computing power as machines having an unlimited tape in only one direction.
- It is neither an advantage (nor a disadvantage) to have multiple tapes in a Turing machine—it can always be converted into an equivalent single tape machine.
- Probably more surprising is the fact that any Turing machine is equivalent to a Turing machine with only two internal states. Of course, this can only be achieved by enlarging the alphabet of the tape symbols.

The idea behind a Turing machine was first proposed by Alan Turing (1912-1954), in his famous paper:

> Alan Turing, "*On Computable Numbers, with an Application to the Entsgheidungsproblem*", Proceedings of the London Mathematical Society (Turing, 1936).

Almost simultaneously, Alonzo Church (1903-1995) proposed a different, but equivalent, notion of computability, using what became known as λ-calculus:

> Alonzo Church, "*An Unsolvable Problem of Elementary Number Theory*", American Journal of Mathematics (Church, 1936).

In their attempt to solve some problems related to the foundations of mathematics, both had the need to rigorously defining the concept of "computation". Note that, at that time, a "computer" was always seen as a human computer, i.e., someone that performs pencil-and-paper calculations.

Basically, we say that something is **computable** if it can be produced, in finite time, by a process equivalent to our usual intuitive notion of **algorithm** or **effective procedure**. This definition was proposed by Alan Turing in his 1936 paper, where he also refers as equivalent the idea of "effective calculability" of Alonzo Church.

Again, from Minsky (1967):

> *"The idea of an algorithm or effective procedure arises whenever we are presented with a set of instructions about how to behave. This happens when, in the course of working on a problem, we discover that a certain procedure, if properly carried out, will end up giving us the answer. Once we make such a discovery, the task of finding the solution is reduced from a matter of intellectual discovery to a mere matter of effort; of carrying out the discovered procedure—obeying the specified instructions."*

This notion of algorithm motivated a conjecture, known as the **Church-Turing thesis**, stating that something is computable by a human being following an algorithm (of course, ignoring

resources, such as time, number of sheets of paper and pencils), if and only if it is computable by a Turing machine. There is no proof for this claim. However, 80 years have passed since the papers of Church and Turing and no counterexample has yet been found!

Example 7.2
Consider the following example of a Turing machine, taken from the original paper of Turing (Turing, 1936). The set of instructions for the machine is:

Configuration		*Behaviour*	
m-config.	*symbol*	*operations*	*final m-config.*
b	*None*	*P0, R*	*c*
c	*None*	*R*	*e*
e	*None*	*P1, R*	*f*
f	*None*	*R*	*b*

If started on an empty tape, the machine writes alternating 0's and 1's on the tape, leaving a blank square between each pair of digits.

Problem 7.1
What happens if, in the previous example, the tape is not totally blank?

One of the most important results included in Turing's paper (Turing, 1936) shows that it is possible to build **universal Turing machines**. A universal Turing machine is a machine that can simulate any other Turing machine. A direct consequence of this discovery is the notion of stored-program computer. All but the most trivial computers are universal, in the sense that they can mimic the actions of other computers. Nevertheless, it is possible to construct a universal standard Turing machine using 4 states and 6 symbols. Moreover, it was recently suggested (although still debated) that a universal Turing machine without a halting state is possible using 2 states and 3 symbols!

One of the most famous problems in computability theory, that cannot be systematically solved by a computer, is to decide if another program, `prog`, will halt for a certain input `x`. Next we show that such program cannot exist.

Let us consider that such program, `will_halt(prog,x)`, exists. Then, consider a program `prog`, such that

```
prog(x)
    if(will_halt(x,x) == true)
        while(true)
    else
        exit
```

Now, consider what happens if `prog` is called, having as argument a description of itself, i.e., what happens when executing `prog(prog)`. Notice that, inside `prog`, there is a call to `will_halt(prog,prog)`. If this call returns `true`, i.e., if our hypothetical program that decides if a program will halt for a certain input says that it will halt, then `prog` enters an infinite loop and does not halt. On the contrary, if `will_halt(prog,prog)` returns `false`, i.e., if it indicates that `prog` will not stop when called with itself as argument, then it stops. This contradiction shows that it is not possible to build a program that, for all possible cases, verifies if some other program halts or not.

7.3 Kolmogorov complexity (or algorithmic entropy)

Suppose that we want to represent a certain object by a finite string of bits. Generally, an object may have several possible representations (or descriptions), but each representation should describe only one object (we want to unambiguously reconstruct the object from its description). Then, the length of the shortest of all descriptions available for the object can be used as a measure of its **complexity** (and, in fact, of its randomness).

However, the meaning of "**description**" must be defined precisely or otherwise we may run into similar problems as the one known as the **Berry paradox**, where a certain number is defined as "*the smallest positive integer not definable in fewer than twelve words*". If this number exists, then we have just described it in eleven words, which contradicts its definition! On the other hand, if the number does not exist, then we have to conclude that all positive integers can be described in less than twelve words!

The approach that follows relies on algorithms to measure the quantity of information conveyed by a certain string, x. One way to address this problem could involve building a (compact) Turing machine, T_x, that, when started on a blank tape, would write the string x on the tape and halt. Then, the description of x would be a string describing T_x, $\langle T_x \rangle$, using some fixed encoding for the Turing machines and, therefore, the length of the description would be $|\langle T_x \rangle|$.

However, instead of starting the machine on a blank tape, another possible approach is using an auxiliary string written on it. In this case, the string x is described using a Turing machine M and an input w that is used by M, and the description of x is given by some encoding $\langle M, w \rangle$ (note that, in principle, this encoding needs to include information that permits separating $\langle M \rangle$ from w).

Assume that a certain specification method, f, associates at most an object, x, with a description, p. We can think of f as a decompression function (or program), from a set of descriptions, P, into the set of objects, X, i.e., $f : P \to X$. To be useful, descriptions should be finite. This implies that **it is only possible to describe a countable number of different objects**.

For most functions f, the length of the description of object x according to f depends not only on x, but also on f. However, in order to objectively assess the complexity of the objects, we need functions that assign a complexity value to x that depends on x alone. Moreover, if p is the description of x with respect to f, then x has to be generated from f and p through "**an effective procedure**" or, in other words, f has to be a computable function.

Definition 7.3 (Algorithmic complexity)
Given a computable function, f, we define the algorithmic complexity of an object,[16] *x, according to f, as*

$$K_f(x) = \min\{|p| : f(p) = x\},$$

where $x, p \in \{0,1\}^*$, i.e., it corresponds to the minimum description length, over all descriptions that produce x through f. If there is no p able to produce x using f, then we say that $K_f(x) = \infty$ for such x. Therefore, although computable, f is not required to be a total function.

Notice that we can think of p as a program and f as a (specialized) computer. If $K_f(x) = \infty$, then it is not possible to produce x with the computer f. Also, notice that, without the computability requirement, f could be any arbitrary partial function from strings to strings, complying only to the condition that $|\{x : K_f(x) = k\}| \leq 2^k$, where k-bit strings are used as the description of strings x for which $K_f(x) = k$. Of course, this would allow us to trivially and arbitrarily find, for a certain x, a f such that $K_f(x) = 0$ (by mapping the empty string to it)!

Consider now r distinct specification methods, $f_1, f_2, \ldots, f_r$. It is easy to construct a new method, f, that assigns to each x a complexity $K_f(x)$ that exceeds only by about $\log r$ bits the minimum of $K_{f_1}(x), K_{f_2}(x), \ldots, K_{f_r}(x)$. The idea is to use the first $\log r$ bits of p to identify the method f_i and then the appropriate program for f_i. Hence, we can say that

$$K_f(x) \leq K_{f_i}(x) + c, \forall i,$$

i.e., that the description method f **minorizes** (additively) every description method f_i.

The development of the theory of algorithmic information (also known as **Kolmogorov complexity**) is made possible by the remarkable fact that the class of computable functions possesses a universal element that is additively optimal. Under this relatively natural restriction on the class of description methods (i.e., computability), we obtain a well-behaved hierarchy of complexities and hence a minimal element.

Definition 7.4 (Conditional algorithmic complexity)
Let $x, y, p \in \mathbb{N}$ (recall that this is the same as considering $x, y, p \in \{0,1\}^*$, according to the usual enumeration of strings). Any partial recursive function ϕ, together with p and y, such that $\phi(\langle y, p\rangle) = x$, is a description of x. The complexity K_ϕ of x conditioned to y is defined by

$$K_\phi(x|y) = \min\{|p| : \phi(\langle y, p\rangle) = x\},$$

and $K_\phi(x|y) = \infty$ if there are no such p. Then, p is a program to compute x by ϕ, given y.

[16] Without loss of generality, from now on we will refer to these objects as strings and also consider that they are binary, i.e., $x, p \in \{0,1\}^*$. Also, as referred before, we assume the usual string ordering, $\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \ldots$ and the implicit one-to-one correspondence between binary strings and non-negative integers (for example, the natural number "5" has the same meaning as the string "10").

The key point of this theory is not that the universal description method necessarily gives the shortest description in each case, but instead that no other description method can improve on it infinitely often by more than a fixed constant.

Definition 7.5 (Kolmogorov complexity)
Let us fix an additively optimal universal ϕ_0 (the reference function) and define the conditional Kolmogorov complexity $K(x|y) = K_{\phi_0}(x|y)$. Also, fix a particular Turing machine (the reference machine), U, that computes ϕ_0. The unconditional Kolmogorov complexity is defined by

$$K(x) = K(x|\epsilon).$$

Theorem 7.1
For every computable function, f, there exists a constant, c, such that $K(f(x)) \leq K(x) + c$, for every x such that $f(x)$ is defined.

Theorem 7.2 (Non-computability of $K(x)$)
The Kolmogorov complexity is not computable.

To show why the Kolmogorov complexity is not a computable function, consider the following pseudo-code, that calls a hypothetical function "`K`", that calculates the Kolmogorov complexity of the string in the nth position of the usual string ordering, $K(n)$:

```
print_complex_string(min_k)
    n = 0
    while(true)
        if(K(n) > min_k)
            print(string(n))
            exit
        else
            n++
```

Obviously, for a value of "`min_k`" greater than the total size of the programs involved, the program would print a string with Kolmogorov complexity greater than the size of the program used to generate it, which is a contradiction! Therefore, it is not possible to have a program that calculates the Kolmogorov complexity of arbitrary strings.

Theorem 7.3 (Rice's theorem)
All non-trivial semantic properties of programs are undecidable. A non-trivial property is one which is neither true for every program, nor false for every program.

References

Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*, **58**(2), 345–363.

Cutland, N. J. (1980). *Computability: An introduction to recursive function theory*. Cambridge University Press.

Golomb, S. (1966). Run-length encodings. *IEEE Trans. on Information Theory*, **12**(3), 399–401.

Hamming, R. (1950). Error detecting and error correcting codes. *Bell System Technical Journal*, **29**(2), 147–160.

Hartley, R. V. L. (1928). Transmission of information. *Bell System Technical Journal*, **7**(3), 535–563.

Huffman, D. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, **40**(9), 1098–1101.

Minsky, M. L. (1967). *Computation: Finite and infinite machines*. Prentice-Hall, Inc., Englewood Cliffs, NJ.

Nyquist, H. (1924). Certain factors affecting telegraph speed. *Trans. of the American Institute of Electrical Engineers*, **XLIII**, 412–422.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, **27**, 379–423,623–656.

Shannon, C. E. (1951). Prediction and entropy of printed English. *Bell Labs Technical Journal*, **30**(1), 50–64.

Sipser, M. (2012). *Introduction to the theory of computation*. Cengage Learning, 3rd edition.

Storer, J. A. and Szymanski, T. G. (1982). Data compression via textual substitution. *Journal of the ACM*, **29**(4), 928–951.

Turing, A. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, **42**(2), 230–265.

Welch, T. A. (1984). A technique for high-performance data compression. *IEEE Computer*, **17**(6), 8–19.

Ziv, J. and Lempel, A. (1977). A universal algorithm for sequential data compression. *IEEE Trans. on Information Theory*, **23**, 337–343.

Ziv, J. and Lempel, A. (1978). Compression of individual sequences via variable-rate coding. *IEEE Trans. on Information Theory*, **24**(5), 530–536.